ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



Họ và tên sinh viên: Đào Duy Hưng

Mã số sinh viên: 3122410158

LAB03 LOCAL SEARCH

Môn học : **Trí tuệ nhân tạo nâng cao**

Giảng viên : **ThS. Đỗ Như Tài**

TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1: Tìm kiếm cục bộ trên me cung - Simple hill climbing.....2
1.1. Giới thiệu bài toán tìm kiếm cục bộ trên mê cung.....2
1.1.1. Mục tiêu bài thực hành.....2
1.1.2. Biểu diễn mê cung và trạng thái tìm kiếm.....2
1.2. Mô hình hóa bài toán tìm kiếm.....3
1.2.1. Trạng thái bắt đầu và trạng thái đích.....3
1.2.2. Tập hành động và hàm sinh trạng thái lân cận.....3
1.2.3. Hàm đánh giá khoảng cách Manhattan.....5
1.3. Thuật toán Simple Hill Climbing.....5
1.3.1. Nguyên lý hoạt động của thuật toán.....5
1.3.2. Cơ chế lựa chọn trạng thái lân cận.....6
1.3.3. Triển khai thuật toán bằng Python.....6
1.4. Kết quả thực nghiệm.....7
1.4.1. Kết quả tìm kiếm trên L_maze.....7
1.4.2. Trực quan hóa quá trình tìm kiếm bằng animation.....9
1.4.3. Ảnh hưởng của thứ tự hướng di chuyển và ngẫu nhiên hóa.....10
1.5. Nhận xét.....11
CHƯƠNG 2: Mô hình hoá bài toán N quân hậu.....12
2.1. Giới thiệu bài toán n-Queens.....12
2.1.1. Phát biểu bài toán và mục tiêu tối ưu.....12
2.1.2. Biểu diễn trạng thái bằng vector n phần tử.....13
2.2. Xây dựng các thành phần của bài toán.....14
2.2.1. Hàm khởi tạo bàn cờ ngẫu nhiên.....14
2.2.2. Hàm hiển thị trạng thái bàn cờ.....15
2.2.3. Hàm mục tiêu đếm số cặp quân hậu xung đột.....15
2.3. Xác định không gian trạng thái lân cận.....16
2.3.1. Phép di chuyển một quân hậu trong cùng cột.....16
2.3.2. Công thức xác định số trạng thái lân cận.....17
2.3.3. Điều kiện xác định nghiệm hợp lệ.....18
2.4. Kết quả thực nghiệm trên n-Queens.....18
2.4.1. Khảo sát bàn cờ ngẫu nhiên với n = 4.....19
2.4.2. Khảo sát bàn cờ ngẫu nhiên với n = 8.....20
2.4.3. So sánh với cấu hình không xung đột [1, 3, 0, 2].....21

2.5. Nhận xét..... 23
CHƯƠNG 3: Steepest-ascent và Stochastic hill climbing.....25
3.1. Cơ sở lý thuyết các thuật toán Hill Climbing.....25
3.1.1. Thuật toán Steepest-ascent Hill Climbing..... 25
3.1.2. Thuật toán Stochastic Hill Climbing.....26
3.1.3. Thuật toán First-choice Hill Climbing.....26
3.2. Triển khai thuật toán trên bài toán n-Queens..... 27
3.2.1. Triển khai Steepest-ascent – Task 1..... 27
3.2.2. Triển khai Inefficient Stochastic – Task 2.....28
3.2.3. Triển khai Efficient Stochastic – Task 3..... 29
3.3. Thiết lập thực nghiệm và phương pháp đánh giá.....29
3.3.1. Dữ liệu khởi tạo và random seed..... 29
3.3.2. Các chỉ số đánh giá hiệu suất thuật toán..... 30
3.4. Kết quả thực nghiệm..... 31
3.4.1. Kết quả thực nghiệm với n = 4.....31
3.4.2. Kết quả thực nghiệm với n = 8.....32
3.4.3. So sánh tỷ lệ thành công và chi phí tính toán.....33
3.4.4. Trực quan hóa quá trình hội tụ.....35
3.5. Nhận xét..... 36
CHƯƠNG 4: Random forest và Stimulated annealing.....38
4.1. Hạn chế của các phương pháp Hill Climbing..... 38
4.1.1. Hiện tượng cực trị cục bộ (Local Optimum).....38
4.1.2. Hiện tượng Plateau và Ridge..... 39
4.2. Thuật toán Random Restart.....40
4.2.1. Nguyên lý khởi tạo lại trạng thái ngẫu nhiên.....40
4.2.2. Triển khai Random Restart cho các biến thể Hill Climbing..... 41
4.2.3. Giới hạn số lần khởi động và lưu nghiệm tốt nhất.....41
4.3. Thuật toán Simulated Annealing.....42
4.3.1. Nguyên lý hoạt động của Simulated Annealing.....42
4.3.2. Xác suất chấp nhận trạng thái xấu hơn..... 43
4.3.3. Vai trò của nhiệt độ và tốc độ làm nguội.....44
4.3.4. Triển khai Simulated Annealing trên n-Queens.....44
4.4. Kết quả thực nghiệm..... 45
4.4.1. Kết quả Random Restart trên n = 4 và n = 8.....45

4.4.2. Kết quả Simulated Annealing trên n = 4 và n = 8.....47
4.4.3. Khảo sát nhiệt độ ban đầu và tốc độ làm nguội.....48
4.4.4. So sánh tỷ lệ thành công, số lần đánh giá và thời gian thực thi.....50
4.5. Nhận xét.....51
CHƯƠNG 5: Khảo sát lịch giảm nhiệt độ.....54
5.1. Vai trò của lịch giảm nhiệt độ trong Simulated Annealing.....54
5.2. Các phương pháp giảm nhiệt độ.....55
5.2.1. Lịch giảm nhiệt hàm mũ (Exponential).....55
5.2.2. Lịch giảm nhiệt tuyến tính (Linear).....55
5.2.3. Lịch giảm nhiệt nhanh (Fast).....56
5.2.4. Lịch giảm nhiệt Logarithmic.....57
5.3. Khảo sát ảnh hưởng của tham số.....57
5.3.1. Ảnh hưởng của nhiệt độ ban đầu T_0.....57
5.3.2. Ảnh hưởng của hệ số alpha.....58
5.3.3. Ảnh hưởng của hệ số eta.....58
5.3.4. So sánh xác suất chấp nhận bước xấu theo thời gian.....59
5.4. Kết quả thực nghiệm.....60
5.4.1. Biểu đồ so sánh bốn lịch giảm nhiệt độ.....60
5.4.2. So sánh chất lượng nghiệm khi áp dụng các lịch nhiệt cho n-Queens.62
5.4.3. So sánh tỷ lệ thành công và thời gian thực thi.....63
5.4.4. Trực quan hóa quá trình hội tụ.....64
5.5. Nhận xét.....65
CHƯƠNG 6: Tối ưu hàm đếm xung đột bằng Numba.....68
6.1. Bài toán tối ưu hiệu suất tính toán.....68
6.1.1. Vai trò của hàm đếm xung đột trong n-Queens.....68
6.1.2. Hạn chế của phương pháp đếm xung đột trực tiếp.....69
6.2. Các phương pháp tính số cặp xung đột.....69
6.2.1. Phương pháp Naive với độ phức tạp $O(n^2)$.....69
6.2.2. Phương pháp đếm theo hàng và đường chéo với O(n).....70
6.2.3. Áp dụng công thức tổ hợp C(k, 2).....70
6.2.4. Tối ưu thực thi bằng Numba JIT.....71
6.3. Thiết lập thực nghiệm.....72
6.3.1. Các kích thước bàn cờ được khảo sát.....72
6.3.2. Phương pháp đo thời gian thực thi.....72

6.3.3. Loại trừ thời gian biên dịch JIT lần đầu....73
6.4. Kết quả thực nghiệm....73
6.4.1. Kiểm chứng tính đúng đắn của ba hàm đếm xung đột....73
6.4.2. So sánh thời gian thực thi theo kích thước bàn cờ....74
6.4.3. Biểu đồ log-log thời gian thực thi....76
6.4.4. Phân tích hệ số tăng tốc khi sử dụng Numba....77
6.5. Nhận xét....79
CHƯƠNG 7: Tìm kiếm cục bộ cho Traveling salesman problem (TSP)..... 81
7.1. Giới thiệu bài toán Traveling Salesman Problem....81
7.1.1. Phát biểu bài toán và mục tiêu tối ưu....81
7.1.2. Biểu diễn trạng thái bằng hoán vị các thành phố....81
7.1.3. Hàm mục tiêu tính tổng độ dài chu trình khép kín....82
7.2. Mô hình hóa bài toán tìm kiếm cục bộ....83
7.2.1. Khởi tạo tập thành phố và đường đi ngẫu nhiên....83
7.2.2. Toán tử lân cận hoán đổi hai thành phố (Swap)....83
7.2.3. Điều kiện hợp lệ của một đường tour....84
7.3. Triển khai các thuật toán tìm kiếm cục bộ....84
7.3.1. Steepest-ascent Hill Climbing....84
7.3.2. Stochastic Hill Climbing....85
7.3.3. First-choice Hill Climbing....86
7.3.4. Random Restarts....86
7.3.5. Simulated Annealing....87
7.4. Kết quả thực nghiệm....87
7.4.1. Thiết lập dữ liệu và các kích thước TSP....87
7.4.2. So sánh độ dài tour và tỷ lệ cải thiện....88
7.4.3. So sánh số lần đánh giá và thời gian thực thi....91
7.4.4. Khảo sát hiệu suất khi tăng số thành phố....93
7.4.5. Trực quan hóa đường tour trước và sau tối ưu....95
7.5. Nhận xét....96
CHƯƠNG 8: Tổng hợp thí nghiệm và phân tích..... 99
8.1. Mục tiêu và phạm vi thực nghiệm tổng hợp....99
8.1.1. Các thuật toán được lựa chọn để so sánh....99
8.1.2. Điều kiện thực nghiệm và thiết lập random seed....100
8.1.3. Các chỉ số đánh giá....100

8.2. So sánh thuật toán trên bài toán n-Queens..... 101
8.2.1. Kết quả tổng hợp trên n = 4..... 101
8.2.2. Kết quả tổng hợp trên n = 8..... 102
8.2.3. So sánh tỷ lệ thành công và giá trị hàm mục tiêu..... 103
8.2.4. So sánh thời gian và số lần đánh giá hàm mục tiêu..... 105
8.3. Phân tích quá trình hội tụ..... 105
8.3.1. Biểu đồ hội tụ của các thuật toán..... 105
8.3.2. Khả năng cải thiện trong giai đoạn đầu.....107
8.3.3. Hiện tượng dừng sớm tại cực trị cục bộ.....108
8.4. Khảo sát khả năng mở rộng.....108
8.4.1. Thiết lập thực nghiệm với n = 4, 8, 12, 16, 20.....108
8.4.2. Kết quả thời gian thực thi theo kích thước bài toán.....109
8.4.3. Phân tích đồ thị log-log..... 110
8.4.4. Đánh giá khả năng mở rộng của các thuật toán..... 110
8.5. Phần thực nghiệm nâng cao về toán tử lân cận..... 111
8.5.1. Các toán tử di chuyển được khảo sát.....111
8.5.2. Ảnh hưởng đến kích thước không gian lân cận..... 112
8.5.3. So sánh tỷ lệ thành công và chất lượng nghiệm.....113
8.5.4. Nhận xét về hiệu quả của từng toán tử.....114
8.6. Đánh giá tổng hợp các phương pháp tìm kiếm cục bộ..... 115
8.6.1. Hiện tượng Local Optimum, Plateau và Ridge..... 115
8.6.2. Hiệu quả của Random Restart và Simulated Annealing..... 115
8.6.3. Sự đánh đổi giữa chất lượng nghiệm và chi phí tính toán..... 116
8.6.4. Thời gian thực thi và khả năng mở rộng..... 116
8.6.5. Hạn chế của các thuật toán và điều kiện áp dụng..... 116
8.7. Kết luận bài thực hành tổng hợp..... 117

LỜI MỞ ĐẦU

Trong bối cảnh trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong học tập, nghiên cứu và thực tiễn, việc hiểu rõ các nguyên lý, phương pháp và cách xây dựng hệ thống trí tuệ nhân tạo là nền tảng cần thiết đối với sinh viên theo định hướng công nghệ. Môn học Trí tuệ nhân tạo nâng cao giúp người học tiếp cận các kiến thức và vấn đề cốt lõi của lĩnh vực AI, đồng thời rèn luyện khả năng phân tích, tư duy logic và vận dụng kiến thức vào những bài toán thực tế.

Assignment này được thực hiện nhằm củng cố và hệ thống hóa những kiến thức đã được học thông qua việc nghiên cứu, phân tích và trả lời các câu hỏi theo từng chủ đề. Thay vì chỉ tiếp cận kiến thức ở mức lý thuyết, quá trình thực hiện bài tập giúp tôi hiểu rõ hơn cách các khái niệm AI được hình thành, cách chúng liên kết với nhau và cách có thể áp dụng vào các bài toán cụ thể.

Trong quá trình thực hiện, tôi tập trung vào việc tự tìm hiểu nội dung, đối chiếu với tài liệu môn học, phân tích vấn đề và trình bày kết quả theo hướng rõ ràng, có lập luận. Qua đó, tôi không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện khả năng nghiên cứu tài liệu, giải quyết vấn đề và đánh giá một hệ thống AI dưới nhiều góc độ khác nhau.

Thông qua assignment này và các bài tập tiếp theo, tôi kỳ vọng có thể xây dựng nền tảng kiến thức vững chắc hơn về Trí tuệ nhân tạo, đồng thời từng bước nâng cao khả năng vận dụng kiến thức vào các bài toán thực tế. Đây cũng là cơ sở để tôi tiếp tục phát triển các kỹ năng chuyên môn và nghiên cứu sâu hơn trong lĩnh vực AI trong quá trình học tập và định hướng nghề nghiệp sa

CHƯƠNG 1: Tìm kiếm cục bộ trên me cung - Simple hill climbing

1.1. Giới thiệu bài toán tìm kiếm cục bộ trên mê cung

1.1.1. Mục tiêu bài thực hành

Bài thực hành nhằm tìm hiểu nguyên lý tìm kiếm cục bộ (Local Search) thông qua bài toán di chuyển trong mê cung. Khác với tìm kiếm theo đường đi như BFS hoặc A*, phương pháp Hill Climbing tập trung lựa chọn trạng thái lân cận có giá trị hàm đánh giá tốt hơn trạng thái hiện tại mà không cần xây dựng trước toàn bộ cây tìm kiếm. Qua đó, bài thực hành làm rõ ưu điểm về cách tổ chức đơn giản và hạn chế khi thuật toán chỉ tối ưu theo thông tin cục bộ.

Dựa trên notebook Maze_Example_LocalSearch.ipynb, các nội dung khảo sát gồm đọc và hiển thị dữ liệu mê cung, xác định hành động di chuyển N, E, S, W, xây dựng tập lân cận bằng hàm expand(), tính khoảng cách Manhattan và theo dõi hoạt động của local_search(). Bài thực hành còn thực hiện animation, thay đổi thứ tự xét hướng và chạy nhiều seed để phân tích nguyên nhân thuật toán dừng trước khi đạt đích.

1.1.2. Biểu diễn mê cung và trạng thái tìm kiếm

Mê cung được lưu trong tệp L_maze.txt dưới dạng ma trận ký tự có kích thước 15 × 15. Trong đó, X biểu diễn tường, khoảng trắng là ô có thể di chuyển, S là vị trí xuất phát và G là vị trí đích. Hàm parse_maze() trong mô-đun maze_helper chuyển

dữ liệu văn bản thành mảng NumPy hai chiều; hàm show_maze() hiển thị vị trí các ô trên bản đồ để quan sát cấu trúc hành lang và vật cản.

Mỗi trạng thái được biểu diễn bởi một cặp tọa độ (r, c), trong đó r là chỉ số hàng và c là chỉ số cột, cùng bắt đầu từ 0 theo quy ước của NumPy. Mỗi hành động hợp lệ làm thay đổi một tọa độ đúng một đơn vị. Trong quá trình tìm kiếm, dữ liệu mê cung gốc được giữ nguyên; các dấu đường đi và điểm dừng được tạo trên bản sao phục vụ trực quan hóa.

1.2. Mô hình hóa bài toán tìm kiếm

1.2.1. Trạng thái bắt đầu và trạng thái đích

Từ dữ liệu L_maze, hàm find_pos() xác định trạng thái bắt đầu S = (9, 5) và trạng thái đích G = (2, 12). Mục tiêu của tác tử là chuyển từ S đến G qua những ô trống mà không đi xuyên tường. Điều kiện hoàn thành bài toán là trạng thái hiện tại trùng với tọa độ đích.

Cần phân biệt mục tiêu thực sự là đến ô G và mục tiêu đánh giá cục bộ là làm giảm khoảng cách Manhattan. Khi hai mục tiêu không tương thích do xuất hiện vật cản, một trạng thái có khoảng cách Manhattan nhỏ hơn chưa chắc tạo ra đường đi tiến gần nghiệm theo nghĩa có thể tiếp tục đến đích.

1.2.2. Tập hành động và hàm sinh trạng thái lân cận

Trong notebook, từ điển DIR2POS xác định bốn hướng di chuyển. Hàm set_order() thiết lập thứ tự xét các hướng, còn expand(maze, pos) trả về những cặp gồm ký hiệu hành động và tọa độ kế tiếp đối với các ô không phải tường. Khi bật

RANDOMIZATION, danh sách lân cận được xáo trộn trước khi lựa chọn bước di chuyển.

Bảng 1.1. Quy ước các hành động di chuyển

Hành động	Tên hướng	Độ thay đổi (Δr, Δc)
N	Lên	(−1, 0)
E	Phải	(0, 1)
S	Xuống	(1, 0)
W	Trái	(0, −1)

Chẳng hạn tại S = (9, 5), cả bốn hướng đều hợp lệ. Hai trạng thái lân cận (8, 5) và (9, 6) có khoảng cách Manhattan đến G bằng 13, tốt hơn giá trị hiện tại là 14; trong khi hai hướng còn lại cho giá trị 15. Việc có nhiều trạng thái cùng mức cải thiện cho thấy thứ tự xét hướng có thể quyết định lựa chọn khi các giá trị đánh giá bằng nhau.

1.2.3. Hàm đánh giá khoảng cách Manhattan

Hàm manhattan(pos1, pos2) trong notebook tính tổng chênh lệch tuyệt đối của chỉ số hàng và cột giữa hai trạng thái. Với trạng thái n = (r□, c□) và đích G = (rG, cG), hàm đánh giá được xác định theo công thức:

$$h(n) = |r_n - r_G| + |c_n - c_G|$$

Bài toán được xem là tối thiểu hóa h(n). Một bước đi được chấp nhận khi h(n_mới) nhỏ hơn h(n_hiện_tại).

Tại S = (9, 5), giá trị ban đầu bằng: $h(S) = |9 - 2| + |5 - 12| = 14$

Hàm này chỉ phụ thuộc tọa độ, không xét trực tiếp vị trí tường; do đó giá trị giảm theo h(n) không bảo đảm rằng thuật toán đang đi trên đường đến đích.

1.3. Thuật toán Simple Hill Climbing

1.3.1. Nguyên lý hoạt động của thuật toán

Hill Climbing bắt đầu từ trạng thái S, đánh giá các trạng thái lân cận và chỉ chuyển sang một trạng thái có giá trị hàm mục tiêu tốt hơn. Sau mỗi lần di chuyển, trạng thái mới được sử dụng làm điểm xuất phát cho bước tiếp theo. Quá trình kết thúc khi không còn lân cận nào làm giảm giá trị h(n) hoặc khi đã đạt mục tiêu, nếu chương trình có kiểm tra điều kiện đích trực tiếp.

Do chỉ chấp nhận các bước giảm hàm đánh giá, thuật toán có tính tham lam (greedy). Trong các môi trường có vật cản, thuật toán có thể gặp cực tiểu cục bộ:

trạng thái hiện tại tốt hơn mọi trạng thái lân cận xét theo h(n), nhưng vẫn chưa phải đích. Để vượt tường và đến G, tác tử đôi khi cần tạm thời chấp nhận bước làm tăng khoảng cách Manhattan; quy tắc của Hill Climbing trong notebook không cho phép điều đó.

1.3.2. Cơ chế lựa chọn trạng thái lân cận

Theo mô tả lý thuyết trong tài liệu Lab 03, Simple Hill Climbing chọn bước cải thiện đầu tiên theo thứ tự xét. Tuy nhiên, hàm local_search() của notebook gốc thực hiện tính h(n) cho toàn bộ lân cận, xác định giá trị nhỏ nhất bằng min(obj), sau đó chọn vị trí tương ứng.

Như vậy, chương trình thực tế có cách lựa chọn gần với best-improvement hay Steepest-ascent Hill Climbing theo chiều tối thiểu hóa, không hoàn toàn là first-improvement.

Khi nhiều lân cận cùng đạt giá trị h nhỏ nhất, chương trình chọn lân cận xuất hiện đầu tiên trong danh sách. Vì vậy, thứ tự ORDER hoặc chế độ RANDOMIZATION có thể ảnh hưởng đến lựa chọn trong trường hợp hòa, nhưng không làm thay đổi giá trị đánh giá của từng trạng thái hay cho phép thuật toán đi qua bước xấu hơn.

1.3.3. Triển khai thuật toán bằng Python

Các thành phần chính của chương trình gồm set_order() để thiết lập thứ tự và chế độ xáo trộn; expand() để sinh các trạng thái kế tiếp hợp lệ; manhattan() để tính hàm mục tiêu; và local_search(maze, vis, animation, verbose) để thực hiện tìm kiếm.

Trong mỗi vòng lặp, chương trình tạo danh sách neighbors, tính obj cho các lân cận, xác định best_obj và chỉ cập nhật current nếu best_obj nhỏ hơn current_obj.

Hàm local_search() trả về một từ điển có các khóa solution, path, reached và maze_anim. Trong cài đặt gốc, solution là tọa độ cuối cùng nơi thuật toán dừng, không mặc nhiên là một nghiệm đã đến G; path và reached được đặt bằng None.

Khi animation=True, các khung hình di chuyển được ghi vào maze_anim. Tùy chọn verbose=True in tọa độ hiện tại, giá trị heuristic và các lân cận để theo dõi; tham số vis tồn tại trong chữ ký hàm nhưng chưa được sử dụng trong phần thân hàm gốc.

Do đó, việc đánh giá thuật toán có hoàn thành mục tiêu hay không phải dựa trên phép so sánh trực tiếp solution với G, thay vì chỉ dựa vào sự tồn tại của giá trị solution hoặc thông báo từ công cụ animation.

1.4. Kết quả thực nghiệm

1.4.1. Kết quả tìm kiếm trên L_maze

Với thứ tự hướng NESW và không xáo trộn, thuật toán bắt đầu từ S = (9, 5) với h(S) = 14. Sau tám bước di chuyển, trạng thái cuối cùng là (5, 9), có h = 6 và khác vị trí đích G = (2, 12).

Dãy các trạng thái đã chấp nhận là:

(9, 5) → (8, 5) → (7, 5) → (6, 5) → (5, 5) → (5, 6) → (5, 7) → (5, 8) → (5, 9).

Bảng 1.2. Thông số kết quả tìm kiếm trên L_maze

Chỉ số	**Kết quả**

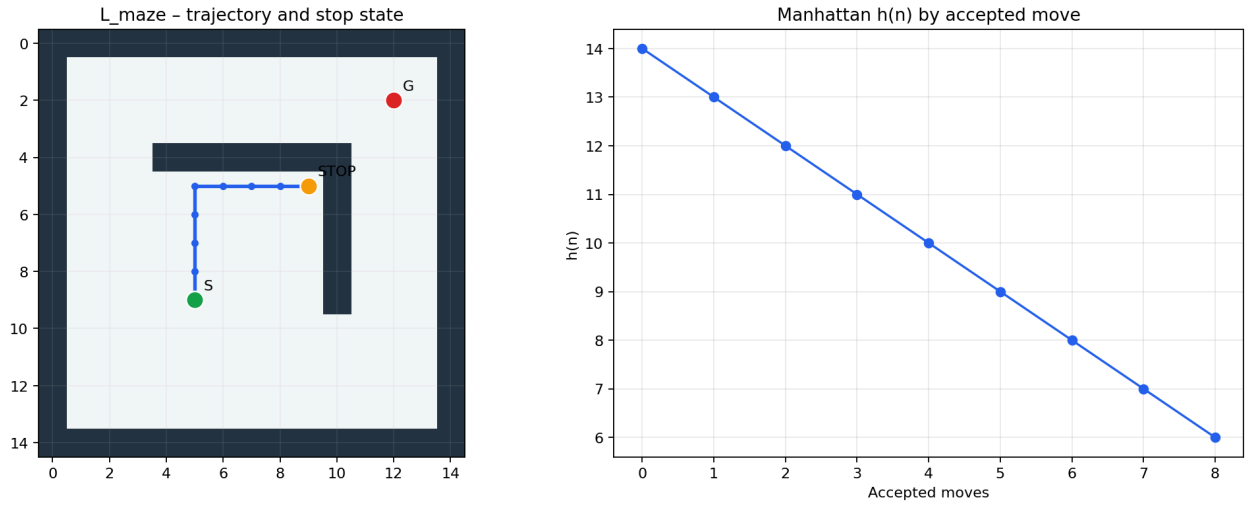
Kích thước mê cung	15 × 15
Điểm bắt đầu S	(9, 5)
Điểm đích G	(2, 12)
Vị trí thuật toán dừng	(5, 9)
Số bước đã thực hiện	8
h(S) và h tại điểm dừng	14 và 6
Đã đến đích G	Không
Đường ngắn nhất do BFS đối chứng	16 bước

Tại trạng thái dừng (5, 9), hai ô lân cận còn đi được là (6, 9) và (5, 8), mỗi ô đều có h = 7, lớn hơn h hiện tại bằng 6. Hai hướng còn lại bị tường chặn. Vì chương trình chỉ chấp nhận bước làm giảm h(n), nó không thể tiếp tục di chuyển.

Phép kiểm tra riêng bằng BFS xác nhận tồn tại đường đi từ S đến G với độ dài 16 bước; do đó, việc dừng là do hạn chế của quy tắc tối ưu cục bộ, không phải vì mê cung vô nghiệm.

1.4.2. Trực quan hóa quá trình tìm kiếm bằng animation

Khi bật animation=True, notebook lưu mỗi trạng thái vừa được chấp nhận thành một khung hình. Phần trực quan hóa bổ sung thể hiện đường đi bằng nét màu xanh, đánh dấu S màu xanh lá, G màu đỏ và trạng thái dừng màu cam. Biểu đồ bên cạnh mô tả sự thay đổi h(n) sau mỗi bước được chấp nhận.



Hình 1.1. Quỹ đạo tìm kiếm trên L_maze và giá trị Manhattan theo số bước di chuyển.

(Hình đã được chèn trực tiếp trong bản Word.)

Quan sát Hình 1.1 cho thấy tác tử di chuyển lên bốn bước rồi sang phải bốn bước và dừng tại đầu hành lang bị tường chắn phía trên và bên phải. Giá trị h(n) giảm liên tục từ 14 xuống 6, mỗi bước giảm đúng 1.

Đường hội tụ giảm đều thể hiện quá trình cải thiện theo heuristic nhưng không đồng nghĩa đã tìm được lời giải, bởi điểm dừng vẫn cách đích một khoảng Manhattan bằng 6.

1.4.3. Ảnh hưởng của thứ tự hướng di chuyển và ngẫu nhiên hóa

Nhằm khảo sát tác động của ORDER và RANDOMIZATION, notebook thực hiện sáu cấu hình thứ tự hướng cố định và mười lượt xáo trộn với seed từ 0 đến 9. Những lần chạy giữ nguyên mê cung, hàm Manhattan và điều kiện chấp nhận bước; chỉ thay cách sắp xếp lân cận trước khi lựa chọn.

Bảng 1.3. Kết quả thay đổi thứ tự hướng và ngẫu nhiên hóa

Cấu hình	Số lượt	Vị trí dừng	Số bước	Đạt G
NESW	1	(5, 9)	8	Không
ENWS	1	(5, 9)	8	Không
SENW	1	(5, 9)	8	Không
WSEN	1	(5, 9)	8	Không

NWSE	1	(5, 9)	8	Không
SNWE	1	(5, 9)	8	Không
Ngẫu nhiên, seed 0–9	10	(5, 9)	8	0/10

Cả 16 lượt chạy đều dừng tại (5, 9) sau tám bước, h(stop) = 6 và không lượt nào tới G.

Kết quả này chứng tỏ thay đổi thứ tự hướng hay xáo trộn lân cận chưa giúp thoát khỏi cực tiểu cục bộ trong L_maze đang xét. Các đường đi trung gian có thể khác nhau khi xuất hiện những lựa chọn cùng giá trị h(n), nhưng các phép thử không làm thay đổi trạng thái dừng hay chất lượng kết quả cuối cùng.

1.5. Nhận xét

Thực nghiệm cho thấy Hill Climbing có thể giảm hàm đánh giá một cách đều đặn mà vẫn không tìm được đích. Trên L_maze, thuật toán thực hiện tám bước cải thiện liên tiếp, đưa h(n) từ 14 xuống 6, song dừng tại (5, 9) vì không còn ô lân cận hợp lệ có giá trị h thấp hơn. Đây là một cực tiểu cục bộ theo khoảng cách Manhattan: trạng thái hiện tại tốt hơn các lân cận nhưng chưa phải nghiệm của bài toán tìm đường.

Nguyên nhân chủ yếu là hàm Manhattan chỉ tính độ chênh lệch tọa độ, không xét cấu trúc tường và nhu cầu đi vòng. Khi gặp vật cản hình chữ L, để tới G cần có những bước tạm thời không cải thiện hoặc làm tăng h(n). Quy tắc greedy trong

notebook không chấp nhận các bước này, nên thuật toán dừng sớm mặc dù BFS đối chứng đã xác định một đường đi hợp lệ dài 16 bước.

Kết quả từ sáu thứ tự cố định và mười seed ngẫu nhiên cũng cho thấy randomization không tự động giải quyết hiện tượng mắc kẹt. Đổi thứ tự chủ yếu ảnh hưởng đến việc xử lý những lân cận có giá trị bằng nhau; nó không thay đổi điều kiện chấp nhận h(mới) < h(hiện tại). Vì vậy, cần đánh giá tính hiệu quả của các cơ chế ngẫu nhiên trên nhiều môi trường và nhiều trạng thái khởi tạo thay vì chỉ dựa vào một lượt chạy.

Một lưu ý về phương pháp triển khai là cách chọn nút trong local_search() của notebook gốc sử dụng min() trên toàn bộ lân cận, gần với best-improvement hơn first-improvement. Việc nhận diện đúng quy tắc này giúp diễn giải kết quả khách quan và là tiền đề để so sánh các biến thể Hill Climbing trong những bài thực hành tiếp theo.

Nhìn chung, bài thực hành làm rõ lợi thế của tìm kiếm cục bộ về nguyên lý triển khai đơn giản, đồng thời minh họa hạn chế về khả năng thoát cực trị cục bộ và sự phụ thuộc vào hàm đánh giá.

CHƯƠNG 2: Mô hình hoá bài toán N quân hậu

2.1. Giới thiệu bài toán n-Queens

2.1.1. Phát biểu bài toán và mục tiêu tối ưu

Bài toán n quân hậu (n-Queens) yêu cầu bố trí n quân hậu trên bàn cờ kích thước n × n sao cho không có hai quân hậu nào tấn công nhau. Theo quy tắc di chuyển của

quân hậu trong cờ vua, hai quân hậu có thể tấn công nhau khi nằm trên cùng hàng, cùng cột hoặc cùng đường chéo. Mục tiêu của bài toán là tìm một cấu hình thỏa mãn đồng thời tất cả các điều kiện không xung đột.

Trong Lab 03 – Local Search, bài toán được tiếp cận dưới góc độ tối ưu hóa trạng thái. Thay vì xây dựng đường đi từ một trạng thái đầu đến một đích cố định, chương trình đánh giá chất lượng từng cách đặt hậu bằng số cặp quân hậu đang tấn công nhau. Hàm mục tiêu được biểu diễn như sau:

f(board) = conflicts(board)

Mục tiêu là tối thiểu hóa f(board). Một nghiệm hợp lệ đạt f(board) = 0; mọi cấu hình có f(board) > 0 vẫn tồn tại ít nhất một cặp hậu xung đột.

Bài thực hành sử dụng notebook n_queens.ipynb để khảo sát ba hàm random_board(n), show_board(board) và conflicts(board), đồng thời sinh các trạng thái lân cận và quan sát kết quả đối với n = 4, n = 8. Phạm vi của chương này là mô hình hóa bài toán và phân tích không gian lân cận; các thuật toán Hill Climbing sẽ được triển khai và so sánh ở những chương tiếp theo.

2.1.2. Biểu diễn trạng thái bằng vector n phần tử

Mỗi trạng thái được biểu diễn dưới dạng vector:

board = [q_0, q_1, …, q_{n-1}]

Trong đó, chỉ số j xác định cột và q_j là chỉ số hàng chứa quân hậu của cột đó. Cả hàng lẫn cột đều được đánh số từ 0 đến n − 1 theo quy ước của Python.

Ví dụ, cấu hình [1, 3, 0, 2] cho biết bốn quân hậu lần lượt nằm tại các tọa độ (1, 0), (3, 1), (0, 2) và (2, 3), với tọa độ viết theo thứ tự (hàng, cột).

Cách biểu diễn này mặc định mỗi cột có đúng một quân hậu nên loại bỏ hoàn toàn xung đột cùng cột. Tuy nhiên, hai hay nhiều phần tử của vector vẫn có thể bằng nhau hoặc tạo ra quan hệ cùng đường chéo; do đó, biểu diễn trạng thái chưa tự bảo đảm nghiệm hợp lệ.

Với mỗi phần tử được chọn trong n giá trị hàng, số cấu hình có thể biểu diễn là n^n. Khi n = 4, không gian này gồm $4^4 = 256$ cấu hình; khi n = 8, số cấu hình là $8^8 = 16.777.216$. Đây là không gian của mọi vector hợp lệ theo quy tắc một quân hậu mỗi cột, chứ không phải số lời giải không xung đột.

Kích thước không gian trạng thái tăng nhanh cho thấy sự cần thiết của hàm mục tiêu và các chiến lược cải thiện cục bộ.

2.2. Xây dựng các thành phần của bài toán

2.2.1. Hàm khởi tạo bàn cờ ngẫu nhiên

Hàm random_board(n) trong notebook tạo mảng NumPy có n phần tử, mỗi phần tử được chọn ngẫu nhiên trong khoảng từ 0 đến n − 1. Kết quả luôn bảo đảm đúng một quân hậu trong mỗi cột, nhưng không kiểm tra điều kiện cùng hàng hoặc đường chéo. Vì vậy, bàn cờ khởi tạo ngẫu nhiên có thể chứa nhiều xung đột và được sử dụng làm điểm xuất phát cho bài toán tối ưu hóa.

Để có thể tái lập kết quả thực nghiệm, phần mã bổ sung sử dụng np.random.seed(20261015) trước khi tạo các bàn cờ. Với seed này,

random_board(4) tạo cấu hình [3, 0, 3, 1], còn random_board(8) tạo [6, 3, 2, 3, 1, 0, 5, 2].

Cố định seed cho phép đối chiếu chính xác bảng kết quả và hình minh họa, song một seed đơn lẻ không đại diện cho mọi trạng thái ngẫu nhiên có thể phát sinh.

2.2.2. Hàm hiển thị trạng thái bàn cờ

Hàm show_board(board) chuyển vector trạng thái thành hình bàn cờ ô sáng – tối xen kẽ và đặt ký hiệu quân hậu lên đúng vị trí đã chỉ định. Chương trình sử dụng Matplotlib để trực quan hóa cấu hình, giúp nhận biết nhanh các quân hậu cùng hàng hoặc cùng đường chéo.

Hàm này chỉ phục vụ hiển thị và không có nhiệm vụ điều chỉnh vị trí để giảm xung đột.

Trong thí nghiệm, chương trình hiển thị ba trường hợp: bàn cờ 4-Queens ngẫu nhiên, cấu hình 4-Queens [1, 3, 0, 2] và bàn cờ 8-Queens ngẫu nhiên. Các hình trực quan là kết quả đánh giá mô hình trạng thái, không phải lời giải thu được từ một thuật toán Hill Climbing.

2.2.3. Hàm mục tiêu đếm số cặp quân hậu xung đột

Hàm conflicts(board) tính tổng số cặp quân hậu tấn công nhau. Do mỗi cột có đúng một quân hậu, việc đếm chỉ cần xét trùng hàng và hai hướng đường chéo.

Với quân hậu tại cột j, hàng q□, các nhóm dùng để đếm là hàng q□, đường chéo j + q□ và đường chéo j − q□. Trong mã nguồn, nhóm đường chéo thứ hai được cộng thêm n để sử dụng làm chỉ số mảng không âm.

Nếu một hàng hoặc đường chéo chứa k quân hậu, số cặp hậu xung đột trong nhóm đó là:

$C(k, 2) = k(k - 1)/2$

Do đó, giá trị f(board) được tính bằng tổng C(k, 2) trên toàn bộ các nhóm hàng và hai nhóm đường chéo. Cách tổ chức bộ đếm cho phép tránh việc duyệt trực tiếp mọi cặp hậu trong phần cài đặt hiện có của notebook.

Ví dụ, với board = [3, 0, 3, 1], hai quân hậu ở cột 0 và cột 2 cùng hàng 3 tạo thành một cặp xung đột, nên f(board) = 1. Đối với board = [1, 3, 0, 2], không có hai quân hậu nào cùng hàng hoặc cùng đường chéo nên f(board) = 0.

Trên bàn cờ 8-Queens ngẫu nhiên của thí nghiệm, hàm mục tiêu trả về f(board) = 7. Các kết quả này đã được đối chiếu với phép kiểm tra độc lập trong notebook.

2.3. Xác định không gian trạng thái lân cận

2.3.1. Phép di chuyển một quân hậu trong cùng cột

Toán tử lân cận được sử dụng trong Bài 2 là chuyển một quân hậu sang hàng khác nhưng giữ nguyên cột.

Cụ thể, chương trình chọn cột j, sao chép vector trạng thái hiện tại, thay board[j] bằng một chỉ số hàng mới khác giá trị cũ, rồi giữ nguyên tất cả phần tử còn lại. Hàm local_neighbors(board) trả về các bộ thông tin gồm cột được chọn, hàng mới và bàn cờ sau khi di chuyển.

Nhờ chỉ thay đổi một phần tử, mỗi trạng thái sinh ra vẫn có đúng một quân hậu trong từng cột. Việc sao chép mảng trước khi chỉnh sửa cũng bảo đảm quá trình đánh giá các lân cận không làm thay đổi bàn cờ ban đầu.

Ví dụ, từ cấu hình [3, 0, 3, 1], chuyển quân hậu ở cột 0 từ hàng 3 sang hàng 2 tạo trạng thái [2, 0, 3, 1] với f = 0. Đây là một bước đi cục bộ giúp đạt nghiệm hợp lệ của bài toán 4-Queens.

2.3.2. Công thức xác định số trạng thái lân cận

Với mỗi cột có một quân hậu, quân hậu đó có thể chuyển đến n − 1 hàng khác. Có n cột được lựa chọn, nên tổng số trạng thái lân cận khác nhau khi thực hiện đúng một phép chuyển là:

$N_lân_cận = n(n - 1)$

Đối với n = 4, mỗi bàn cờ có 4 × 3 = 12 trạng thái lân cận; đối với n = 8, con số tăng thành 8 × 7 = 56.

Công thức này không phụ thuộc việc trạng thái đang xét có bao nhiêu xung đột, bởi các thao tác chuyển hàng vẫn được phép thực hiện kể cả khi bàn cờ đã là nghiệm hợp lệ. Notebook đã kiểm chứng số lượng trạng thái được sinh, tính khác nhau của các lân cận và điều kiện mỗi trạng thái mới chỉ thay đổi một phần tử so với vector gốc.

Số lân cận tăng theo bậc n^2, do đó các phương pháp đánh giá toàn bộ lân cận trong mỗi lần tìm kiếm có thể phải tính hàm mục tiêu nhiều hơn khi kích thước bàn cờ tăng. Đây là cơ sở để so sánh các chiến lược Steepest-ascent, Stochastic và First-choice trong chương sau.

2.3.3. Điều kiện xác định nghiệm hợp lệ

Một trạng thái được xác định là lời giải hợp lệ khi có đúng n quân hậu, mỗi cột có một quân hậu và không tồn tại bất kỳ cặp quân hậu nào tấn công nhau.

Trong cách biểu diễn bằng vector, điều kiện số lượng và phân bố theo cột đã được bảo đảm từ khi khởi tạo. Vì vậy, điều kiện kiểm tra nghiệm trong chương trình được rút gọn thành:

f(board) = conflicts(board) = 0

Giá trị f(board) là số nguyên không âm. Khi f(board) > 0, vẫn còn xung đột; khi f(board) = 0, trạng thái đạt giá trị nhỏ nhất có thể của hàm mục tiêu.

Tuy nhiên, từ một bàn cờ chưa hợp lệ, không phải lúc nào cũng tồn tại một nước chuyển đơn lẻ đưa ngay f về 0. Điều này thể hiện rõ trong thí nghiệm n = 8, nơi có nhiều lân cận cải thiện nhưng chưa xuất hiện nghiệm chỉ sau một bước.

2.4. Kết quả thực nghiệm trên n-Queens

Các thí nghiệm sử dụng notebook n_queens_Bai2_Hoan_thien.ipynb và dữ liệu CSV đi kèm, với random seed = 20261015.

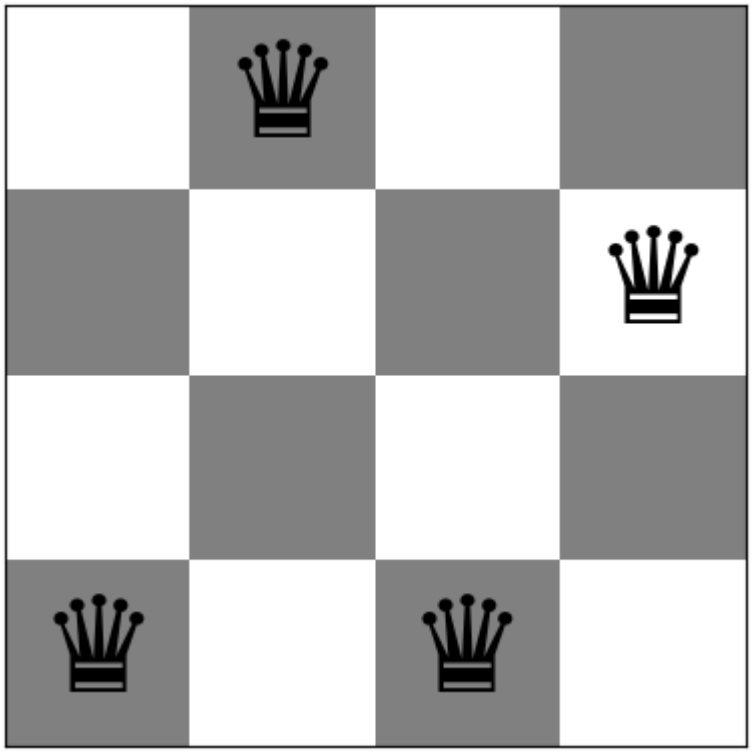
Ba cấu hình được khảo sát là 4-Queens ngẫu nhiên, 8-Queens ngẫu nhiên và 4-Queens tham chiếu [1, 3, 0, 2]. Chương trình đánh giá hàm mục tiêu tại trạng thái ban đầu và trên tất cả lân cận một bước, đồng thời ghi nhận số lân cận cải thiện và số lân cận có f = 0.

2.4.1. Khảo sát bàn cờ ngẫu nhiên với n = 4

Bàn cờ 4-Queens ngẫu nhiên có vector [3, 0, 3, 1] và giá trị mục tiêu f = 1. Hình minh họa cho thấy hai quân hậu nằm cùng hàng ở cột 0 và cột 2.

Từ cấu hình này, toán tử chuyển một quân hậu trong cùng cột sinh được 12 lân cận khác nhau. Trong số đó, chỉ có một trạng thái làm giảm số xung đột xuống 0, còn các trạng thái khác giữ nguyên hoặc làm tăng f.

Kết quả cho thấy, với một trạng thái gần nghiệm, chỉ một phép chuyển phù hợp cũng có thể loại bỏ hoàn toàn xung đột. Tuy nhiên, thuật toán vẫn cần cơ chế đánh giá hoặc lựa chọn lân cận để nhận diện bước cải thiện này; việc sinh lân cận tự nó không đồng nghĩa với đã thực hiện tối ưu hóa.



Hình 2.1. Bàn cờ 4-Queens khởi tạo ngẫu nhiên [3, 0, 3, 1].

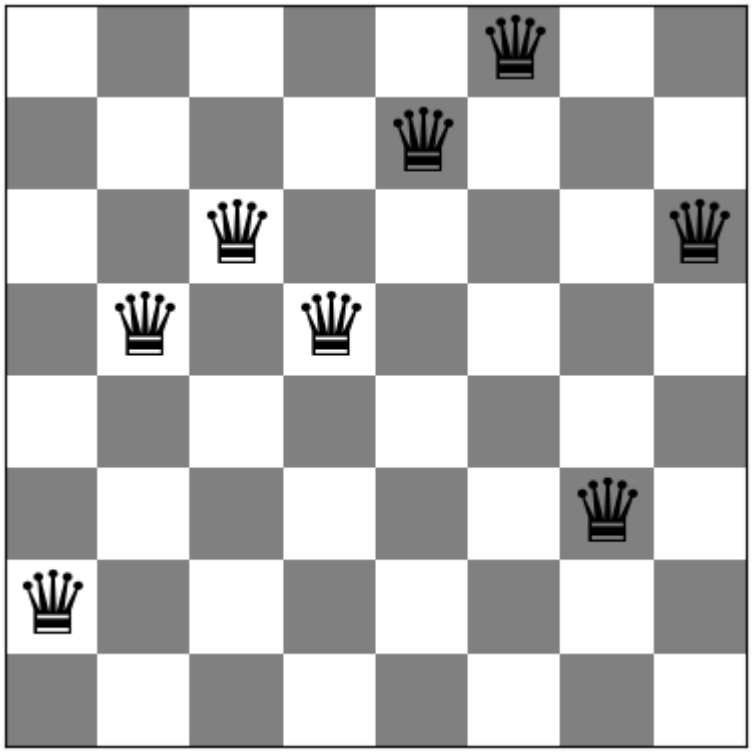
2.4.2. Khảo sát bàn cờ ngẫu nhiên với n = 8

Với n = 8, cấu hình ngẫu nhiên được tạo ra là [6, 3, 2, 3, 1, 0, 5, 2] và có f = 7 cặp xung đột. Trong đó tồn tại cả những cặp quân hậu cùng hàng lẫn các cặp nằm trên cùng đường chéo.

Mặc dù số quân hậu chỉ tăng từ 4 lên 8, kích thước không gian biểu diễn và số lân cận một bước đều lớn hơn đáng kể.

Từ bàn cờ này, chương trình sinh 56 lân cận; 17 lân cận có giá trị f nhỏ hơn 7, trong khi không có lân cận nào đạt f = 0. Giá trị xung đột thấp nhất trong số các lân cận là 4.

Như vậy, có thể giảm số xung đột từ 7 xuống 4 sau một nước đi, nhưng vẫn cần nhiều bước tiếp theo để tìm đến nghiệm không xung đột.

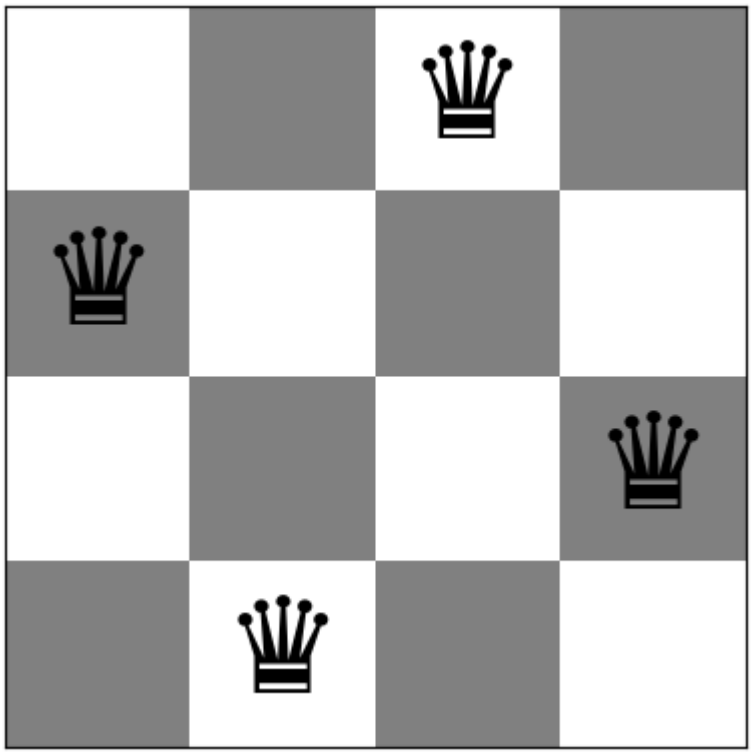


Hình 2.2. Bàn cờ 8-Queens khởi tạo ngẫu nhiên [6, 3, 2, 3, 1, 0, 5, 2].

2.4.3. So sánh với cấu hình không xung đột [1, 3, 0, 2]

Cấu hình tham chiếu [1, 3, 0, 2] được sử dụng để kiểm chứng điều kiện nghiệm hợp lệ. Trong trạng thái này, bốn quân hậu nằm ở các hàng khác nhau và không có hai quân hậu nào chung đường chéo. Do đó, hàm conflicts(board) cho kết quả bằng 0.

Mặc dù đã đạt nghiệm, bàn cờ này vẫn có 12 lân cận nếu áp dụng phép chuyển một quân hậu sang hàng khác; tất cả những lân cận đó đều có $f \geq 1$ và không cải thiện so với trạng thái gốc.



Hình 2.3. Bàn cờ 4-Queens tham chiếu [1, 3, 0, 2], không có xung đột.

Bảng 2.1. Tổng hợp kết quả khảo sát ba cấu hình n-Queens

Cấu hình	n	f ban đầu	Số lân cận	f nhỏ nhất của lân cận	Số lân cận cải thiện	Số lân cận có f = 0
4-Queens ngẫu nhiên	4	1	12	0	1	1
4-Queens tham chiếu	4	0	12	1	0	0
8-Queens ngẫu nhiên	8	7	56	4	17	0

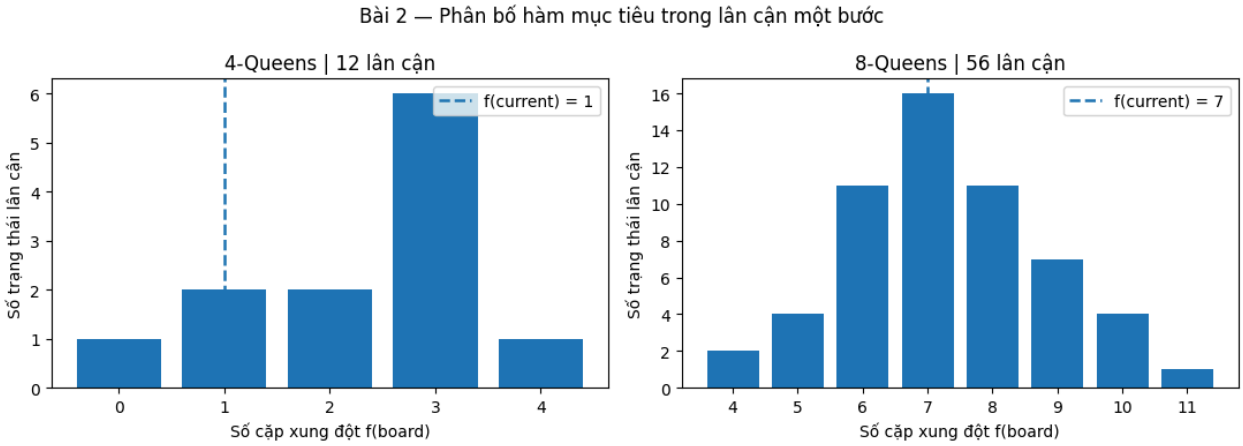
Bảng 2.1 cho thấy số lân cận phụ thuộc vào kích thước n chứ không phụ thuộc trực tiếp vào giá trị hàm mục tiêu. Cả hai cấu hình 4-Queens đều có 12 lân cận, mặc dù một cấu hình còn xung đột và một cấu hình đã là nghiệm hợp lệ.

Đồng thời, số lân cận cải thiện cũng khác biệt giữa các trạng thái: bàn cờ tham chiếu không còn bước cải thiện, trong khi bàn cờ 8-Queens có 17 lựa chọn giúp giảm số xung đột.

Biểu đồ phân bố hàm mục tiêu của toàn bộ lân cận một bước cho thấy đối với 4-Queens ngẫu nhiên, phần lớn các nước đi làm tăng số cặp xung đột thay vì cải thiện.

Với 8-Queens, các giá trị f của lân cận phân bố trong khoảng 4 đến 11, trong đó nhiều trạng thái vẫn có số xung đột xấp xỉ cấu hình hiện tại là 7.

Điều này phản ánh sự khác biệt đáng kể về chất lượng của những nước di chuyển hợp lệ và vai trò của hàm mục tiêu trong việc lựa chọn bước tìm kiếm.



Hình 2.4. Phân bố số cặp xung đột của toàn bộ lân cận một bước ở n = 4 và n = 8.

2.5. Nhận xét

Qua bài thực hành, có thể nhận thấy việc biểu diễn bàn cờ bằng vector n phần tử là một phương án phù hợp cho tìm kiếm cục bộ. Cách mô hình hóa này bảo đảm mỗi cột luôn có đúng một quân hậu, giúp loại bỏ xung đột theo cột ngay từ đầu và giảm số trường hợp phải xét so với biểu diễn tùy ý n vị trí trên bàn cờ. Tuy vậy, mô hình vẫn cho phép xung đột theo hàng và đường chéo, nên cần sử dụng hàm conflicts(board) để xác định chất lượng của từng trạng thái.

Kết quả thực nghiệm cho thấy giá trị hàm mục tiêu thay đổi đáng kể sau một phép di chuyển hợp lệ. Với 4-Queens ngẫu nhiên, một trong 12 lân cận đưa f từ 1 về 0, trong khi 8-Queens ngẫu nhiên có 17 trong 56 lân cận cải thiện nhưng không có lựa chọn nào giải được bài toán ngay lập tức. Cấu hình [1, 3, 0, 2] minh chứng rõ điều kiện nghiệm tối ưu toàn cục khi f = 0; tất cả các lân cận của cấu hình này đều có chi phí xung đột lớn hơn.

Bên cạnh đó, toán tử thay đổi hàng của một quân hậu xác định trực tiếp kích thước và cấu trúc lân cận mà thuật toán có thể khảo sát. Số lân cận tăng từ 12 lên 56 khi n tăng từ 4 lên 8, khiến chi phí đánh giá toàn bộ lân cận có xu hướng tăng. Điều này cho thấy cách lựa chọn lân cận sẽ ảnh hưởng đến hiệu quả của các thuật toán Steepest-ascent Hill Climbing, Stochastic Hill Climbing và First-choice Hill Climbing.

Nhìn chung, Bài 2 đã hoàn thiện việc mô hình hóa trạng thái, hàm mục tiêu, toán tử di chuyển và điều kiện xác định nghiệm cho bài toán n-Queens. Kết quả trong chương là cơ sở để triển khai và đánh giá các chiến lược tối ưu hóa cục bộ trong những bài thực hành tiếp theo. Trong phạm vi chương này, các kết quả chỉ là khảo sát trạng thái và lân cận một bước, chưa phải kết quả hội tụ của một thuật toán tìm kiếm hoàn chỉnh.

CHƯƠNG 3: Steepest-ascent và Stochastic hill climbing

3.1. Cơ sở lý thuyết các thuật toán Hill Climbing

Tìm kiếm leo đồi (Hill Climbing) là một nhóm phương pháp tìm kiếm cục bộ, trong đó thuật toán duy trì một trạng thái hiện tại và lựa chọn các bước di chuyển làm cải thiện giá trị hàm mục tiêu. Đối với bài toán n quân hậu đã mô hình hóa ở Chương 2, trạng thái là vector n phần tử, mỗi phần tử cho biết hàng của quân hậu tương ứng với một cột. Hàm mục tiêu f(board) = conflicts(board) đếm tổng số cặp hậu có thể tấn công nhau, do đó quá trình tìm kiếm hướng tới việc tối thiểu hóa f; nghiệm hợp lệ đạt f = 0.

Ba biến thể được khảo sát trong chương có cùng hàm mục tiêu và toán tử lân cận: di chuyển đúng một quân hậu sang một hàng khác trong cùng cột. Điểm khác biệt nằm ở cách phát hiện và chấp nhận bước cải thiện. Các thuật toán chỉ nhận nước đi có f(mới) < f(hiện tại), nên giá trị hàm mục tiêu giảm nghiêm ngặt sau mỗi bước đã chấp nhận. Tuy nhiên, quy tắc này có thể khiến thuật toán dừng khi chưa đạt nghiệm, đặc biệt tại những trạng thái không còn lân cận cải thiện.

3.1.1. Thuật toán Steepest-ascent Hill Climbing

Steepest-ascent Hill Climbing, khi áp dụng cho hàm mục tiêu cần tối thiểu hóa, đánh giá toàn bộ trạng thái lân cận và chọn trạng thái có giá trị f nhỏ nhất. Gọi N(x) là tập trạng thái lân cận của trạng thái hiện tại x. Thuật toán tìm x* thuộc N(x) sao

cho f(x*) nhỏ nhất, rồi chỉ chuyển sang x* nếu f(x*) < f(x). Nếu không tồn tại lân cận cải thiện, quá trình tìm kiếm kết thúc.

Với toán tử chuyển hàng một quân hậu, mỗi bàn cờ có M = n(n − 1) lân cận. Bởi vậy, một vòng khảo sát đầy đủ phải đánh giá M nước đi nếu trạng thái hiện tại chưa có f = 0. Phương pháp có ưu điểm lựa chọn được bước cải thiện mạnh nhất trong số các lân cận được xét, song chi phí mỗi vòng lớn hơn khi n tăng. Ngoài ra, chọn tốt nhất ở từng bước không đồng nghĩa với việc tìm được nghiệm toàn cục.

3.1.2. Thuật toán Stochastic Hill Climbing

Stochastic Hill Climbing đưa yếu tố ngẫu nhiên vào việc chọn nước đi cải thiện. Trong biến thể Inefficient Stochastic được yêu cầu ở Task 2, chương trình vẫn đánh giá toàn bộ M lân cận của trạng thái hiện tại, nhưng thay vì chỉ giữ lân cận có giá trị f thấp nhất, chương trình thu thập tất cả lân cận thỏa f(mới) < f(hiện tại), sau đó chọn ngẫu nhiên một phần tử trong tập này.

Nhờ cách chọn đó, những lần chạy từ cùng một bàn cờ có thể hình thành các quỹ đạo tối ưu hóa khác nhau. Tuy nhiên, phương pháp vẫn phải tính f cho toàn bộ lân cận ở mỗi vòng, do đó không loại bỏ được chi phí quét toàn bộ không gian lân cận. Đồng thời, lựa chọn ngẫu nhiên trong tập các bước cải thiện không bảo đảm chọn được bước có mức giảm f lớn nhất.

3.1.3. Thuật toán First-choice Hill Climbing

First-choice Hill Climbing là cách hiện thực Efficient Stochastic trong Task 3. Thuật toán tạo ngẫu nhiên từng trạng thái lân cận, đánh giá f rồi chấp nhận ngay khi phát hiện một nước đi có giá trị mục tiêu nhỏ hơn trạng thái hiện tại. Nếu nước đi

không cải thiện, chương trình thử một nước đi khác mà không cần lập danh sách đầy đủ của toàn bộ lân cận.

Trong notebook, các nước đi được lấy mẫu ngẫu nhiên có hoàn lại. Vì vậy, cùng một nước đi có thể được thử nhiều lần, còn một số lân cận khác có thể chưa được xét. Để tránh vòng lặp thử kéo dài không giới hạn, chương trình đặt ngưỡng k lần thử liên tiếp không cải thiện. Ngưỡng mặc định được tính gần đúng theo k = ceil[M × ln(1/p)], với p = 0,05 và M = n(n − 1). Cách lựa chọn này nhằm giảm xác suất bỏ sót một nước đi cải thiện cố định dưới giả định lấy mẫu đều, chứ không chứng minh chắc chắn mọi nước đi đã được kiểm tra.

Do đó, trạng thái dừng TRY_LIMIT của First-choice phải được phân biệt với LOCAL_MINIMUM của hai phương pháp quét toàn bộ: TRY_LIMIT chỉ cho biết đã hết ngân sách lấy mẫu mà chưa tìm được bước cải thiện, không khẳng định rằng không tồn tại lân cận tốt hơn.

3.2. Triển khai thuật toán trên bài toán n-Queens

Cả ba thuật toán được hiện thực trên cùng notebook n_queens_Bai3_Hoan_thien.ipynb. Chương trình sử dụng các hàm random_board(n), conflicts(board) và local_neighbors(board) của các phần đã hoàn thiện, không thay đổi quy ước biểu diễn trạng thái từ Chương 2. Để bảo đảm dữ liệu đầu vào không bị sửa ngoài ý muốn, mỗi thuật toán tạo bản sao vector trước khi cập nhật các vị trí quân hậu. Mọi kết quả được trả về dưới dạng từ điển, gồm bàn cờ sau tìm kiếm, số bước, lịch sử f, số lần gọi conflicts(), trạng thái kết thúc và thời gian thực thi.

3.2.1. Triển khai Steepest-ascent – Task 1

Task 1 được hoàn thiện bằng hàm steepest_ascent(board, max_steps=1000). Chương trình đánh giá trạng thái đầu để xác định current_f, sau đó duyệt các bộ dữ liệu do local_neighbors(current) sinh ra. Biến best_f lưu giá trị tốt nhất tính được trong vòng duyệt; biến best_board lưu trạng thái tương ứng. Chỉ khi best_f nhỏ hơn current_f, chương trình mới cập nhật bàn cờ, tăng số bước được chấp nhận và thêm giá trị mới vào history.

Nếu trạng thái ban đầu hoặc trạng thái mới có f = 0, kết quả được gắn SOLVED. Nếu đã kiểm tra toàn bộ lân cận mà không có bước cải thiện, kết quả nhận nhãn LOCAL_MINIMUM. Trường hợp đạt max_steps nhưng chưa giải được bài toán được gắn STEP_LIMIT. Vì mỗi vòng lặp xét toàn bộ M lân cận, số lần đánh giá bên trong hàm tuân theo công thức evaluations = 1 + M × số lượt quét đã thực hiện; số lượt quét có thể bao gồm lượt cuối dùng để xác nhận không còn cải thiện.

3.2.2. Triển khai Inefficient Stochastic – Task 2

Task 2 được triển khai trong hàm stochastic_all_improvements(board, seed=0, max_steps=1000). Tại mỗi vòng, chương trình xét toàn bộ trạng thái lân cận, đưa những ứng viên có candidate_f < current_f vào danh sách improving, rồi sử dụng bộ sinh ngẫu nhiên np.random.default_rng(seed) để chọn một nước đi từ danh sách này.

Nếu danh sách improving rỗng, chương trình xác định trạng thái hiện tại không có lân cận cải thiện theo toán tử đã chọn và kết thúc với LOCAL_MINIMUM. Số lần đánh giá hàm mục tiêu vẫn gồm một lần đối với trạng thái đầu và M lần cho mỗi vòng quét. Khác với Steepest-ascent, thứ tự tiến tới nghiệm và số vòng tìm kiếm có thể thay đổi theo seed do lựa chọn ngẫu nhiên không bắt buộc là lân cận tốt nhất.

3.2.3. Triển khai Efficient Stochastic – Task 3

Task 3 sử dụng hàm first_choice_hill_climbing(board, seed=0, max_steps=1000, max_no_improvement=None). Ở mỗi lần thử, chương trình chọn ngẫu nhiên một cột, sau đó chọn ngẫu nhiên một hàng khác với hàng hiện tại. Bàn cờ ứng viên được tạo bằng cách sao chép vector và thay đúng một phần tử, bảo đảm cùng toán tử lân cận với hai Task trước.

Nếu f của ứng viên nhỏ hơn current_f, nước đi được chấp nhận ngay, bộ đếm số lần thử không cải thiện được đặt về 0 và số bước tăng một đơn vị. Nếu không cải thiện, bộ đếm failures được tăng lên. Khi failures đạt ngưỡng, thuật toán dừng với TRY_LIMIT. Theo cấu hình mặc định, n = 4 có M = 12 và ngưỡng 36 lần thử; n = 8 có M = 56 và ngưỡng 168 lần thử liên tiếp không cải thiện. Bên cạnh STEP_LIMIT và SOLVED, trạng thái TRY_LIMIT giúp nhận diện rõ giới hạn của chiến lược lấy mẫu ngẫu nhiên.

3.3. Thiết lập thực nghiệm và phương pháp đánh giá

3.3.1. Dữ liệu khởi tạo và random seed

Thí nghiệm minh họa sử dụng đúng hai trạng thái ngẫu nhiên đã tạo ở Bài 2 với seed 20261015. Cấu hình 4-Queens là [3, 0, 3, 1] có f ban đầu bằng 1; cấu hình 8-Queens là [6, 3, 2, 3, 1, 0, 5, 2] có f ban đầu bằng 7. Cả ba thuật toán nhận cùng vector khởi tạo để hạn chế khác biệt do dữ liệu đầu vào. Hai thuật toán có yếu tố ngẫu nhiên sử dụng seed được truyền trực tiếp vào hàm.

Để tránh đánh giá hiệu suất chỉ từ một ví dụ, notebook tiếp tục tạo 20 bàn cờ ngẫu nhiên cho mỗi kích thước n = 4 và n = 8. Trên mỗi bàn cờ, cả ba thuật toán đều

được thực hiện, tổng cộng 20 × 2 × 3 = 120 lượt chạy. Seed tạo dữ liệu được xác định theo công thức 20261015 + 1000n + chỉ số lượt thử (bắt đầu từ 0); vì vậy, các thuật toán được so sánh trên cùng trạng thái ban đầu ở từng lượt. Phần thực nghiệm sử dụng giới hạn tối đa 1000 bước cải thiện cho mỗi thuật toán.

3.3.2. Các chỉ số đánh giá hiệu suất thuật toán

Giá trị final_f biểu diễn số cặp hậu xung đột tại thời điểm kết thúc; final_f = 0 tương ứng với nghiệm hợp lệ. Chỉ số steps đếm số bước di chuyển thực sự được chấp nhận, không đếm các ứng viên được thử nhưng bị loại. Lịch sử history lưu giá trị f tại trạng thái ban đầu và sau từng bước chấp nhận, phục vụ phân tích đồ thị hội tụ.

Chỉ số evaluations đếm số lần gọi conflicts() bên trong một lượt thực thi hàm tìm kiếm, kể cả lần đánh giá trạng thái đầu. Những lệnh gọi bổ sung bên ngoài hàm để kiểm tra hoặc vẽ hình không được tính. Chỉ số elapsed_ms đo thời gian thực thi bằng time.perf_counter(), không bao gồm thời gian tạo đồ thị. Tỷ lệ thành công là số lần chạy đạt final_f = 0 chia cho tổng số lượt thực nghiệm. Để đánh giá khách quan, chương trình đồng thời ghi nhãn SOLVED, LOCAL_MINIMUM, TRY_LIMIT hoặc STEP_LIMIT.

Các chỉ số này phản ánh những khía cạnh khác nhau: ít bước chấp nhận không có nghĩa là ít lần đánh giá, và ít lần đánh giá cũng không nhất thiết đồng nghĩa thời gian chạy thấp hơn. Hơn nữa, thời gian được đo trên từng lượt chạy có thể bị ảnh hưởng bởi môi trường thực thi; kết luận cần dựa trên cả số liệu trung bình nhiều lần lẫn bản chất của thuật toán.

3.4. Kết quả thực nghiệm

3.4.1. Kết quả thực nghiệm với n = 4

Trên bàn cờ [3, 0, 3, 1], cả ba phương pháp tìm được trạng thái [2, 0, 3, 1] có f = 0 sau đúng một bước cải thiện. Steepest-ascent và Stochastic (all) đều sử dụng 13 lần đánh giá, bao gồm một lần đánh giá ban đầu và 12 lần đối với các lân cận. First-choice tìm được nước đi giải bài toán sau 11 lần gọi conflicts(), trong đó có 10 lần đánh giá ứng viên ngẫu nhiên.

Bảng 3.1. Kết quả một lần chạy trên bàn cờ 4-Queens, f ban đầu = 1

Thuật toán	f cuối	Bước cải thiện	Lần gọi conflicts()	Thời gian (ms)	Trạng thái
Steepest-ascent	0	1	13	0,070	SOLVED
Stochastic (all)	0	1	13	0,097	SOLVED
First-choice	0	1	11	0,070	SOLVED

Kết quả này cho thấy First-choice có thể tránh việc đánh giá toàn bộ lân cận nếu gặp sớm một nước đi tốt. Tuy vậy, trên ví dụ có quy mô nhỏ và chỉ có một bước cải thiện dẫn tới nghiệm, chênh lệch về số lần đánh giá chưa đủ để khẳng định hiệu quả

trung bình. Những phép thử ở nhiều trạng thái ban đầu được sử dụng ở mục 3.4.3 nhằm đánh giá khả năng lặp lại của kết quả.

3.4.2. Kết quả thực nghiệm với n = 8

Trên cấu hình [6, 3, 2, 3, 1, 0, 5, 2], giá trị hàm mục tiêu ban đầu bằng 7. Cả ba thuật toán đều giảm số cặp xung đột về 1 nhưng chưa đạt f = 0. Steepest-ascent chấp nhận 3 bước, thực hiện 225 lần đánh giá và kết thúc với LOCAL_MINIMUM. Stochastic (all) đi 6 bước, thực hiện 393 lần đánh giá rồi cũng dừng tại LOCAL_MINIMUM. First-choice chấp nhận 5 bước, sử dụng 226 lần đánh giá và dừng với TRY_LIMIT.

Bảng 3.2. Kết quả một lần chạy trên bàn cờ 8-Queens, f ban đầu = 7

Thuật toán	f cuối	Bước cải thiện	Lần gọi conflicts()	Thời gian (ms)	Trạng thái
Steepest-ascent	1	3	225	1,684	LOCAL_MINIMUM
Stochastic (all)	1	6	393	2,586	LOCAL_MINIMUM
First-choice	1	5	226	2,390	TRY_LIMIT

Steepest-ascent sử dụng ít bước hơn do mỗi lần lựa chọn lân cận có giá trị f thấp nhất trong những trạng thái đã xét. Stochastic (all) có thể tiến từng bước nhỏ hơn do chọn ngẫu nhiên trong tập cải thiện, nên đường hội tụ dài hơn và số lượt quét lân cận lớn hơn. Với First-choice, việc kết thúc tại f = 1 sau TRY_LIMIT chỉ khẳng định không gặp bước cải thiện trong số lần lấy mẫu đã quy định; không nên diễn giải là đã kiểm tra hết mọi lân cận.

3.4.3. So sánh tỷ lệ thành công và chi phí tính toán

Để kiểm tra tính ổn định của các nhận xét, bảng thống kê tổng hợp ghi nhận kết quả của 20 lượt chạy cho từng thuật toán ở mỗi kích thước. Tổng số lượt chạy là 120, sử dụng lại cùng một bàn cờ khởi tạo cho cả ba thuật toán ở từng lượt thử.

Bảng 3.3. Kết quả trung bình từ 20 lần chạy cho mỗi thuật toán và kích thước

n	Thuật toán	Thành công	f cuối TB	Bước TB	Gọi conflicts() TB	Thời gian TB (ms)
4	Steepest-ascent	55%	0,45	1,65	26,20	0,120
4	Stochastic (all)	50%	0,50	1,85	29,20	0,111
4	First-choice	25%	0,80	1,90	37,05	0,260

8	Steepest-ascent	25%	1,30	3,05	213,80	1,421
8	Stochastic (all)	10%	1,55	4,20	286,60	1,825
8	First-choice	20%	1,15	4,85	197,80	1,865

Với n = 4, Steepest-ascent đạt 55% thành công (11/20 lượt), Stochastic (all) đạt 50% (10/20) và First-choice đạt 25% (5/20). Giá trị f cuối trung bình lần lượt là 0,45; 0,50 và 0,80. Ở kích thước này, Steepest-ascent có số lần đánh giá trung bình thấp nhất trong ba phương pháp, đạt 26,20; Stochastic (all) đạt 29,20; First-choice đạt 37,05. Như vậy, trên tập trạng thái đã thử, việc lấy mẫu ngẫu nhiên có hoàn lại không mặc nhiên tiết kiệm đánh giá hơn quét toàn bộ lân cận.

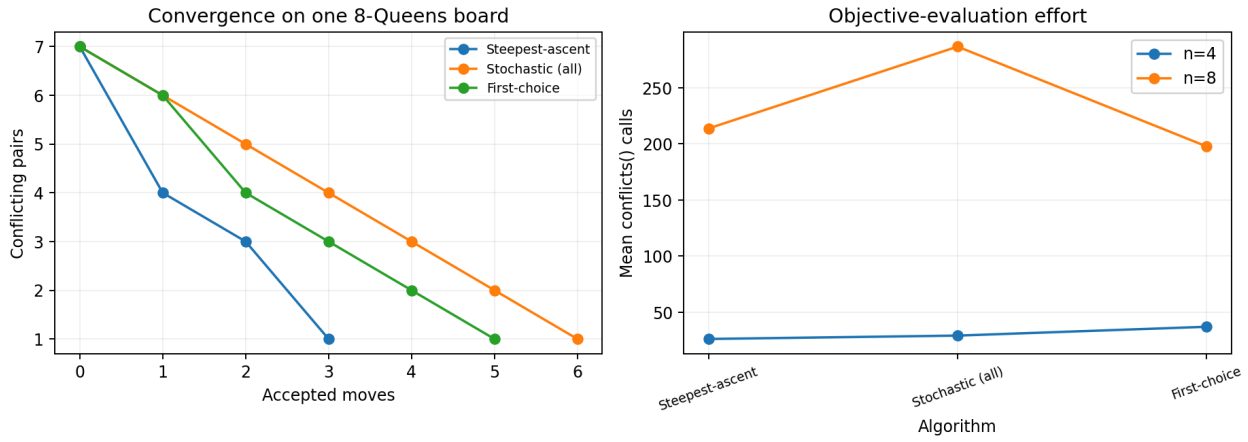
Với n = 8, tỷ lệ thành công giảm còn 25% (5/20) đối với Steepest-ascent, 10% (2/20) đối với Stochastic (all) và 20% (4/20) đối với First-choice. First-choice có giá trị f cuối trung bình thấp nhất trong ba thuật toán, bằng 1,15, và số lần đánh giá trung bình bằng 197,80. Steepest-ascent có f trung bình 1,30 với 213,80 lần đánh giá, còn Stochastic (all) có f trung bình 1,55 với 286,60 lần đánh giá. Kết quả cho thấy việc chọn cách di chuyển ảnh hưởng đồng thời đến tỷ lệ đạt nghiệm và khối lượng tính toán; không tồn tại một thứ tự vượt trội tuyệt đối trên mọi chỉ số của thí nghiệm.

Về thời gian, trung bình ở n = 8 là khoảng 1,421 ms đối với Steepest-ascent, 1,825 ms đối với Stochastic (all) và 1,865 ms đối với First-choice. Mặc dù First-choice

gọi conflicts() ít hơn Steepest-ascent trong các lượt n = 8, thời gian trung bình của nó vẫn lớn hơn trong số liệu thu được. Khác biệt này phù hợp với việc mỗi chiến lược có chi phí sinh và lựa chọn ứng viên khác nhau, song thời gian thực thi trong các lượt đo ngắn có thể chịu nhiễu; không nên xem số liệu này như một kết luận hiệu suất tuyệt đối.

3.4.4. Trực quan hóa quá trình hội tụ

Hình 3.1 thể hiện hai góc nhìn bổ sung. Biểu đồ bên trái theo dõi f(board) sau mỗi bước được chấp nhận trên cùng một bàn cờ 8-Queens, còn biểu đồ bên phải so sánh số lần gọi conflicts() trung bình từ 20 lượt chạy theo mỗi kích thước.



Hình 3.1. Quá trình hội tụ trên một bàn cờ 8-Queens và số lần gọi conflicts() trung bình sau 20 lượt chạy cho mỗi kích thước.

Đường của Steepest-ascent giảm từ 7 xuống 4, 3 rồi 1 sau ba bước; Stochastic (all) giảm lần lượt từ 7 xuống 6, 5, 4, 3, 2 và 1 sau sáu bước; First-choice giảm từ 7 xuống 6, 4, 3, 2 và 1 sau năm bước. Các đường đều giảm nghiêm ngặt vì cả ba thuật toán không cho phép chấp nhận nước đi có f lớn hơn hoặc bằng trạng thái hiện tại.

Hình hội tụ minh họa rõ khác biệt về số bước đã chấp nhận, nhưng trục hoành không phản ánh trực tiếp số ứng viên đã đánh giá hay thời gian. Trong khi đó, biểu đồ công sức đánh giá cho thấy ở n = 8, Stochastic (all) tốn nhiều lần gọi conflicts() trung bình nhất; ở n = 4, First-choice lại có số lần đánh giá trung bình cao nhất. Các quan sát này củng cố nhận xét rằng hiệu quả của một chiến lược tìm kiếm không nên được đánh giá bằng một chỉ số duy nhất.

3.5. Nhận xét

Qua thực nghiệm, ba phương pháp Hill Climbing có cùng mục tiêu giảm số cặp quân hậu tấn công nhau nhưng khác nhau về cơ chế lựa chọn lân cận. Steepest-ascent đánh giá toàn bộ các nước đi và chọn phương án cải thiện mạnh nhất tại mỗi bước. Stochastic (all) cũng quét toàn bộ lân cận nhưng chọn ngẫu nhiên từ tập các nước đi cải thiện. First-choice chỉ kiểm tra từng ứng viên được lấy mẫu và chấp nhận ngay khi tìm thấy bước tốt hơn, giúp tránh một lượt quét đầy đủ trong những trường hợp thuận lợi.

Trên ví dụ 4-Queens ngẫu nhiên, cả ba thuật toán tìm được nghiệm f = 0 sau một bước. Tuy nhiên, ở 8-Queens mẫu, tất cả cùng dừng ở f = 1 mà chưa giải được bài toán. Hiện tượng này minh họa hạn chế của việc chỉ chấp nhận bước giảm nghiêm ngặt: quá trình tối ưu có thể bị cản trở bởi cực tiểu cục bộ hoặc bởi giới hạn lấy mẫu ở First-choice. Cần phân biệt LOCAL_MINIMUM được xác nhận bằng việc quét toàn bộ lân cận với TRY_LIMIT chỉ phản ánh đã hết ngân sách lấy mẫu.

Kết quả 20 lần chạy cho mỗi kích thước cho thấy khi n tăng từ 4 lên 8, tỷ lệ đạt nghiệm giảm rõ ở cả ba biến thể, trong khi số lần đánh giá trung bình tăng đáng kể. Steepest-ascent cho tỷ lệ thành công cao nhất trong nhóm được khảo sát trên cả hai kích thước, nhưng First-choice có mức f cuối trung bình thấp hơn và số lần gọi hàm

mục tiêu thấp hơn tại n = 8. Vì thế, lựa chọn thuật toán cần dựa trên mục tiêu ưu tiên: xác suất đạt nghiệm, giá trị f cuối cùng, số phép đánh giá hoặc thời gian xử lý. Các hạn chế còn tồn tại là cơ sở khảo sát Random Restart và Simulated Annealing ở Chương 4.

CHƯƠNG 4: Random forest và Stimulated annealing

4.1. Hạn chế của các phương pháp Hill Climbing

Trong Chương 3, ba biến thể Hill Climbing đã được sử dụng để giảm số cặp quân hậu tấn công nhau trên bàn cờ n-Queens. Mỗi trạng thái được biểu diễn bằng một vector n phần tử, trong đó vị trí phần tử biểu diễn cột và giá trị phần tử biểu diễn hàng của quân hậu. Hàm mục tiêu f(board) = conflicts(board) cần được tối thiểu hóa; khi f(board) = 0, bàn cờ không có cặp quân hậu xung đột. Toán tử lân cận được giữ thống nhất là chuyển một quân hậu sang một hàng khác trong cùng cột.

Các thuật toán Steepest-ascent, Stochastic Hill Climbing và First-choice Hill Climbing đều dựa trên những bước cải thiện nghiêm ngặt, tức chỉ chấp nhận trạng thái mới khi f(mới) nhỏ hơn f(hiện tại). Quy tắc này làm cho giá trị hàm mục tiêu giảm liên tục sau mỗi bước được chấp nhận, nhưng đồng thời hạn chế khả năng khám phá những vùng không gian trạng thái phải đi qua bước không cải thiện mới có thể đến nghiệm.

Bài thực hành này khảo sát Random Restart và Simulated Annealing như hai cơ chế khác nhau nhằm khắc phục một phần hạn chế của những phương pháp Hill Climbing truyền thống.

4.1.1. Hiện tượng cực trị cục bộ (Local Optimum)

Cực trị cục bộ là trạng thái không có nước đi lân cận nào có giá trị hàm mục tiêu tốt hơn, mặc dù giá trị hiện tại chưa phải tối ưu trên toàn bộ không gian trạng thái.

Đối với bài toán tối thiểu hóa n-Queens, trạng thái x được xem là cực tiểu cục bộ theo toán tử đã chọn nếu:

$f(x) \leq f(y)$, với mọi y thuộc N(x).

Trong đó N(x) là tập trạng thái lân cận của x. Tuy nhiên, f(x) vẫn có thể lớn hơn 0, nghĩa là bàn cờ còn tồn tại các cặp hậu xung đột.

Trong thực nghiệm Bài 3, từ một bàn cờ 8-Queens có f ban đầu bằng 7, cả ba thuật toán Hill Climbing đều giảm số xung đột xuống 1 nhưng chưa tìm được nghiệm hợp lệ. Kết quả này minh họa hạn chế của việc chỉ lựa chọn các trạng thái cải thiện tại mỗi bước.

Đây là cơ sở để nghiên cứu những phương pháp có khả năng thay đổi điểm xuất phát hoặc chấp nhận các bước di chuyển tạm thời làm tăng giá trị hàm mục tiêu.

4.1.2. Hiện tượng Plateau và Ridge

Plateau là vùng trong không gian trạng thái mà nhiều trạng thái lân cận có cùng giá trị hàm mục tiêu. Khi thuật toán chỉ chấp nhận những bước cải thiện nghiêm ngặt, các bước có $\Delta = 0$ sẽ không được thực hiện. Điều này có thể khiến thuật toán dừng mặc dù việc di chuyển qua một số trạng thái bằng nhau có thể dẫn đến trạng thái tốt hơn.

Ridge mô tả khu vực có hướng cải thiện khó tiếp cận bằng một bước đơn lẻ của toán tử lân cận. Trong bài toán n-Queens, một cấu hình có thể cần phối hợp nhiều lần di chuyển quân hậu để giảm tổng số xung đột. Khi chỉ được thay đổi vị trí một quân hậu tại mỗi bước, thuật toán có thể gặp khó khăn trong việc xác định hướng cải thiện.

Plateau và Ridge cho thấy hiệu quả của Hill Climbing phụ thuộc không chỉ vào hàm mục tiêu mà còn vào cấu trúc không gian trạng thái và toán tử lân cận.

Trong notebook Bài 4, các hiện tượng này được sử dụng để giải thích cơ sở lý thuyết của những phương pháp tìm kiếm mở rộng, thay vì được đo lường thành các chỉ số thực nghiệm riêng biệt.

4.2. Thuật toán Random Restart

4.2.1. Nguyên lý khởi tạo lại trạng thái ngẫu nhiên

Random Restart là chiến lược thực hiện nhiều lượt tìm kiếm cục bộ từ những trạng thái khởi tạo khác nhau. Khi một lượt Hill Climbing dừng mà chưa tìm được nghiệm f = 0, chương trình sinh một bàn cờ ngẫu nhiên mới và tiếp tục thực hiện tìm kiếm.

Mục đích của cơ chế này là mở rộng phạm vi khám phá không gian trạng thái và giảm sự phụ thuộc vào một điểm khởi tạo duy nhất.

Trong Task 4, lần chạy đầu sử dụng bàn cờ được truyền vào chương trình. Từ lượt chạy thứ hai, các trạng thái khởi tạo mới được sinh bằng bộ sinh số ngẫu nhiên. Sau mỗi lượt, nghiệm tốt nhất được cập nhật nếu tìm thấy bàn cờ có số cặp xung đột thấp hơn.

Chiến lược Random Restart không thay đổi nguyên lý lựa chọn nước đi của thuật toán Hill Climbing bên trong. Thay vào đó, phương pháp tăng cơ hội tìm nghiệm bằng cách cho thuật toán bắt đầu lại từ nhiều khu vực khác nhau của không gian trạng thái.

4.2.2. Triển khai Random Restart cho các biến thể Hill Climbing

Trong notebook, Task 4 được hoàn thiện bằng hàm random_restart_hill_climbing(). Hàm nhận trạng thái ban đầu, tên thuật toán Hill Climbing, random seed, số lượt chạy tối đa và số bước tối đa của mỗi lượt tìm kiếm.

Chương trình hỗ trợ ba biến thể đã triển khai ở Chương 3 gồm Steepest-ascent, Stochastic (all) và First-choice.

Tại mỗi lượt thực hiện, thuật toán sử dụng bàn cờ hiện tại làm trạng thái đầu, thực hiện tối ưu hóa cục bộ và trả về giá trị hàm mục tiêu cuối cùng. Chương trình cộng dồn số bước được chấp nhận và số lần gọi conflicts() của tất cả các lượt chạy.

Khi tìm được trạng thái có f nhỏ hơn nghiệm tốt nhất đã ghi nhận, chương trình cập nhật best_board và best_f. Nếu best_f đạt 0, quá trình tìm kiếm kết thúc ngay lập tức.

Để bảo đảm khả năng tái lập, việc sinh trạng thái ngẫu nhiên sử dụng np.random.default_rng(seed). Các biến thể có lựa chọn ngẫu nhiên cũng được truyền seed tương ứng cho từng lượt thực hiện.

4.2.3. Giới hạn số lần khởi động và lưu nghiệm tốt nhất

Theo yêu cầu Task 4, chương trình được giới hạn tối đa 100 lượt thực hiện Hill Climbing, tính cả lần chạy đầu tiên.

Hai đại lượng được phân biệt như sau:

runs = số lượt thực hiện Hill Climbing.

restarts_used = runs − 1.

Do đó, khi max_starts = 100, số lần khởi động lại tối đa là 99.

Kết quả trả về gồm nghiệm tốt nhất, giá trị hàm mục tiêu, số lượt chạy, số lần khởi động lại, số bước cải thiện, số lần đánh giá conflicts(), thời gian thực thi và trạng thái kết thúc.

Nếu chương trình tìm được f = 0, trạng thái kết thúc là SOLVED. Ngược lại, nếu hết ngân sách mà chưa tìm được nghiệm, chương trình trả về RESTART_LIMIT. Trạng thái này chỉ phản ánh giới hạn thực nghiệm, không chứng minh bài toán không tồn tại lời giải.

4.3. Thuật toán Simulated Annealing

4.3.1. Nguyên lý hoạt động của Simulated Annealing

Simulated Annealing là phương pháp tìm kiếm ngẫu nhiên lấy cảm hứng từ quá trình làm nguội vật liệu. Điểm khác biệt quan trọng so với các thuật toán Hill Climbing truyền thống là Simulated Annealing có thể chấp nhận những bước di chuyển làm tăng giá trị hàm mục tiêu.

Ở giai đoạn nhiệt độ cao, thuật toán duy trì khả năng khám phá các trạng thái khác nhau và có thể rời khỏi một vùng cực tiểu cục bộ. Khi nhiệt độ giảm, xác suất chấp nhận các trạng thái xấu hơn cũng giảm, khiến quá trình tìm kiếm có xu hướng ổn định tại những nghiệm tốt hơn.

Trong bài toán n-Queens, thuật toán bắt đầu từ một cấu hình bàn cờ, sau đó sinh ngẫu nhiên một trạng thái lân cận bằng cách chuyển một quân hậu sang hàng khác trong cùng cột.

Chương trình duy trì hai trạng thái riêng biệt là trạng thái hiện tại và nghiệm tốt nhất từng được tìm thấy. Việc lưu nghiệm tốt nhất là cần thiết vì thuật toán có thể chấp nhận một nước đi làm tăng số cặp xung đột so với trạng thái trước đó.

4.3.2. Xác suất chấp nhận trạng thái xấu hơn

Gọi Δ là độ chênh lệch hàm mục tiêu giữa trạng thái mới và trạng thái hiện tại:

$\Delta = f(\text{mới}) - f(\text{hiện tại})$.

Quy tắc chấp nhận trạng thái trong notebook được xác định như sau.

Khi $\Delta < 0$, trạng thái mới được chấp nhận vì làm giảm số cặp xung đột. Khi $\Delta = 0$, chương trình cũng chấp nhận trạng thái mới để cho phép di chuyển trên những vùng có giá trị hàm mục tiêu bằng nhau.

Khi $\Delta > 0$, trạng thái mới chỉ được chấp nhận theo xác suất:

$P(\text{chấp nhận}) = \exp(-\Delta/T)$.

Trong đó T là nhiệt độ hiện tại của hệ thống.

Nếu Δ càng lớn thì xác suất chấp nhận càng nhỏ. Ngược lại, khi nhiệt độ T cao, cùng một bước di chuyển xấu hơn sẽ có khả năng được chấp nhận cao hơn.

Ví dụ, khi $\Delta = 1$ và $T = 3$:

$P = \exp(-1/3) \approx 0{,}7165$.

Khi nhiệt độ giảm xuống $T = 0{,}5$:

$P = \exp(-2) \approx 0{,}1353$.

Điều này cho thấy nhiệt độ có vai trò điều chỉnh mức độ chấp nhận các trạng thái kém hơn, giúp thuật toán cân bằng giữa khám phá không gian tìm kiếm và cải thiện chất lượng nghiệm.

4.3.3. Vai trò của nhiệt độ và tốc độ làm nguội

Trong Task 5, notebook sử dụng lịch giảm nhiệt dạng hàm mũ:

$T(t) = T_0 \times \alpha^t$

Với $T_0 > 0$ và $0 < \alpha < 1$.

Trong đó T_0 là nhiệt độ ban đầu, α là hệ số làm nguội và t là chỉ số vòng lặp.

Khi α gần bằng 1, nhiệt độ giảm chậm hơn, giúp thuật toán duy trì khả năng chấp nhận bước xấu trong thời gian dài. Ngược lại, khi α nhỏ, nhiệt độ giảm nhanh hơn và quá trình tìm kiếm sớm ưu tiên những bước không làm tăng hàm mục tiêu.

Bài thực hành sử dụng hai giá trị nhiệt độ đầu $T_0 = 0,5$ và $T_0 = 3,0$, kết hợp hai hệ số làm nguội $\alpha = 0,950$ và $\alpha = 0,995$.

Việc khảo sát những cấu hình này nhằm đánh giá ảnh hưởng của nhiệt độ đến số lần chấp nhận bước xấu, thời gian chạy và tỷ lệ tìm được nghiệm không xung đột.

4.3.4. Triển khai Simulated Annealing trên n-Queens

Task 5 được triển khai bằng hàm simulated_annealing_queens(), với tham số gồm trạng thái ban đầu, random seed, nhiệt độ T_0, hệ số α và số vòng lặp tối đa.

Trong mỗi vòng lặp, chương trình sinh một trạng thái lân cận hợp lệ, tính giá trị conflicts() của trạng thái đó và xác định Δ.

Nếu trạng thái mới làm giảm hoặc giữ nguyên hàm mục tiêu, nước đi được chấp nhận. Nếu trạng thái mới làm tăng hàm mục tiêu, chương trình sử dụng xác suất phụ thuộc nhiệt độ để quyết định có chấp nhận hay không.

Bên cạnh trạng thái hiện tại, thuật toán lưu lại nghiệm tốt nhất thông qua best_board và best_f. Khi tìm thấy f = 0, quá trình tìm kiếm kết thúc. Nếu đạt giới hạn vòng lặp trước khi tìm được nghiệm, trạng thái kết thúc là ITERATION_LIMIT.

Trong thiết lập thực nghiệm, số lượt sinh lân cận tối đa được đặt bằng 2.500. Số lần gọi conflicts() bằng số lượt thử lân cận cộng thêm một lần đánh giá trạng thái ban đầu.

Chương trình cũng lưu lại history_current và history_best để phục vụ trực quan hóa quá trình tìm kiếm.

4.4. Kết quả thực nghiệm

4.4.1. Kết quả Random Restart trên n = 4 và n = 8

Trên bàn cờ 4-Queens mẫu có f ban đầu bằng 1, cả ba biến thể Random Restart đều tìm được nghiệm f = 0 ngay trong lượt thực hiện đầu tiên.

Đối với bàn cờ 8-Queens mẫu có f ban đầu bằng 7, Random Restart – Steepest-ascent cần 7 lượt thực hiện, tương ứng 6 lần khởi động lại. Stochastic (all) tìm được nghiệm ngay trong lượt đầu, trong khi First-choice cần 7 lượt thực hiện.

Kết quả một lần chạy cho thấy số lượt khởi động có thể khác nhau đáng kể giữa các thuật toán, ngay cả khi chúng sử dụng chung bàn cờ ban đầu.

Bảng 4.1 tổng hợp kết quả từ 20 lượt chạy trên mỗi kích thước.

n	Thuật toán	Tỷ lệ thành công	Restart TB	Đánh giá TB
4	RR – Steepest-ascent	100%	1,85	78,45
4	RR – Stochastic (all)	100%	2,95	131,75
4	RR – First-choice	100%	2,00	108,90
8	RR – Steepest-ascent	100%	8,70	2.216,10
8	RR – Stochastic (all)	100%	5,80	2.129,20
8	RR – First-choice	100%	10,45	2.503,95

Bảng 4.1. Kết quả thực nghiệm Random Restart trên n = 4 và n = 8.

Cả ba biến thể Random Restart đều đạt tỷ lệ thành công 100% trong phạm vi 20 lượt thử của mỗi kích thước. Tuy nhiên, số lần khởi động lại và số lần đánh giá hàm mục tiêu có sự khác biệt.

Đặc biệt, trên n = 8, Stochastic (all) có số lần khởi động lại trung bình thấp nhất là 5,80 lần, trong khi First-choice cần trung bình 10,45 lần.

Điều này cho thấy việc lựa chọn chiến lược Hill Climbing bên trong Random Restart vẫn ảnh hưởng đáng kể đến tổng chi phí tìm kiếm.

4.4.2. Kết quả Simulated Annealing trên n = 4 và n = 8

Với cấu hình chuẩn $T_0 = 3,0$ và $\alpha = 0,995$, Simulated Annealing đạt tỷ lệ thành công 100% trên n = 4 và 95% trên n = 8.

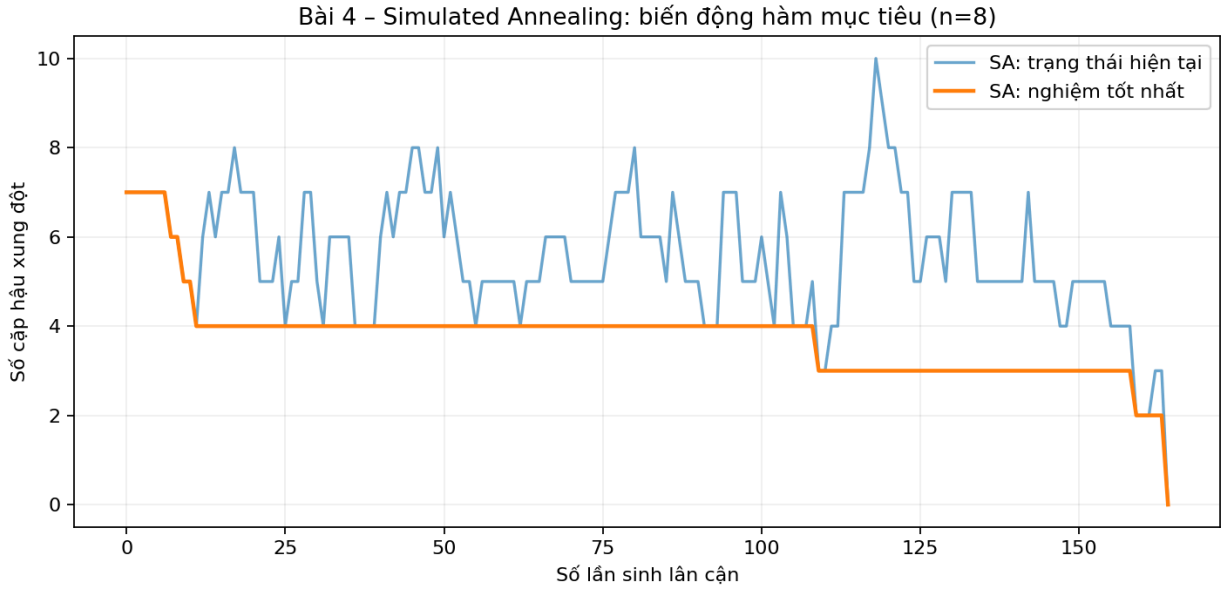
Ở n = 4, thuật toán cần trung bình 58,70 lần đánh giá hàm mục tiêu và có thời gian chạy trung bình khoảng 0,429 ms.

Ở n = 8, số lần đánh giá trung bình tăng lên 1.016,60 và thời gian chạy trung bình khoảng 11,016 ms.

Giá trị best_f trung bình trên n = 8 là 0,05. Điều này tương ứng với 19 lượt tìm được nghiệm không xung đột và một lượt chưa đạt f = 0 trong giới hạn thực nghiệm.

Biểu đồ hội tụ cho thấy hàm mục tiêu của trạng thái hiện tại có thể dao động trong quá trình tìm kiếm, trong khi nghiệm tốt nhất từng ghi nhận chỉ giảm hoặc giữ nguyên.

Trong ví dụ minh họa trên n = 8, số cặp xung đột của trạng thái hiện tại có thời điểm tăng lên 10, nhưng thuật toán vẫn tiếp tục khám phá và cuối cùng tìm được trạng thái có f = 0.



Hình 4.1. Biến động hàm mục tiêu hiện tại và nghiệm tốt nhất của Simulated Annealing trên bàn cờ 8-Queens.

Hình này minh họa trực tiếp khả năng chấp nhận bước xấu của Simulated Annealing, đồng thời giải thích vì sao chương trình cần lưu riêng nghiệm tốt nhất thay vì chỉ sử dụng trạng thái cuối cùng.

4.4.3. Khảo sát nhiệt độ ban đầu và tốc độ làm nguội

Để đánh giá ảnh hưởng của lịch nhiệt, thí nghiệm sử dụng bốn cấu hình kết hợp hai giá trị T_0 và hai hệ số α.

Các thuật toán được thực hiện trên cùng các trạng thái khởi tạo và cùng ngân sách tối đa 2.500 lượt thử.

T_0	α	Thành công n = 8	Bước xấu được chấp nhận TB
0,5	0,950	100%	0,35
0,5	0,995	90%	3,25
3,0	0,950	100%	5,55
3,0	0,995	95%	60,70

Bảng 4.2. Ảnh hưởng của nhiệt độ ban đầu và tốc độ làm nguội đến Simulated Annealing trên n = 8.

Kết quả cho thấy nhiệt độ ban đầu cao kết hợp với hệ số làm nguội lớn làm tăng đáng kể số bước xấu được chấp nhận.

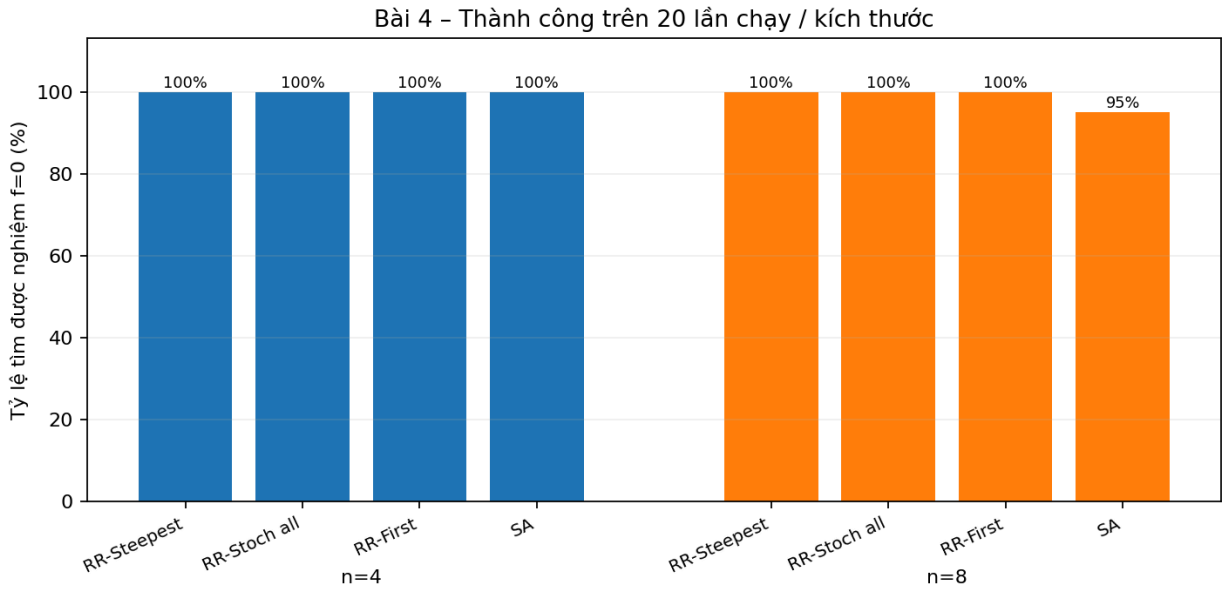
Với $T_0 = 3,0$ và $\alpha = 0,995$, thuật toán chấp nhận trung bình 60,70 bước xấu trong mỗi lượt chạy. Trong khi đó, ở $T_0 = 0,5$ và $\alpha = 0,950$, giá trị này chỉ là 0,35.

Tuy nhiên, cấu hình chấp nhận nhiều bước xấu nhất không đạt tỷ lệ thành công cao nhất trong thí nghiệm. Cụ thể, cấu hình $T_0 = 3,0$ và $\alpha = 0,995$ đạt 95%, thấp hơn tỷ lệ 100% của cấu hình $T_0 = 3,0$ và $\alpha = 0,950$.

Kết quả này cho thấy nhiệt độ cao và làm nguội chậm có thể tăng khả năng khám phá nhưng không bảo đảm tìm được nghiệm tốt hơn trong một ngân sách hữu hạn.

4.4.4. So sánh tỷ lệ thành công, số lần đánh giá và thời gian thực thi

Hình 4.2 tổng hợp tỷ lệ tìm được nghiệm $f = 0$ của ba biến thể Random Restart và Simulated Annealing trên hai kích thước bàn cờ.



Hình 4.2. So sánh tỷ lệ thành công của Random Restart và Simulated Annealing trên n = 4 và n = 8.

Trên n = 4, tất cả các phương pháp đều tìm được nghiệm ở cả 20 lượt chạy. Trên n = 8, các biến thể Random Restart tiếp tục đạt 100%, trong khi Simulated Annealing chuẩn đạt 95%.

Tuy nhiên, kết quả cần được xem xét cùng chi phí tính toán. Tại n = 8, Random Restart – Steepest-ascent sử dụng trung bình 2.216,10 lần đánh giá, Stochastic (all) sử dụng 2.129,20 lần, First-choice sử dụng 2.503,95 lần và Simulated Annealing sử dụng 1.016,60 lần.

Về thời gian chạy trung bình, các giá trị tương ứng lần lượt là khoảng 15,921 ms, 16,753 ms, 25,621 ms và 11,016 ms.

Như vậy, trong dữ liệu khảo sát, Simulated Annealing sử dụng ít lần đánh giá hơn và có thời gian trung bình thấp hơn ba biến thể Random Restart, nhưng chưa đạt tỷ lệ thành công tuyệt đối trên n = 8.

Cần lưu ý rằng hai phương pháp sử dụng ngân sách tìm kiếm khác nhau. Random Restart cho phép nhiều lượt Hill Climbing, còn Simulated Annealing chỉ thực hiện một chuỗi tìm kiếm với giới hạn số lần thử lân cận.

Do đó, không thể kết luận một phương pháp vượt trội hoàn toàn chỉ dựa trên tỷ lệ thành công hoặc thời gian thực thi. Việc đánh giá cần xét đồng thời chất lượng nghiệm, ngân sách, số lần đánh giá và điều kiện kết thúc.

4.5. Nhận xét

Qua bài thực hành, Random Restart và Simulated Annealing cho thấy hai cách tiếp cận khác nhau nhằm khắc phục hạn chế của các phương pháp Hill Climbing truyền thống.

Random Restart thay đổi trạng thái khởi tạo sau mỗi lần tìm kiếm chưa đạt nghiệm. Phương pháp này giúp mở rộng phạm vi khảo sát của thuật toán và giảm sự phụ thuộc vào một cực tiểu cục bộ cụ thể. Trong các thí nghiệm trên n = 4 và n = 8, cả

ba biến thể Random Restart đều đạt tỷ lệ thành công 100% khi được phép thực hiện tối đa 100 lượt tìm kiếm.

Tuy nhiên, số lần khởi động lại và số lần đánh giá hàm mục tiêu tăng đáng kể khi kích thước bàn cờ lớn hơn. Điều này cho thấy khả năng tìm nghiệm tốt có thể đi kèm chi phí tính toán cao.

Simulated Annealing sử dụng một cơ chế khác khi cho phép chấp nhận những bước di chuyển làm tăng hoặc giữ nguyên số cặp xung đột. Khả năng này giúp thuật toán tiếp tục khám phá ngay cả khi các bước cải thiện trực tiếp không còn dễ tìm thấy.

Biểu đồ hội tụ minh họa sự dao động của trạng thái hiện tại, đồng thời cho thấy giá trị nghiệm tốt nhất luôn được bảo toàn. Đây là điểm khác biệt quan trọng so với Hill Climbing chỉ chấp nhận các bước giảm hàm mục tiêu.

Kết quả khảo sát nhiệt độ cho thấy nhiệt độ ban đầu và tốc độ làm nguội ảnh hưởng rõ rệt đến số lần chấp nhận bước xấu. Tuy nhiên, việc duy trì nhiệt độ cao lâu hơn không đồng nghĩa với tỷ lệ thành công cao hơn trong mọi trường hợp.

Với cấu hình chuẩn $T_0 = 3{,}0$ và $\alpha = 0{,}995$, Simulated Annealing đạt 100% thành công trên $n = 4$ và 95% trên $n = 8$. Phương pháp này sử dụng ít lần đánh giá hàm mục tiêu hơn các biến thể Random Restart trong dữ liệu khảo sát, nhưng vẫn còn trường hợp chưa tìm được nghiệm trong ngân sách quy định.

Nhìn chung, bài thực hành cho thấy hiệu quả của tìm kiếm cục bộ phụ thuộc không chỉ vào chiến lược lựa chọn trạng thái lân cận mà còn vào khả năng thoát khỏi cực trị cục bộ, cách kiểm soát ngẫu nhiên và ngân sách tính toán.

Random Restart phù hợp khi có thể thực hiện nhiều lượt tìm kiếm từ các điểm xuất phát khác nhau, trong khi Simulated Annealing cung cấp khả năng khám phá linh hoạt thông qua quy tắc chấp nhận trạng thái dựa trên nhiệt độ.

Những kết quả này là cơ sở cho Chương 5, nơi các lịch giảm nhiệt độ trong Simulated Annealing được khảo sát chi tiết hơn nhằm làm rõ tác động đến quá trình hội tụ và chất lượng nghiệm.

CHƯƠNG 5: Khảo sát lịch giảm nhiệt độ

5.1. Vai trò của lịch giảm nhiệt độ trong Simulated Annealing

Trong Chương 4, thuật toán Simulated Annealing (SA) đã được sử dụng để tìm cấu hình n quân hậu không xung đột bằng cách cho phép chấp nhận một số nước di chuyển làm tăng giá trị hàm mục tiêu. Khác với Hill Climbing chỉ chấp nhận những bước cải thiện nghiêm ngặt, SA sử dụng nhiệt độ T để kiểm soát mức độ chấp nhận các trạng thái kém hơn. Vì vậy, cách thay đổi nhiệt độ theo thời gian là thành phần quan trọng quyết định hành vi khám phá và hội tụ của thuật toán.

Đối với bài toán n-Queens, mỗi trạng thái là một vector n phần tử thể hiện vị trí hàng của từng quân hậu theo cột. Hàm mục tiêu f(board) = conflicts(board) đếm số cặp quân hậu tấn công nhau; nghiệm hợp lệ được xác định khi f(board) = 0. Xét độ chênh lệch Δ = f(mới) − f(hiện tại), các nước đi có $\Delta \leq 0$ được chấp nhận, còn nước đi xấu hơn ($\Delta > 0$) được chấp nhận với xác suất:

$$P(\text{chấp nhận} \mid \Delta, T) = \exp(-\Delta/T), \text{ với } T > 0.$$

Khi nhiệt độ cao, thuật toán có thể di chuyển qua những trạng thái có nhiều xung đột hơn, góp phần tránh mắc kẹt tại cực tiểu cục bộ. Khi nhiệt độ giảm, xác suất chấp nhận bước xấu nhỏ dần và quá trình tìm kiếm có xu hướng tập trung vào những trạng thái đã có chất lượng tốt. Nếu nhiệt độ bằng 0, bước xấu không được chấp nhận trong cách triển khai của notebook. Lịch giảm nhiệt, hay cooling schedule, chính là hàm xác định T(t) tại vòng lặp t.

Bài thực hành sử dụng notebook Simulated_Annealing_Schedules.ipynb để khảo sát bốn lịch Exponential, Linear, Fast và Logarithmic. Bên cạnh việc vẽ đồ thị T(t), chương trình áp dụng cùng cơ chế sinh lân cận của n-Queens để so sánh số lần thử

nước đi, giá trị nghiệm tốt nhất, tỷ lệ thành công và thời gian thực thi. Nội dung chương chỉ diễn giải các kết quả thu được trong phạm vi thí nghiệm đã thiết lập, không mặc nhiên xem một lịch nhiệt là tối ưu trong mọi điều kiện.

5.2. Các phương pháp giảm nhiệt độ

5.2.1. Lịch giảm nhiệt hàm mũ (Exponential)

Hàm temperature_exponential(time, initial_temperature, alpha) trong notebook xác định nhiệt độ theo công thức:

$T(t) = T_0 \times \alpha^t$, với $T_0 > 0$ và $0 < \alpha < 1$.

Trong đó, T_0 là tham số nhiệt độ ban đầu, α là hệ số giảm nhiệt và t là số thứ tự vòng lặp. Khi α càng gần 1, nhiệt độ giảm càng chậm. Ngược lại, giá trị α nhỏ làm nhiệt độ giảm mạnh ngay từ những vòng đầu. Nhiệt độ của lịch hàm mũ luôn dương tại mọi vòng lặp hữu hạn nếu T_0 dương, nhưng tiến dần đến 0 khi số vòng lặp tăng.

Trong hình minh họa bốn lịch nhiệt, với $T_0 = 3$ và $\alpha = 0,97$, nhiệt độ tại t = 1 bằng 2,91, tại t = 50 còn khoảng 0,6542 và tại t = 120 còn khoảng 0,0776. Đây là lịch được sử dụng trong Task 5 của Bài 4, nhưng thí nghiệm so sánh n-Queens ở Bài 5 dùng $\alpha = 0,995$, do đó đường nhiệt khi chạy thuật toán giảm chậm hơn so với hình khảo sát công thức.

5.2.2. Lịch giảm nhiệt tuyến tính (Linear)

Hàm temperature_linear(time, initial_temperature, eta) được định nghĩa bởi:

$T(t) = T_0 - \eta t$, với $\eta > 0$.

Tham số η xác định độ giảm nhiệt độ cố định tại mỗi bước. Với $T_0 = 3$ và $\eta = 0{,}025$, nhiệt độ giảm từ 2,975 tại $t = 1$ xuống 1,75 tại $t = 50$, còn 0,5 tại $t = 100$ và bằng 0 tại $t = 120$. So với lịch hàm mũ, cách giảm tuyến tính dễ dự đoán thời điểm nhiệt độ đạt 0 thông qua tỷ số T_0/η.

Cần phân biệt công thức gốc trong notebook với cách sử dụng khi tìm kiếm: temperature_linear() có thể trả về nhiệt độ âm nếu t vượt T_0/η. Trong hàm cooling_temperature() của phần thực nghiệm SA, giá trị nhiệt độ được chặn dưới ở 0 theo $T = \max(0, T_0 - \eta t)$, nhằm tránh áp dụng xác suất chấp nhận ở nhiệt độ âm. Vì vậy, sau thời điểm đạt 0, thuật toán không còn chấp nhận bước xấu.

5.2.3. Lịch giảm nhiệt nhanh (Fast)

Hàm temperature_fast(time, initial_temperature) trong notebook thực hiện:

$T(t) = T_0/(1 + t)$.

Đây là quy tắc giảm nhiệt theo tỉ lệ nghịch với thời gian, không sử dụng hệ số α hoặc η. Với $T_0 = 3$, nhiệt độ ở $t = 1$ chỉ còn 1,5; tại $t = 50$ giảm xuống khoảng 0,0588 và tại $t = 120$ còn khoảng 0,0248. Như vậy, so với những cấu hình Exponential và Linear đã chọn, Fast làm giảm mạnh xác suất chấp nhận bước xấu ngay ở giai đoạn đầu.

Đặc điểm này khiến thuật toán nhanh chóng ưu tiên những trạng thái không làm tăng số xung đột. Tuy nhiên, giảm nhiệt nhanh cũng làm giảm cơ hội đi qua các vùng phải tạm thời chấp nhận bước xấu để thoát khỏi cực tiểu cục bộ. Hiệu quả thực tế vì vậy cần được kiểm chứng bằng nhiều lần chạy, thay vì đánh giá chỉ từ tốc độ giảm nhiệt của công thức.

5.2.4. Lịch giảm nhiệt Logarithmic

Hàm temperature_logarithmic(time, initial_temperature) được định nghĩa với $t > 0$:

$T(t) = T_0/\ln(1 + t)$.

Lịch logarithmic có tốc độ giảm chậm ở giai đoạn muộn. Với $T_0 = 3$, các giá trị tại $t = 1$, $t = 50$ và $t = 120$ lần lượt xấp xỉ 4,3281; 0,7630 và 0,6255. Do đó, ngay cả khi số vòng lặp đã lớn, lịch này vẫn duy trì nhiệt độ đủ để một số nước đi xấu hơn có cơ hội được chấp nhận.

Một chi tiết quan trọng trong notebook là tất cả thí nghiệm bổ sung sử dụng t bắt đầu từ 1 để tránh trường hợp mẫu số $\ln(1 + 0) = 0$. Đồng thời, $T(1) = T_0/\ln 2$ lớn hơn T_0, nên T_0 ở đây là hệ số của công thức chứ không phải nhiệt độ đúng bằng tại bước đầu. Tương tự, lịch Fast bắt đầu tại $T_0/2$ khi $t = 1$. Vì vậy, "dùng cùng T_0" giữa các lịch có nghĩa là dùng chung tham số công thức, không đồng nghĩa nhiệt độ thực tế của bước đầu giống nhau.

5.3. Khảo sát ảnh hưởng của tham số

5.3.1. Ảnh hưởng của nhiệt độ ban đầu T_0

Tham số T_0 quyết định thang nhiệt độ tổng thể của bốn lịch. Khi các thông số khác giữ nguyên, tăng T_0 thường làm tăng xác suất chấp nhận một nước đi xấu hơn có cùng giá trị Δ. Với lịch hàm mũ, đồ thị độ nhạy trong notebook so sánh $T_0 = 0,5$; 1,5 và 3,0 khi cố định $\alpha = 0,97$, cho thấy các đường nhiệt có cùng hình dạng giảm tương đối nhưng nằm ở những mức nhiệt độ khác nhau.

Thí nghiệm SA bổ sung trên 8-Queens thử $T_0 = 0,5$ và $T_0 = 3,0$. Với Exponential và $\alpha = 0,995$, số bước xấu được chấp nhận trung bình tăng từ 2,875 lên 64,000 khi T_0 tăng. Tuy nhiên, tỷ lệ tìm được nghiệm giảm từ 87,5% xuống 62,5% trong tập tám lượt chạy mỗi cấu hình. Kết quả cho thấy khả năng chấp nhận nhiều bước xấu hơn không tự động chuyển thành chất lượng nghiệm tốt hơn trong ngân sách hữu hạn.

5.3.2. Ảnh hưởng của hệ số alpha

Đối với lịch Exponential, tham số α quyết định tốc độ suy giảm theo hàm mũ. Hình khảo sát so sánh $\alpha = 0,90$; 0,97 và 0,995 với $T_0 = 3$: $\alpha = 0,90$ làm nhiệt độ tiến gần 0 khá sớm, $\alpha = 0,97$ giảm ở mức trung gian, còn $\alpha = 0,995$ duy trì nhiệt độ cao hơn trong toàn bộ khoảng $t = 1$ đến 120.

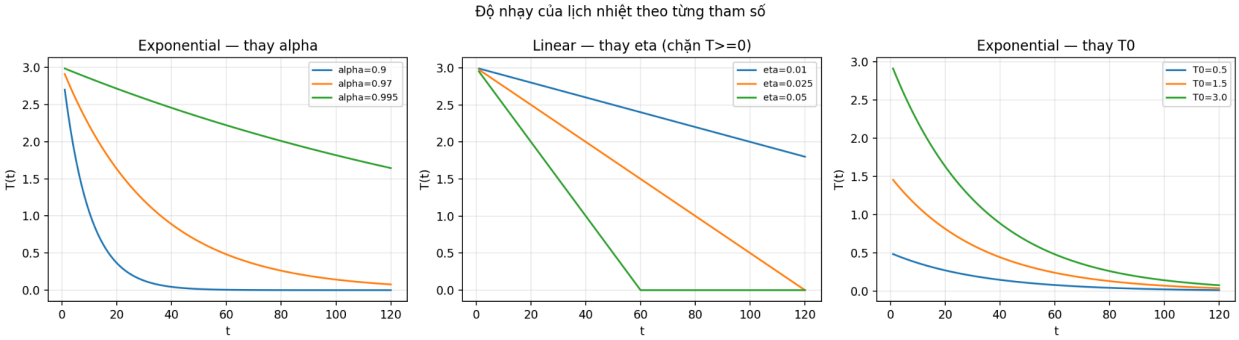
Trong phép thử độ nhạy trên $n = 8$, khi $T_0 = 3,0$, tăng α từ 0,970 lên 0,995 làm số bước xấu được chấp nhận trung bình tăng từ 10,50 lên 64,00. Tỷ lệ thành công quan sát tương ứng là 75,0% và 62,5%. Đây là kết quả của tám lượt chạy cho mỗi cấu hình, phản ánh sự đánh đổi giữa thăm dò rộng hơn và việc tận dụng ngân sách để hội tụ, chưa đủ để khẳng định một hệ số α luôn vượt trội.

5.3.3. Ảnh hưởng của hệ số eta

Đối với lịch Linear, η xác định mức giảm nhiệt theo mỗi vòng lặp. Đồ thị tham số so sánh $\eta = 0,010$; 0,025 và 0,050 khi $T_0 = 3$. Với $\eta = 0,050$, nhiệt độ đạt 0 ở $t = 60$; với $\eta = 0,025$, nhiệt độ đạt 0 ở $t = 120$; còn $\eta = 0,010$ vẫn giữ $T = 1,8$ tại $t = 120$. Như vậy, η càng lớn thì hệ thống càng nhanh dừng việc chấp nhận các bước xấu hơn.

Trong thí nghiệm 8-Queens với $T_0 = 3,0$, khi η tăng từ 0,002 lên 0,006, số bước xấu được chấp nhận trung bình giảm từ 250,00 xuống 86,50, trong khi tỷ lệ thành công

tăng từ 50,0% lên 75,0%. Đối với $T_0 = 0,5$, hai tỷ lệ tương ứng là 75,0% và 87,5%. Kết quả cho thấy tác động của η cần được xem xét cùng T_0, giới hạn 1.500 lượt thử và cấu hình dữ liệu đầu vào.



Hình 5.1. Độ nhạy của lịch giảm nhiệt theo các tham số α, η và T_0.

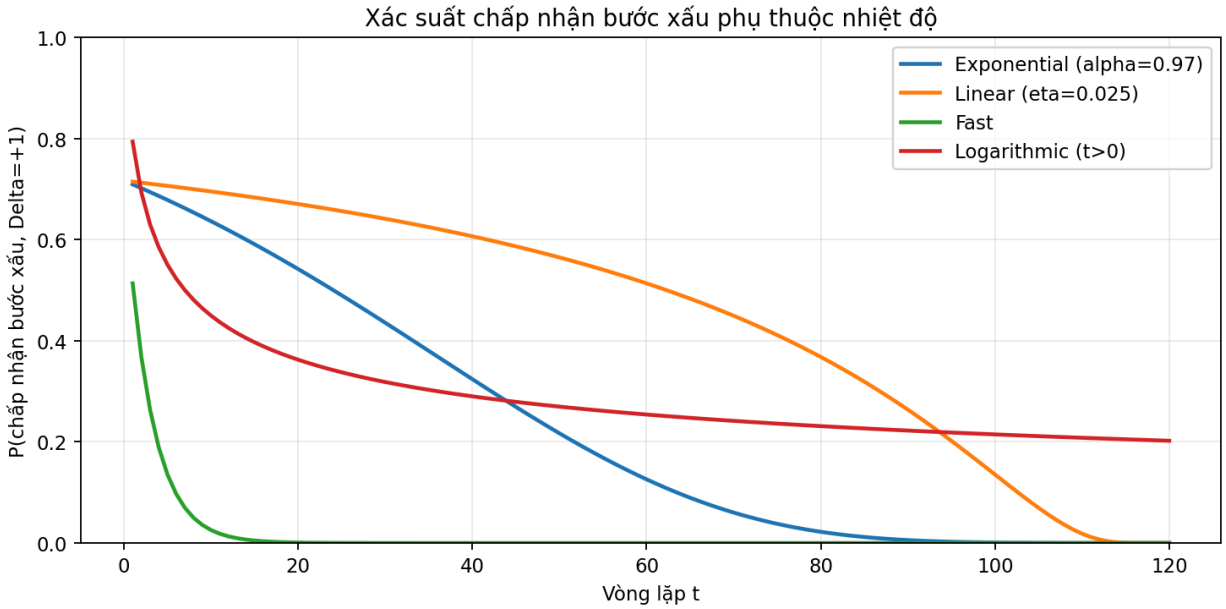
5.3.4. So sánh xác suất chấp nhận bước xấu theo thời gian

Để so sánh trực tiếp khả năng chấp nhận một nước đi xấu hơn, notebook dùng $\Delta = +1$ và tính $p(t) = \exp[-1/T(t)]$ cho bốn lịch với $T_0 = 3$, $\alpha = 0,97$ và $\eta = 0,025$. Khi nhiệt độ nhỏ dần, xác suất này cũng giảm; riêng lịch Linear đạt xác suất 0 sau khi nhiệt độ bị chặn ở 0.

Quan sát đồ thị cho thấy Fast giảm xác suất mạnh nhất trong giai đoạn sớm, trong khi Linear duy trì xác suất tương đối cao ở phần lớn khoảng đầu và giữa. Exponential giảm nhanh hơn Linear trong cấu hình đang xét. Logarithmic có xác suất cao tại những bước đầu và tiếp tục duy trì xác suất lớn hơn đáng kể so với Exponential và Fast ở các vòng cuối.

Tuy nhiên, thứ tự giữa Linear và Logarithmic không cố định theo toàn bộ thời gian: Linear duy trì nhiệt độ cao hơn ở nhiều vòng đầu, rồi giảm về 0; Logarithmic giảm chậm và tiếp tục cho phép chấp nhận bước xấu về sau. Do đó, kết luận "lịch nào

giữ khả năng chấp nhận lâu hơn" phụ thuộc vào thời điểm khảo sát và bộ tham số cụ thể, không chỉ phụ thuộc vào tên của lịch nhiệt.



Hình 5.2. Xác suất chấp nhận bước xấu có Δ = +1 theo vòng lặp của bốn lịch nhiệt.

5.4. Kết quả thực nghiệm

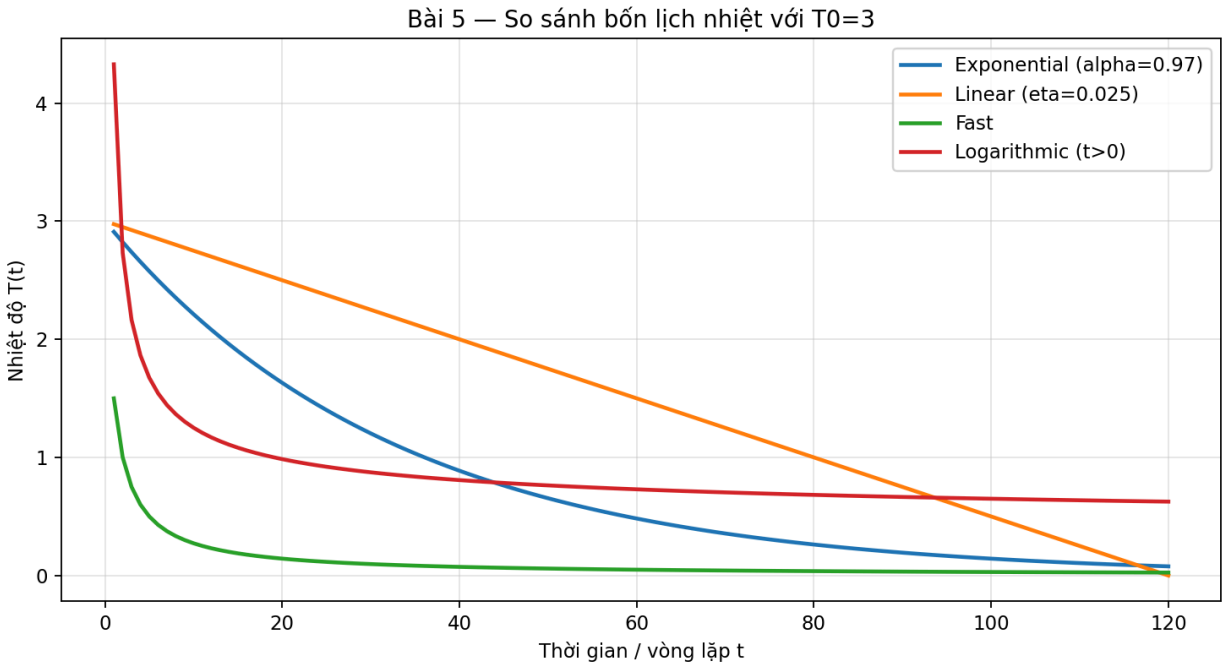
5.4.1. Biểu đồ so sánh bốn lịch giảm nhiệt độ

Để khảo sát các công thức nhiệt độ, notebook sử dụng cùng tham số $T_0 = 3$, đặt $\alpha = 0,97$ cho Exponential, $\eta = 0,025$ cho Linear và lấy $t = 1, 2, \ldots, 120$. Những giá trị đặc trưng được tổng hợp trong Bảng 5.1.

Bảng 5.1. Giá trị nhiệt độ của bốn lịch tại một số vòng lặp ($T_0 = 3$, $\alpha = 0,97$, $\eta = 0,025$).

Lịch nhiệt	T(1)	T(10)	T(50)	T(100)	T(120)

Exponential	2,9100	2,2123	0,6542	0,1427	0,0776
Linear	2,9750	2,7500	1,7500	0,5000	0,0000
Fast	1,5000	0,2727	0,0588	0,0297	0,0248
Logarithmic	4,3281	1,2511	0,7630	0,6500	0,6255



Hình 5.3. So sánh bốn đường nhiệt độ T(t) trong khoảng t = 1 đến 120.

Hình so sánh và Bảng 5.1 thể hiện rõ sự khác biệt về tốc độ giảm nhiệt. Fast có nhiệt độ thấp nhất tại nhiều thời điểm khảo sát, Exponential tiếp tục giảm về gần 0, Linear duy trì mức nhiệt khá cao trước khi chạm 0 và Logarithmic vẫn giữ nhiệt độ đáng kể ở các vòng muộn. Các đường này chỉ mô tả lịch nhiệt đã chọn, chưa phản ánh trực tiếp tỷ lệ tìm lời giải của SA.

5.4.2. So sánh chất lượng nghiệm khi áp dụng các lịch nhiệt cho n-Queens

Để khảo sát chất lượng nghiệm, chương trình sử dụng biểu diễn n-Queens giống Task 5 của Bài 4: mỗi cột có đúng một quân hậu, nước đi lân cận chuyển một quân hậu sang hàng khác trong cùng cột và hàm mục tiêu đếm số cặp hậu xung đột. Mỗi lịch chạy 20 lượt trên n = 4 và 20 lượt trên n = 8. Trong cùng một lượt, bốn lịch nhận cùng bàn cờ khởi tạo và cùng seed tìm kiếm, giúp giảm chênh lệch do điều kiện đầu vào.

Cấu hình thực nghiệm SA khác với cấu hình vẽ đường nhiệt ở mục 5.4.1: $T_0 = 3$, $\alpha = 0{,}995$ và $\eta = 0{,}002$; ngân sách tối đa là 1.500 lần sinh lân cận. Bài toán được xem là thành công khi nghiệm tốt nhất từng gặp đạt best_f = 0, không nhất thiết chỉ xét trạng thái hiện tại ở cuối lượt chạy. Kết quả được ghi nhận trong Bảng 5.2.

Bảng 5.2. So sánh Simulated Annealing theo bốn lịch nhiệt; mỗi cấu hình gồm 20 lần chạy.

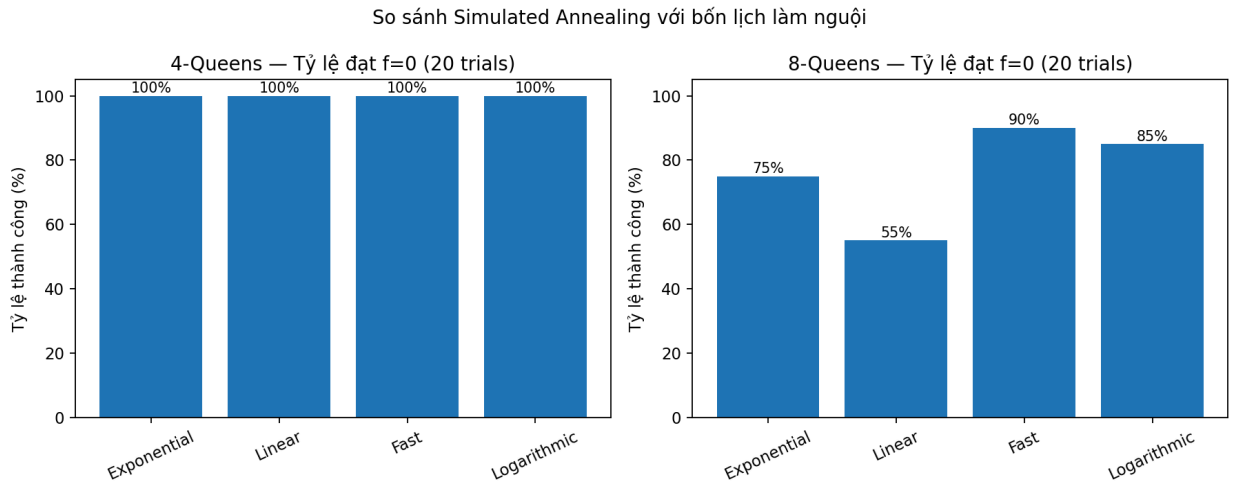
n	Lịch nhiệt	Thành công	best_f TB	Lần đánh giá TB	Bước xấu chấp nhận TB	Thời gian TB (ms)
4	Exponential	100%	0,00	98,55	20,90	2,42
4	Linear	100%	0,00	69,55	18,70	1,70
4	Fast	100%	0,00	46,05	0,70	1,18
4	Logarithmic	100%	0,00	63,70	8,05	1,67
8	Exponential	75%	0,25	943,05	62,70	23,85
8	Linear	55%	0,45	1.266,35	225,15	32,04
8	Fast	90%	0,10	422,00	0,55	10,38

8	Logarithmic	85%	0,15	770,65	39,40	20,16

Với n = 4, cả bốn lịch đều tìm được nghiệm không xung đột trong cả 20 lượt chạy. Tuy nhiên, chi phí tìm kiếm khác nhau: Fast sử dụng trung bình 46,05 lần đánh giá hàm mục tiêu, thấp hơn ba lịch còn lại. Khi tăng lên n = 8, tỷ lệ thành công có sự phân hóa rõ rệt. Fast đạt 90% (18/20 lượt), Logarithmic đạt 85% (17/20), Exponential đạt 75% (15/20), còn Linear đạt 55% (11/20). Giá trị best_f trung bình lần lượt là 0,10; 0,15; 0,25 và 0,45. Các số liệu này phản ánh chất lượng nghiệm quan sát trong thí nghiệm, không phải mức độ bảo đảm thành công của từng phương pháp.

5.4.3. So sánh tỷ lệ thành công và thời gian thực thi

Tỷ lệ thành công trên 4-Queens đều bằng 100%, vì thế khó đánh giá sự khác biệt chất lượng giữa các lịch nhiệt chỉ dựa trên kích thước này. Ngược lại, các kết quả trên 8-Queens cho thấy Fast có đồng thời tỷ lệ thành công cao nhất và thời gian chạy trung bình thấp nhất trong bộ thử nghiệm, khoảng 10,38 ms. Linear có tỷ lệ thành công thấp nhất và thời gian chạy trung bình lớn nhất, khoảng 32,04 ms.



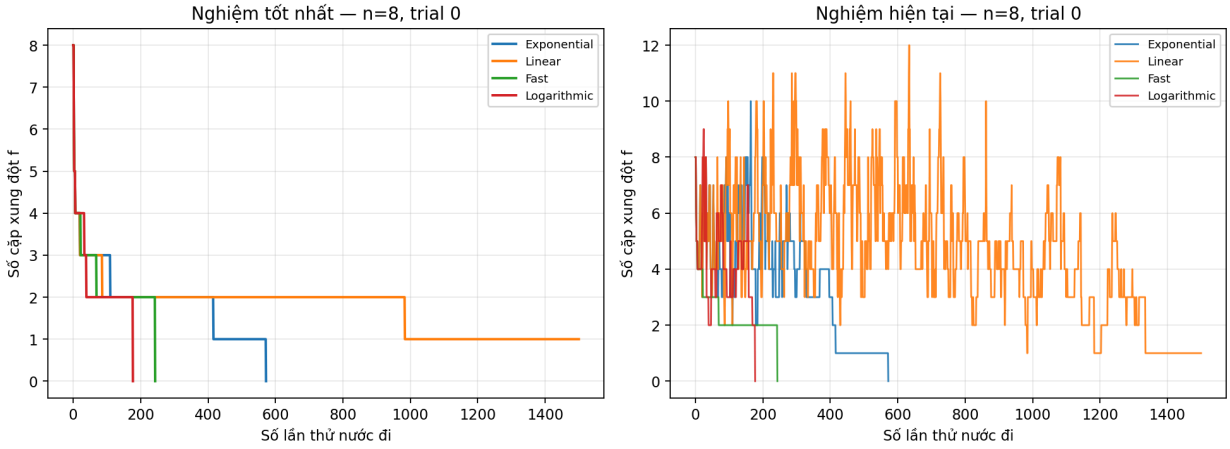
Hình 5.4. Tỷ lệ thành công của Simulated Annealing với bốn lịch giảm nhiệt trên 4-Queens và 8-Queens, mỗi cấu hình 20 lượt.

Sự chênh lệch thời gian cần được diễn giải cùng số lần lặp. Trong notebook, quá trình tìm kiếm dừng ngay khi best_f đạt 0 hoặc khi hết 1.500 lần thử lân cận. Các lượt không tìm được nghiệm thường sử dụng hết ngân sách. Vì vậy, thời gian trung bình phụ thuộc đồng thời vào lịch nhiệt, tốc độ tìm nghiệm và số lượt chạm giới hạn, chứ không chỉ vào chi phí tính nhiệt độ ở mỗi vòng.

Thí nghiệm cũng cho thấy Linear chấp nhận trung bình 225,15 bước xấu trên n = 8, cao hơn rõ so với Fast (0,55), nhưng có tỷ lệ thành công thấp hơn. Điều này củng cố nhận định rằng duy trì khả năng khám phá cao không mặc nhiên đồng nghĩa với việc tìm được nghiệm tốt trong một số vòng thử cố định. Để đánh giá khả năng khái quát, cần lặp thí nghiệm với nhiều seed, ngân sách và kích thước khác; thời gian tuyệt đối còn phụ thuộc cấu hình máy chạy.

5.4.4. Trực quan hóa quá trình hội tụ

Hình hội tụ được xây dựng từ một lượt thử minh họa trên n = 8, dùng cùng bàn cờ khởi tạo cho cả bốn lịch. Hình bên trái ghi lại giá trị best_f, tức số cặp xung đột thấp nhất từng được tìm thấy; hình bên phải biểu diễn current_f của trạng thái đang được SA duy trì tại mỗi thời điểm.



Hình 5.5. So sánh quá trình hội tụ của nghiệm tốt nhất và trạng thái hiện tại trên một bàn cờ 8-Queens.

Trên biểu đồ best_f, các đường không tăng vì chương trình chỉ cập nhật nghiệm tốt nhất khi tìm thấy giá trị f nhỏ hơn. Trong lượt minh họa này, Exponential, Fast và Logarithmic đều đạt best_f = 0 trước khi hết ngân sách; Linear giảm đến best_f = 1 nhưng chưa tìm được nghiệm không xung đột sau 1.500 lượt thử. Đây là diễn biến của một seed minh họa, không được sử dụng thay cho bảng kết quả nhiều lần chạy.

Trong biểu đồ current_f, giá trị xung đột có thể tăng hoặc giảm qua các lượt, đặc biệt dưới những lịch duy trì nhiệt độ đủ lớn để chấp nhận các bước xấu. So sánh hai biểu đồ cho thấy tại một thời điểm, trạng thái hiện tại có thể kém hơn nghiệm tốt nhất đã ghi nhận. Vì vậy, lưu riêng best_board và current_board là cần thiết để bảo đảm thuật toán trả lại nghiệm tốt nhất từng tìm thấy, ngay cả khi kết thúc ở một trạng thái chưa tốt bằng.

5.5. Nhận xét

Qua bài thực hành, lịch giảm nhiệt độ được xác định là yếu tố điều chỉnh trực tiếp xác suất chấp nhận trạng thái xấu hơn của Simulated Annealing. Exponential giảm

nhiệt theo lũy thừa và phụ thuộc vào hệ số α; Linear giảm một lượng cố định η mỗi vòng; Fast giảm theo tỉ lệ nghịch 1/(1+t); trong khi Logarithmic giảm chậm hơn ở những vòng muộn. Các quy tắc này tạo ra mức độ khám phá và hội tụ khác nhau, dù chúng được áp dụng cho cùng bài toán và cùng toán tử lân cận.

Khảo sát tham số cho thấy tăng T_0 hoặc tăng α trong lịch hàm mũ thường làm tăng thời gian duy trì nhiệt độ cao, còn tăng η trong lịch tuyến tính khiến nhiệt độ giảm nhanh hơn. Ảnh hưởng đó thể hiện rõ qua số lần chấp nhận bước xấu trong thí nghiệm 8-Queens. Tuy nhiên, số bước xấu được chấp nhận nhiều hơn không bảo đảm tỷ lệ đạt nghiệm cao hơn; một chiến lược khám phá rộng có thể sử dụng nhiều ngân sách cho các trạng thái chưa cải thiện đáng kể.

Trong bộ dữ liệu gồm 20 lần chạy cho mỗi lịch và mỗi kích thước, cả bốn lịch đạt 100% trên 4-Queens. Trên 8-Queens, Fast đạt tỷ lệ thành công cao nhất là 90%, giá trị best_f trung bình nhỏ nhất là 0,10 và thời gian chạy trung bình thấp nhất là khoảng 10,38 ms. Linear đạt 55% và có số bước xấu được chấp nhận lớn nhất. Kết quả này chỉ phản ánh các tham số, seed và ngân sách 1.500 lượt thử đã sử dụng; không đủ cơ sở kết luận Fast là lịch giảm nhiệt tốt nhất trong mọi bài toán hoặc mọi điều kiện thực thi.

Một lưu ý phương pháp là việc đặt cùng tham số T_0 không tạo ra cùng nhiệt độ thực tế tại vòng đầu của tất cả lịch nhiệt khi bắt đầu t = 1. Đặc biệt, Fast có $T(1) = T_0/2$ còn Logarithmic có $T(1) = T_0/\ln 2$. Bên cạnh đó, việc chặn nhiệt độ tuyến tính ở 0 và cơ chế dừng khi gặp best_f = 0 đều có thể ảnh hưởng đến số lần đánh giá cũng như thời gian thực nghiệm. Vì vậy, các chi tiết triển khai phải được mô tả rõ khi đối chiếu thuật toán.

Nhìn chung, Bài 5 cho thấy sự đánh đổi giữa khả năng khám phá không gian trạng thái và tốc độ hội tụ của Simulated Annealing. Chọn lịch nhiệt phù hợp cần dựa

trên chất lượng nghiệm, tỷ lệ thành công, số lần đánh giá và ngân sách tính toán, thay vì chỉ xét nhiệt độ giảm nhanh hay chậm. Những kết quả này bổ sung cho Chương 4 và tạo cơ sở đánh giá toàn diện hơn các thuật toán tìm kiếm cục bộ trong phần tổng hợp của Lab 03.

CHƯƠNG 6: Tối ưu hàm đếm xung đột bằng Numba

6.1. Bài toán tối ưu hiệu suất tính toán

6.1.1. Vai trò của hàm đếm xung đột trong n-Queens

Trong các bài thực hành trước, bài toán n quân hậu (n-Queens) được mô hình hóa dưới dạng một vector có n phần tử, trong đó chỉ số phần tử biểu diễn cột và giá trị phần tử biểu diễn hàng đặt quân hậu. Do mỗi cột có đúng một quân hậu, bài toán chỉ cần kiểm tra các xung đột theo hàng và hai hướng đường chéo. Hàm mục tiêu f(board) = conflicts(board) trả về số cặp quân hậu có thể tấn công nhau. Mục tiêu của các thuật toán tìm kiếm cục bộ là làm giảm f(board), và một nghiệm hợp lệ thỏa mãn f(board) = 0.

Trong Steepest-ascent và Stochastic Hill Climbing, chương trình liên tục đánh giá giá trị hàm mục tiêu trên các trạng thái lân cận. Với toán tử chuyển một quân hậu sang hàng khác trong cùng cột, một trạng thái có $n(n-1)$ lân cận. Khi số quân hậu hoặc số lần tìm kiếm tăng, việc tính lại số cặp xung đột trở thành một thành phần đáng kể của chi phí thực thi. Vì thế, tối ưu hàm conflicts() có thể cải thiện hiệu suất toàn bộ quá trình tìm kiếm mà không cần thay đổi nguyên tắc lựa chọn trạng thái của các thuật toán.

Bài thực hành sử dụng notebook n_queens_fast_conflict_calculation_with_numba.ipynb để đối chiếu ba cách triển khai: conflicts_naive(), conflicts() và conflicts_jit(). Phần khảo sát tập trung vào

tính đúng đắn của kết quả, thời gian thực thi trên nhiều kích thước bàn cờ và sự khác nhau giữa tối ưu thuật toán với tối ưu mã chạy bằng Numba.

6.1.2. Hạn chế của phương pháp đếm xung đột trực tiếp

Phương pháp trực tiếp có thể đếm xung đột bằng cách xét từng cặp quân hậu và kiểm tra xem hai quân có cùng hàng hoặc cùng đường chéo hay không. Với n quân hậu, tổng số cặp có thể hình thành là $n(n-1)/2$; do vậy, số phép so sánh tăng theo bậc $O(n^2)$. Trong notebook, hàm conflicts_naive() đếm các cặp theo hàng bằng cách duyệt từng hàng, đồng thời dùng vòng lặp lồng nhau để kiểm tra xung đột đường chéo theo các cột phía bên phải.

Khi n nhỏ, phương pháp này thuận tiện để đọc và kiểm chứng. Tuy nhiên, với n lớn, số phép so sánh tăng nhanh khiến việc gọi hàm mục tiêu nhiều lần trở nên tốn kém. Đây là hạn chế chính được dùng làm mốc so sánh với phương pháp sử dụng bộ đếm và phiên bản Numba JIT.

6.2. Các phương pháp tính số cặp xung đột

6.2.1. Phương pháp Naive với độ phức tạp $O(n^2)$

Hàm conflicts_naive(board) nhận vector vị trí quân hậu và trả về tổng số cặp xung đột. Đối với xung đột cùng hàng, hàm đếm số quân trên từng hàng, sau đó tính số cặp tương ứng. Đối với xung đột đường chéo, hàm duyệt một quân hậu tại cột j và lần lượt đối chiếu với những quân ở các cột jj > j. Cách duyệt này bảo đảm một cặp chỉ được xét một lần khi kiểm tra đường chéo.

Vì thao tác đếm trên n hàng và các vòng lặp đối chiếu đường chéo đều có tổng số phép xử lý bậc hai theo n, thời gian thực thi của phương pháp Naive có độ phức tạp

$O(n^2)$. Đây là độ phức tạp của phép tính số xung đột cho một bàn cờ, không phải độ phức tạp của toàn bộ thuật toán Hill Climbing.

6.2.2. Phương pháp đếm theo hàng và đường chéo với O(n)

Hàm conflicts(board) thay việc so sánh từng cặp bằng cách đếm số quân hậu trên mỗi hàng và mỗi đường chéo. Đối với quân hậu ở cột i và hàng q[i], chương trình cập nhật ba bộ đếm: horizontal_cnt[q[i]] cho hàng, diagonal1_cnt[i + q[i]] cho đường chéo thứ nhất và diagonal2_cnt[i − q[i] + n] cho đường chéo thứ hai. Việc cộng thêm n trong chỉ số đường chéo thứ hai giúp tránh chỉ số âm.

Mỗi quân hậu chỉ cần cập nhật ba bộ đếm một lần. Sau đó, chương trình duyệt các bộ đếm để tính tổng số cặp xung đột. Số hàng là n và số đường chéo ở mỗi hướng tỷ lệ với n, nên cả giai đoạn cập nhật và giai đoạn tổng hợp đều tốn O(n) thời gian. Bộ nhớ phụ để lưu các bộ đếm cũng tăng theo O(n).

Cách triển khai này không thay đổi ý nghĩa của hàm mục tiêu mà chỉ thay đổi cách tính toán. Nhờ đó, ba hàm có thể được so sánh trực tiếp trên cùng dữ liệu đầu vào.

6.2.3. Áp dụng công thức tổ hợp C(k, 2)

Nếu có k quân hậu cùng nằm trên một hàng hoặc một đường chéo, bất kỳ hai quân hậu nào trong nhóm cũng tạo thành một cặp xung đột. Số cặp khác nhau được tính bằng công thức tổ hợp:

$C(k, 2) = k(k − 1) / 2.$

Hàm comb2(k) trong notebook hiện thực biểu thức trên bằng phép nhân và phép chia nguyên. Giá trị f(board) được xác định bằng tổng C(k, 2) của tất cả nhóm hàng và cả hai hướng đường chéo. Với cách biểu diễn mỗi cột một quân hậu, không cần

bổ sung bộ đếm theo cột. Hơn nữa, một cặp hậu ở hai cột khác nhau không thể đồng thời vừa cùng hàng vừa cùng đường chéo, nên quy tắc tổng hợp này không đếm trùng một cặp xung đột.

Ví dụ, cấu hình [1, 3, 0, 2] không có hai quân hậu nào cùng hàng hoặc đường chéo nên f(board) = 0. Ngược lại, cấu hình [0, 0, 0, 0] có bốn quân hậu cùng hàng, tạo C(4, 2) = 6 cặp xung đột. Các trường hợp này được sử dụng trong phần kiểm chứng kết quả.

6.2.4. Tối ưu thực thi bằng Numba JIT

Hàm conflicts_jit(board) giữ nguyên nguyên lý đếm xung đột theo hàng và đường chéo, nhưng sử dụng bộ đếm một chiều gồm 5n phần tử và được đánh dấu bằng bộ trang trí @njit() của thư viện Numba. Khi hàm được gọi với một kiểu dữ liệu đầu vào mới, Numba biên dịch mã thích hợp cho kiểu dữ liệu đó; những lần gọi tiếp theo có thể sử dụng mã đã biên dịch.

Trong phiên bản này, chương trình duyệt từng quân hậu để cập nhật bộ đếm, sau đó duyệt các phần tử bộ đếm và cộng số cặp theo comb2_jit(). Do số lần duyệt vẫn tỷ lệ với n, độ phức tạp thời gian của conflicts_jit() vẫn là O(n), và bộ nhớ phụ vẫn là O(n). Lợi ích của Numba chủ yếu đến từ việc giảm chi phí thực thi các vòng lặp và thao tác của Python, không phải từ việc giảm bậc độ phức tạp.

Cần lưu ý rằng biên dịch JIT có chi phí khởi động ở lần gọi đầu. Vì vậy, nếu chỉ tính toán một vài bàn cờ nhỏ, chi phí biên dịch có thể lớn hơn thời gian tiết kiệm được. Ngược lại, khi hàm mục tiêu được gọi lặp lại nhiều lần, chi phí biên dịch có thể được phân bổ trên toàn bộ quá trình tính toán.

6.3. Thiết lập thực nghiệm

6.3.1. Các kích thước bàn cờ được khảo sát

Phần benchmark bổ sung sử dụng tám kích thước bàn cờ: n = 4, 8, 16, 32, 64, 128, 256 và 512. Ở mỗi kích thước, chương trình tạo 20 bàn cờ ngẫu nhiên theo quy ước một quân hậu trên mỗi cột. Bộ sinh số ngẫu nhiên sử dụng seed 20261016 để các dữ liệu đầu vào có thể tái lập.

Ba hàm conflicts_naive(), conflicts() và conflicts_jit() được áp dụng cho cùng các bàn cờ của từng kích thước. Trước khi đo thời gian, notebook đối chiếu kết quả của ba hàm để bảo đảm chúng tính cùng một đại lượng. Cách bố trí này giúp sự khác nhau về thời gian chủ yếu phản ánh phương pháp cài đặt, thay vì thay đổi dữ liệu thử nghiệm.

6.3.2. Phương pháp đo thời gian thực thi

Notebook sử dụng timeit.repeat() để đo thời gian tính số cặp xung đột cho một nhóm gồm 20 bàn cờ. Mỗi cấu hình phương pháp và kích thước được đo 5 lần. Thời gian của một lần đo được chia cho 20 để quy đổi thành micro giây trên mỗi bàn cờ (µs/bàn cờ), sau đó lấy trung vị của năm giá trị làm kết quả đại diện.

Việc sử dụng trung vị giúp hạn chế ảnh hưởng của một số lần đo đột biến do hoạt động của hệ điều hành hoặc tải CPU, mặc dù không loại bỏ hoàn toàn sai số đo. Trong benchmark này, thời gian còn bao gồm chi phí gọi hàm và vòng lặp qua danh sách bàn cờ từ Python; do đó, kết quả nên được hiểu là hiệu suất của toàn bộ cách gọi hàm đã thiết lập, không phải riêng thời gian của từng lệnh máy được biên dịch.

6.3.3. Loại trừ thời gian biên dịch JIT lần đầu

Đối với Numba, lần gọi đầu của một chữ ký đầu vào có thể phải thực hiện thêm bước biên dịch, làm thời gian tăng mạnh. Notebook đã gọi conflicts_jit() trước khi benchmark với cùng dạng mảng số nguyên int64 được dùng cho các bàn cờ thực nghiệm. Nhờ đó, các giá trị ở bảng benchmark chính phản ánh thời gian thực thi sau biên dịch, không gộp chi phí JIT lần đầu.

Để minh họa riêng chi phí biên dịch, chương trình thử một chữ ký mới trên mảng int32. Lần gọi đầu mất khoảng 171.360,136 µs, trong khi trung vị của 200 lần gọi tiếp theo với cùng chữ ký chỉ khoảng 0,380 µs. Đây là phép đo minh họa trên mảng bốn phần tử và chữ ký int32, không phải thông số benchmark của n = 512. Sự chênh lệch cho thấy vì sao cần tách giai đoạn biên dịch khỏi phép so sánh tốc độ chạy giữa các phương pháp.

6.4. Kết quả thực nghiệm

6.4.1. Kiểm chứng tính đúng đắn của ba hàm đếm xung đột

Trước khi so sánh tốc độ, chương trình kiểm tra các giá trị trả về của ba hàm trên 53 bàn cờ, bao gồm các trường hợp nhỏ có kết quả dễ kiểm chứng và những bàn cờ ngẫu nhiên với kích thước đến n = 512. Trong cả 53 trường hợp, conflicts_naive(), conflicts() và conflicts_jit() đều trả về cùng số cặp xung đột. Đối với các bàn cờ có n không quá 32, notebook còn đối chiếu với hàm tham chiếu độc lập thực hiện kiểm tra từng cặp hậu.

Bảng 6.1. Một số trường hợp kiểm chứng tính đúng đắn của hàm mục tiêu

Cấu hình	f theo Naive	f theo bộ đếm	f theo Numba JIT
[1, 3, 0, 2]	0	0	0
[3, 0, 2, 1]	1	1	1
[0, 0, 0, 0]	6	6	6

Kết quả khẳng định ba phương pháp chỉ khác cách tính toán và tốc độ thực hiện, không làm thay đổi ý nghĩa của hàm mục tiêu. Đây là điều kiện cần để so sánh hiệu suất một cách hợp lệ.

6.4.2. So sánh thời gian thực thi theo kích thước bàn cờ

Bảng 6.2 trình bày trung vị thời gian trên mỗi bàn cờ từ năm lượt đo, với 20 bàn cờ ở từng kích thước. Các giá trị sử dụng đơn vị micro giây (µs), không phải mili giây (ms).

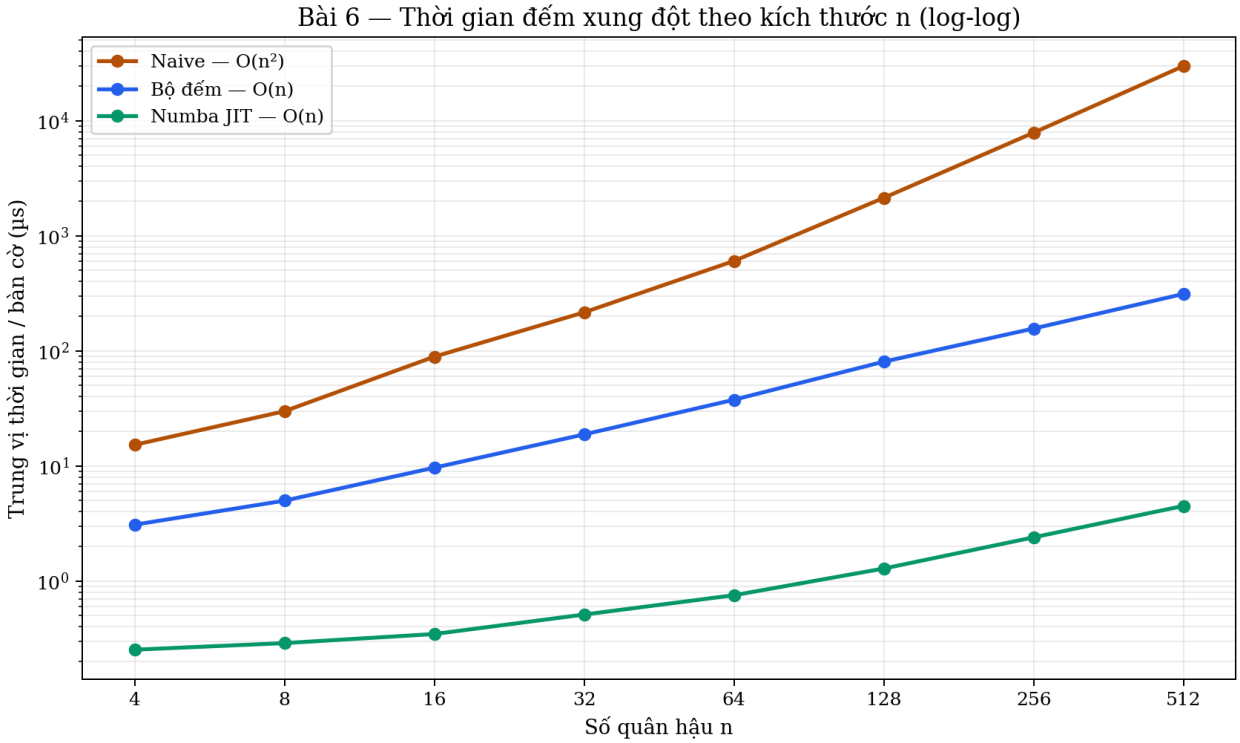
Bảng 6.2. Thời gian thực thi của ba phương pháp theo kích thước n-Queens

n	Naive $O(n^2)$ (µs)	Bộ đếm O(n) (µs)	Numba JIT O(n) (µs)
4	15,232	3,084	0,253

8	29,840	4,985	0,288
16	88,845	9,646	0,346
32	215,779	18,764	0,511
64	603,057	37,558	0,752
128	2.131,811	80,434	1,282
256	7.855,401	156,045	2,394
512	29.777,267	311,915	4,490

Ở kích thước n = 4, Naive mất khoảng 15,232 µs, phương pháp dùng bộ đếm mất 3,084 µs và Numba JIT mất 0,253 µs trên mỗi bàn cờ. Khi tăng đến n = 512, các giá trị tương ứng là khoảng 29.777,267 µs, 311,915 µs và 4,490 µs. Như vậy, chênh lệch giữa phương pháp bậc hai và hai phương pháp tuyến tính gia tăng rõ rệt theo kích thước bài toán.

6.4.3. Biểu đồ log-log thời gian thực thi



Hình 6.1. Thời gian đếm xung đột theo số quân hậu n trên thang log-log, so sánh Naive, bộ đếm tuyến tính và Numba JIT.

Hình 6.1 biểu diễn thời gian trên mỗi bàn cờ theo kích thước n, với cả trục hoành và trục tung sử dụng thang logarit. Đường của Naive tăng nhanh nhất khi n tăng; đường của phương pháp bộ đếm và đường Numba JIT tăng chậm hơn. Nhìn chung, xu hướng phù hợp với phân tích độ phức tạp $O(n^2)$ đối với Naive và $O(n)$ đối với hai phiên bản còn lại.

Tuy nhiên, hình dạng đường thời gian thực nghiệm còn chịu ảnh hưởng của chi phí khởi tạo, cách biểu diễn mảng, chi phí gọi hàm và tải của môi trường chạy. Vì vậy, đồ thị log-log là bằng chứng quan sát được về xu hướng tăng thời gian trong phạm vi dữ liệu khảo sát; việc xác định độ phức tạp thuật toán vẫn dựa vào phân tích cấu trúc vòng lặp và số phép toán của mã nguồn.

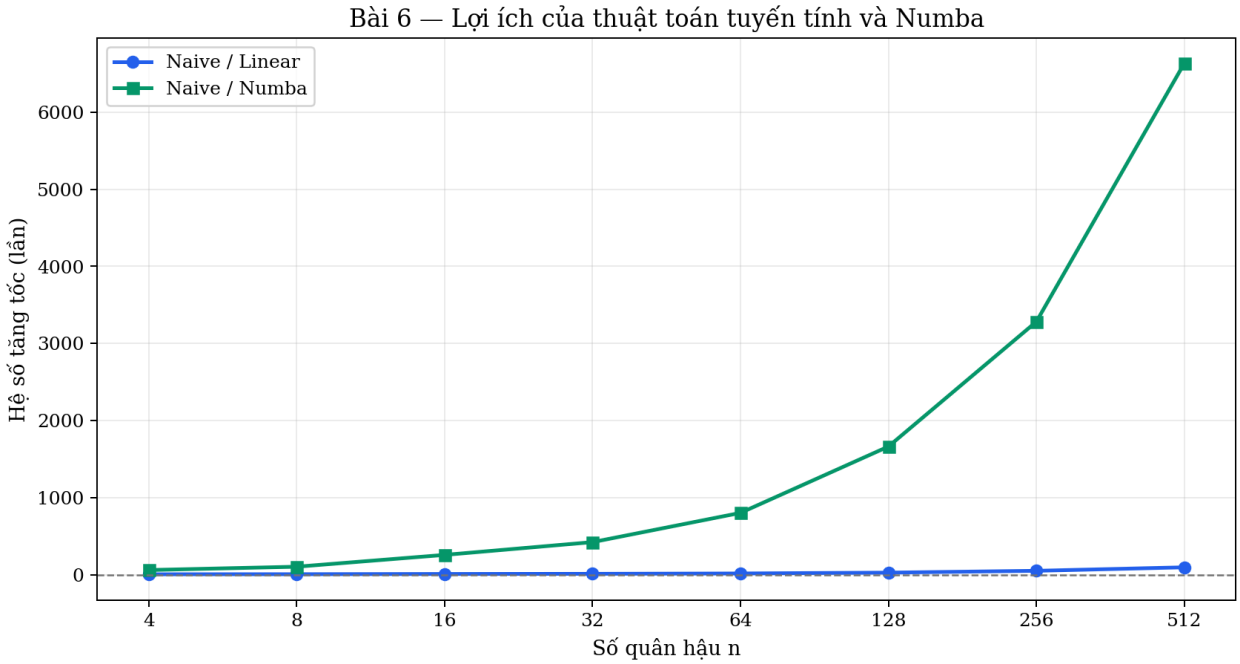
6.4.4. Phân tích hệ số tăng tốc khi sử dụng Numba

Để so sánh mức cải thiện, hệ số tăng tốc được tính bằng tỷ số giữa thời gian của phương pháp mốc và thời gian của phương pháp được tối ưu. Chẳng hạn, hệ số tăng tốc của Numba so với Naive được tính theo công thức:

Speedup(JIT so với Naive) = t_Naive / t_JIT.

Bảng 6.3. Hệ số tăng tốc tại một số kích thước đại diện

n	**Naive / Bộ đếm**	**Naive / Numba**	**Bộ đếm / Numba**
8	5,99 lần	103,47 lần	17,28 lần
128	26,50 lần	1.662,36 lần	62,72 lần
512	95,47 lần	6.632,28 lần	69,47 lần



Hình 6.2. Hệ số tăng tốc của phương pháp đếm tuyến tính và Numba JIT so với Naive theo kích thước bàn cờ.

Tại n = 512, chuyển từ Naive sang cách đếm tuyến tính cho hệ số tăng tốc khoảng 95,47 lần, còn phiên bản Numba JIT nhanh hơn Naive khoảng 6.632,28 lần. Nếu chỉ xét hai cách cài đặt có cùng độ phức tạp O(n), Numba JIT nhanh hơn phương pháp tuyến tính thuần Python khoảng 69,47 lần tại kích thước này.

Những hệ số tăng tốc trên là kết quả của bộ benchmark cụ thể và không thể xem là hằng số cố định cho mọi máy tính. Ngoài ra, đối với các hàm phải gọi ít lần, việc tính gộp chi phí biên dịch JIT có thể làm giảm hoặc đảo ngược lợi ích hiệu suất tổng thể. Việc sử dụng Numba vì thế phù hợp nhất khi số lần gọi hàm đủ lớn để bù chi phí biên dịch.

6.5. Nhận xét

Qua thực nghiệm, cả ba hàm đếm xung đột đều cho kết quả giống nhau trên 53 bàn cờ kiểm thử. Điều này cho thấy việc chuyển từ phương pháp Naive sang bộ đếm theo hàng và đường chéo, sau đó áp dụng Numba JIT, vẫn giữ nguyên tính đúng đắn của hàm mục tiêu. Bài thực hành vì vậy có thể tập trung so sánh hiệu suất mà không phải đánh đổi độ chính xác của số cặp xung đột.

Sự cải thiện đầu tiên đến từ thiết kế thuật toán. Thay vì so sánh các cặp quân hậu theo độ phức tạp $O(n^2)$, phương pháp bộ đếm sử dụng công thức tổ hợp $C(k, 2)$ và duyệt số lượng nhóm tỷ lệ với n, đưa độ phức tạp thời gian xuống $O(n)$. Đây là thay đổi quan trọng về quy mô số phép toán khi số quân hậu tăng. Phương pháp cần thêm bộ nhớ phụ $O(n)$, nhưng đổi lại giảm đáng kể thời gian tính toán trên các bàn cờ lớn.

Sự cải thiện thứ hai đến từ tối ưu cài đặt bằng Numba. Phiên bản conflicts_jit() không có độ phức tạp thấp hơn conflicts(); cả hai đều là $O(n)$. Tuy nhiên, mã đã biên dịch giảm chi phí thực thi so với các thao tác vòng lặp trong Python thuần. Kết quả benchmark ghi nhận sự chênh lệch rõ rệt, đặc biệt tại $n = 512$, khi Numba nhanh hơn phiên bản tuyến tính thuần Python khoảng 69,47 lần.

Bên cạnh lợi ích hiệu suất, kết quả cũng chỉ ra điều kiện cần để so sánh hợp lý: phải làm nóng hàm JIT trước khi đo, sử dụng cùng kiểu dữ liệu đầu vào, kiểm tra tính đúng đắn trên cùng bàn cờ và lặp phép đo nhiều lần. Thời gian biên dịch lần đầu, bộ nhớ đệm, chi phí gọi hàm và tải CPU đều có thể ảnh hưởng đến kết quả, nên không thể khái quát các hệ số tăng tốc thành quy luật cố định cho mọi môi trường.

Nhìn chung, Bài 6 cho thấy tối ưu hóa một hàm mục tiêu được sử dụng lặp lại nhiều lần có thể mang lại lợi ích đáng kể cho các thuật toán tìm kiếm cục bộ. Việc

giảm bậc độ phức tạp giúp thuật toán có khả năng mở rộng tốt hơn; biên dịch JIT tiếp tục giảm thời gian thực thi trên cùng phương pháp tính. Hai hướng tối ưu này bổ trợ nhau nhưng cần được phân biệt rõ khi đánh giá kết quả thực nghiệm.

CHƯƠNG 7: Tìm kiếm cục bộ cho Traveling salesman problem (TSP)

7.1. Giới thiệu bài toán Traveling Salesman Problem

7.1.1. Phát biểu bài toán và mục tiêu tối ưu

Bài toán người du lịch (Traveling Salesman Problem – TSP) yêu cầu tìm một chu trình đi qua tất cả n thành phố, mỗi thành phố đúng một lần, rồi quay trở về điểm xuất phát sao cho tổng chiều dài hành trình nhỏ nhất. Khác với bài toán n-Queens ở những chương trước, hàm mục tiêu của TSP là một đại lượng liên tục không âm được xác định từ khoảng cách giữa các thành phố. Đây là bài toán tối ưu tổ hợp, trong đó số thứ tự ghé thăm có thể tăng rất nhanh khi số thành phố tăng.

Trong Bài 7 của Lab 03 – Local Search, notebook traveling_salesman_problem.ipynb cung cấp các hàm random_tsp(n), random_tour(n), tour_length(tsp, tour) và show_tsp(tsp, tour). Phần thực hành triển khai năm chiến lược tìm kiếm cục bộ gồm Steepest-ascent, Stochastic (all), First-choice, Random Restarts và Simulated Annealing. Các thuật toán được đánh giá theo độ dài tour, tỷ lệ cải thiện, số lần tính hàm mục tiêu và thời gian thực thi. Mục tiêu của thực nghiệm là so sánh kết quả trong cùng điều kiện dữ liệu và toán tử lân cận, không chứng minh các nghiệm thu được là tối ưu toàn cục.

7.1.2. Biểu diễn trạng thái bằng hoán vị các thành phố

Mỗi thành phố được đánh số từ 0 đến n − 1, và một trạng thái tìm kiếm được mô tả bằng hoán vị $\pi = [\pi_0, \pi_1, \ldots, \pi_{n-1}]$. Thứ tự phần tử trong vector cho biết trình tự ghé

thăm. Ví dụ, với bốn thành phố, tour [0, 2, 1, 3] thể hiện thứ tự 0 → 2 → 1 → 3 → 0; cạnh cuối quay về thành phố 0 là thành phần bắt buộc của chu trình.

Mỗi trạng thái hợp lệ phải chứa đủ và chỉ một lần các chỉ số từ 0 đến n − 1. Về mặt biểu diễn, có n! hoán vị. Do bài toán đang xét dùng khoảng cách đối xứng, một chu trình và chu trình đi ngược chiều tương ứng có cùng độ dài; việc chọn thành phố khởi hành cũng không làm thay đổi chu trình về mặt hình học. Tuy vậy, notebook vẫn sử dụng đầy đủ vector hoán vị để thao tác và kiểm thử. Việc dùng biểu diễn hoán vị giúp ngăn tình huống bỏ sót hay thăm lặp thành phố trong quá trình tối ưu.

7.1.3. Hàm mục tiêu tính tổng độ dài chu trình khép kín

Dữ liệu TSP được lưu dưới dạng từ điển gồm pos (tọa độ thành phố) và dist (ma trận khoảng cách). Khoảng cách giữa hai thành phố i và j được tính bằng khoảng cách Euclidean giữa tọa độ (x_i, y_i) và $(x□, y□)$:

$d(i, j) = \sqrt{[(x_i - x□)^2 + (y_i - y□)^2]}$.

Với hoán vị π, hàm mục tiêu trong notebook được xác định bởi:

$f(\pi) = \Sigma(i = 0 \text{ đến } n - 2)\ d(\pi_i, \pi_{i+1}) + d(\pi□_{-1}, \pi_0)$.

Số hạng cuối là cạnh khép kín từ thành phố cuối về thành phố đầu. Hàm tour_length() thực hiện trực tiếp phép cộng này; các thuật toán cùng hướng tới việc tối thiểu hóa $f(\pi)$. Vì thành phố được lấy ngẫu nhiên trong hình vuông đơn vị, độ dài tour ở đây có đơn vị tọa độ mô phỏng, không phải kilômét hay thời gian di chuyển thực tế.

7.2. Mô hình hóa bài toán tìm kiếm cục bộ

7.2.1. Khởi tạo tập thành phố và đường đi ngẫu nhiên

Hàm random_tsp(n) sinh n điểm hai chiều trong miền [0, 1] × [0, 1] bằng NumPy, sau đó sử dụng pdist() và squareform() trong SciPy để tạo ma trận khoảng cách Euclidean đối xứng. Hàm random_tour(n) tạo danh sách các chỉ số thành phố rồi xáo trộn thứ tự bằng random.shuffle(). Mỗi tour khởi tạo có đúng n thành phố và được đưa vào các thuật toán như trạng thái ban đầu.

Trong thí nghiệm minh họa, chương trình đặt seed bằng 20261015 cho cả mô-đun random và NumPy, sinh một bài toán gồm 12 thành phố và một tour khởi tạo chung. Đối với khảo sát nhiều kích thước, ở mỗi cặp (n, lượt chạy), các phương pháp tiếp tục dùng chính xác cùng tập thành phố và cùng tour ban đầu. Nhờ vậy, sự khác biệt về kết quả xuất phát từ cơ chế tìm kiếm và lựa chọn ngẫu nhiên của từng thuật toán, thay vì mỗi phương pháp được giao một bộ dữ liệu khác nhau.

7.2.2. Toán tử lân cận hoán đổi hai thành phố (Swap)

Toàn bộ năm thuật toán sử dụng cùng một toán tử lân cận Swap. Cho tour π và hai vị trí i, j khác nhau, toán tử tạo tour mới bằng cách hoán đổi hai phần tử π_i và π□, giữ nguyên các vị trí khác. Ví dụ, từ [0, 1, 2, 3], thao tác Swap(1, 3) tạo ra [0, 3, 2, 1]. Toán tử không làm thay đổi số lượng thành phố và tự bảo toàn tính hợp lệ của hoán vị.

Với n vị trí, số cách chọn hai vị trí khác nhau không phân biệt thứ tự là:

$|N(\pi)| = C(n, 2) = n(n - 1)/2.$

Vì vậy, tour 12 thành phố có 66 lân cận Swap, còn tour 25 thành phố có 300 lân cận. Trong notebook, swap_neighbor() tạo một lân cận theo hai vị trí chỉ định; all_swap_neighbors() duyệt mọi cặp i nhỏ hơn j. Việc cố định toán tử giúp so sánh trực tiếp các chiến lược lựa chọn nước đi; tuy nhiên, kết quả chỉ phản ánh không gian lân cận Swap, không đại diện cho mọi toán tử TSP như 2-opt.

7.2.3. Điều kiện hợp lệ của một đường tour

Hàm check_tour(tsp, tour) kiểm tra số phần tử của tour bằng n và kết quả sắp xếp tour đúng bằng danh sách [0, 1, …, n − 1]. Sau mỗi lượt chạy, notebook tiếp tục tính lại tour_length(tsp, result['tour']) và đối chiếu với chi phí đã ghi nhận trong kết quả của thuật toán, với sai số số thực cho phép nhỏ hơn 10^{-9}. Những kiểm thử này giúp bảo đảm rằng một kết quả có độ dài ngắn không phải do bỏ sót thành phố hoặc do tính sai cạnh quay về điểm xuất phát.

Một tour hợp lệ chưa nhất thiết là tour ngắn nhất. Trạng thái tối ưu cục bộ theo Swap chỉ thỏa điều kiện không có một lần hoán đổi hai vị trí nào làm giảm độ dài tour; điều này không chứng minh đã đạt chu trình tối ưu toàn cục. Vì vậy, chỉ số dùng để đánh giá chất lượng là độ dài tour tốt nhất thực tế được tìm thấy trong ngân sách của từng thuật toán.

7.3. Triển khai các thuật toán tìm kiếm cục bộ

7.3.1. Steepest-ascent Hill Climbing

Hàm steepest_ascent_tsp() nhận tour khởi tạo và giới hạn tối đa 100 bước cải thiện. Ở mỗi vòng, chương trình sinh toàn bộ lân cận Swap, tính tour_length() cho từng ứng viên, giữ tour có chi phí thấp nhất rồi chỉ chấp nhận nó nếu giảm chi phí hiện

tại theo ngưỡng so sánh số thực trong mã. Khi không tồn tại Swap cải thiện, thuật toán dừng. Vì TSP đang tối thiểu hóa độ dài tour, tên Steepest-ascent theo tài liệu được hiểu là chọn bước cải thiện tốt nhất, tương ứng hướng giảm hàm mục tiêu.

Ưu điểm của phương pháp này là chọn được bước giảm mạnh nhất trong số các Swap tại thời điểm xét. Hạn chế là mỗi lần khảo sát cần đánh giá toàn bộ $n(n-1)/2$ lân cận, nên số lần gọi hàm mục tiêu tăng đáng kể khi số thành phố lớn. Cách lựa chọn tốt nhất theo từng bước cũng không loại bỏ được nguy cơ dừng tại cực tiểu cục bộ.

7.3.2. Stochastic Hill Climbing

Hàm stochastic_hill_climbing_tsp() cũng duyệt toàn bộ các lân cận Swap ở mỗi vòng nhưng thu thập tất cả ứng viên có độ dài nhỏ hơn tour hiện tại. Thay vì chọn ứng viên có chi phí thấp nhất, chương trình dùng random.Random(seed) để lấy ngẫu nhiên một tour trong tập những tour cải thiện. Nếu tập cải thiện rỗng, thuật toán kết thúc.

Cơ chế chọn ngẫu nhiên giúp các lần chạy có thể theo những quỹ đạo khác nhau, qua đó có cơ hội tiếp cận những cực tiểu cục bộ khác nhau. Tuy nhiên, Stochastic (all) vẫn phải tính chi phí của mọi lân cận trước khi chọn nên không tiết kiệm lượt đánh giá như phương pháp chỉ thử từng ứng viên. Độ dài tour giảm nghiêm ngặt sau mỗi lần chấp nhận, nhưng số bước và kết quả cuối có thể phụ thuộc seed.

7.3.3. First-choice Hill Climbing

Hàm first_choice_hill_climbing_tsp() chọn ngẫu nhiên hai vị trí khác nhau, tạo một Swap rồi kiểm tra chi phí. Ngay khi gặp ứng viên có tour ngắn hơn, chương trình cập nhật trạng thái và tiếp tục tìm kiếm từ tour mới, không cần duyệt hết toàn bộ lân cận. Những Swap không cải thiện sẽ làm tăng bộ đếm số lần thử thất bại liên tiếp.

Theo notebook, số lần thất bại liên tiếp tối đa mặc định là min[n(n − 1), 250]; đồng thời có giới hạn 3.000 lần đánh giá ứng viên và 100 bước cải thiện. Nếu đạt giới hạn mà chưa gặp bước tốt hơn, thuật toán dừng. Khác với hai phương pháp quét toàn bộ, việc hết số lần thử của First-choice không chứng minh tour hiện tại là cực tiểu cục bộ vì một số Swap có thể chưa được lấy mẫu.

7.3.4. Random Restarts

Hàm random_restart_tsp() chạy lại Steepest-ascent trên nhiều tour khởi tạo. Lần đầu sử dụng tour được cung cấp; các lượt tiếp theo sinh hoán vị ngẫu nhiên mới. Sau mỗi lượt, chương trình giữ lại tour có độ dài nhỏ nhất từng tìm thấy. Cách triển khai này sử dụng tối đa 8 lượt tìm kiếm, đã bao gồm lần đầu, tương ứng tối đa 7 lần khởi động lại.

Khả năng thử nhiều điểm xuất phát giúp giảm sự phụ thuộc vào một tour ngẫu nhiên ban đầu và cải thiện cơ hội tìm được nghiệm ngắn hơn. Đổi lại, tổng số lần gọi tour_length() và thời gian thực thi có thể lớn hơn đáng kể so với thuật toán chỉ chạy một lần. Trong notebook, Random Restarts thực hiện đủ số lượt đã cấu hình và tổng hợp kết quả, thay vì dừng ngay sau một tour được cho là tốt do không có chứng nhận tối ưu toàn cục.

7.3.5. Simulated Annealing

Hàm simulated_annealing_tsp() áp dụng Swap ngẫu nhiên cùng quy tắc chấp nhận phụ thuộc nhiệt độ. Gọi $\Delta = f(\text{mới}) - f(\text{hiện tại})$. Nếu $\Delta \leq 0$, bước đi được chấp nhận; nếu $\Delta > 0$, thuật toán có thể chấp nhận bước làm tour dài hơn với xác suất:

$P(\text{chấp nhận}) = \exp(-\Delta/T)$.

Nhiệt độ được giảm theo lịch hàm mũ $T(t) = 0{,}20 \times 0{,}995^{t}$. Thí nghiệm đặt tối đa 1.500 lần thử và ngừng sớm nếu nhiệt độ nhỏ hơn 10^{-8}. Chương trình lưu riêng tour tốt nhất từng tìm thấy (best_tour, best_cost) bên cạnh tour hiện tại; bởi vậy, các bước chấp nhận tour dài hơn không làm mất kết quả tốt đã đạt được.

Simulated Annealing cho phép đi qua những trạng thái không cải thiện, nhờ đó có khả năng thoát khỏi một số cực tiểu cục bộ. Tuy nhiên, chất lượng nghiệm phụ thuộc vào nhiệt độ, lịch làm nguội, số lượt thử và trạng thái khởi tạo. Không có bằng chứng trong thí nghiệm rằng lịch nhiệt đang sử dụng luôn tạo tour tối ưu toàn cục.

7.4. Kết quả thực nghiệm

7.4.1. Thiết lập dữ liệu và các kích thước TSP

Phần thực nghiệm gồm một ví dụ trực quan với n = 12, seed 20261015, và khảo sát mở rộng tại n = 6, 10, 15, 20, 25. Ở mỗi kích thước, notebook sinh năm bộ dữ liệu và thực hiện cả năm thuật toán trên cùng bài toán và tour đầu cho mỗi lượt, tạo tổng cộng $5 \times 5 \times 5 = 125$ lượt chạy. Seed cho từng bộ dữ liệu được xác định bằng $20261015 + 100n + k$, trong đó k chạy từ 0 đến 4 (cột trial trong CSV được hiển thị từ 1 đến 5).

Mỗi lượt ghi nhận độ dài tour ban đầu, độ dài tour tốt nhất, phần trăm cải thiện, số bước chấp nhận, số lần đánh giá hàm mục tiêu và thời gian thực thi bằng perf_counter() tính theo mili giây. Phần trăm cải thiện được tính theo công thức:

Improvement (%) = [1 − f(tour tốt nhất)/f(tour ban đầu)] × 100%.

Bảng thống kê lưu trung bình từ năm lượt chạy cho từng kích thước và thuật toán. Tuy sử dụng cùng dữ liệu và cùng toán tử Swap, các phương pháp không có ngân sách đánh giá hoàn toàn giống nhau: Random Restarts có tám lượt tìm kiếm, Simulated Annealing có 1.500 lần thử, trong khi các thuật toán còn lại có quy tắc dừng riêng. Vì vậy, số lượt đánh giá phải được xem cùng với chất lượng tour và thời gian.

7.4.2. So sánh độ dài tour và tỷ lệ cải thiện

Trong thí nghiệm minh họa trên 12 thành phố, độ dài tour ban đầu là 7,176626 đơn vị. Cả năm phương pháp đều tìm được tour có độ dài 2,965786 đơn vị, tương ứng mức cải thiện 58,674%. Kết quả cho thấy nhiều chiến lược có thể đạt cùng chất lượng trên một bài toán cụ thể dù quỹ đạo tìm kiếm và lượng tính toán khác nhau. Việc có cùng giá trị độ dài không đủ để khẳng định đó là nghiệm tối ưu toàn cục.

Bảng 7.1. Kết quả minh họa của năm thuật toán trên cùng bài toán 12 thành phố

Thuật toán	Tour tốt nhất	Cải thiện	Bước chấp nhận	Lần đánh giá	Thời gian (ms)
Steepest-ascent	2,966	58,67%	8	595	1,073

Random Restarts	2,966	58,67%	62	4.628	9,271
Stochastic (all)	2,966	58,67%	19	1.321	3,417
First-choice	2,966	58,67%	13	391	1,588
Simulated Annealing	2,966	58,67%	42	1.501	5,443

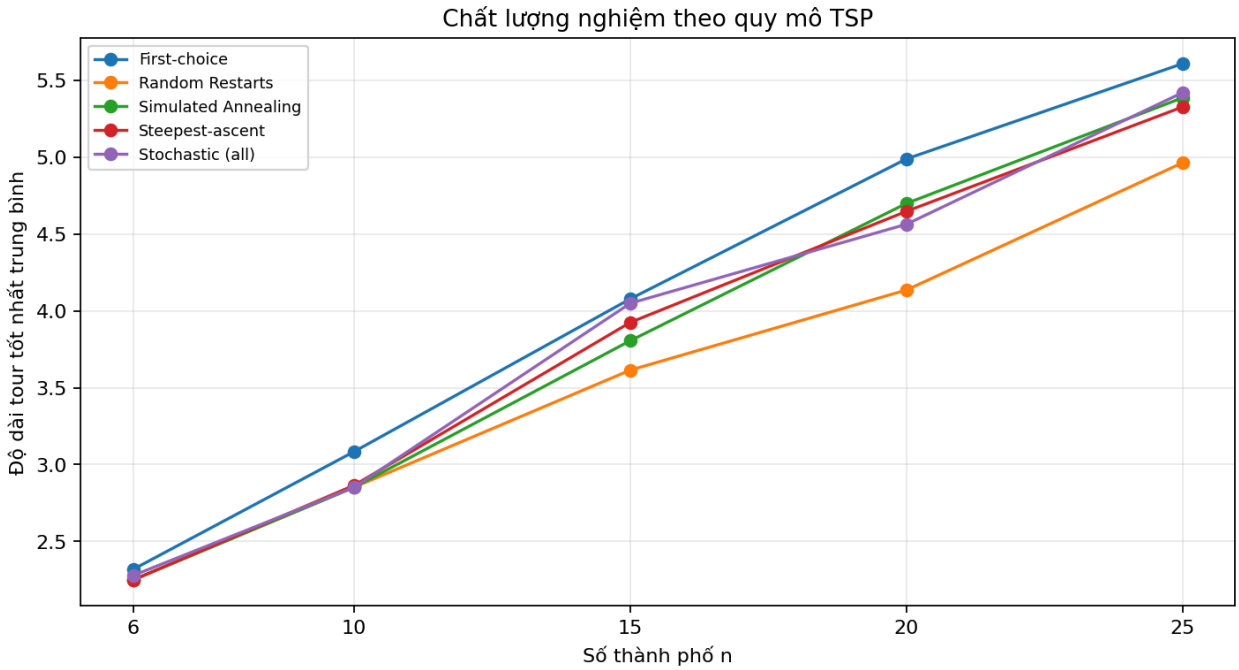
Khi xét trung bình trên năm lượt ở mỗi kích thước, khác biệt giữa các thuật toán thể hiện rõ hơn. Với n = 25, độ dài tour ban đầu trung bình là 12,115. Random Restarts đạt độ dài trung bình thấp nhất trong năm phương pháp (4,962), tương ứng cải thiện trung bình 58,33%. Trong khi đó, First-choice đạt 5,607 và cải thiện 52,88%, là chất lượng kém hơn trong nhóm tại kích thước này. Đây là các giá trị trung bình trên năm bộ dữ liệu, không phải một nghiệm tối ưu tham chiếu chung.

Bảng 7.2. Độ dài tour tốt nhất trung bình theo số thành phố (5 lượt/kích thước)

n	Steepest-ascent	Random Restarts	Stochastic (all)	First-choice	Simulated Annealing
6	2,250	2,250	2,279	2,319	2,250

10	2,866	2,853	2,853	3,085	2,853
15	3,925	3,614	4,049	4,078	3,806
20	4,647	4,135	4,563	4,988	4,698
25	5,325	4,962	5,419	5,607	5,385

Biểu đồ chất lượng theo quy mô cho thấy độ dài tour trung bình có xu hướng tăng khi số thành phố tăng, vì mỗi chu trình phải kết nối nhiều điểm hơn trong cùng vùng tọa độ. Random Restarts có kết quả tốt nhất hoặc đồng hạng tốt nhất trên mọi kích thước được khảo sát; tuy nhiên, phương pháp này được phép tìm kiếm từ nhiều tour khởi tạo và vì thế tiêu thụ ngân sách lớn hơn.



Hình 7.1. Độ dài tour tốt nhất trung bình của năm thuật toán theo số thành phố. Hình đã được chèn trong bản Word.

7.4.3. So sánh số lần đánh giá và thời gian thực thi

Bảng minh họa n = 12 cho thấy First-choice chỉ sử dụng 391 lần đánh giá hàm mục tiêu, ít hơn Steepest-ascent (595), Stochastic (all) (1.321), Simulated Annealing (1.501) và Random Restarts (4.628), dù cả năm tìm được tour có cùng độ dài. Điều này minh họa lợi ích của việc chấp nhận sớm một ứng viên cải thiện thay vì quét toàn bộ lân cận, nhưng không có nghĩa First-choice luôn nhanh nhất hoặc tìm được nghiệm tốt nhất.

Kết quả trung bình tại n = 25 cho thấy sự đánh đổi rõ hơn giữa chất lượng và ngân sách. Random Restarts tìm được tour ngắn nhất trung bình nhưng phải dùng 45.908 lần đánh giá và 169,386 ms. First-choice sử dụng 1.180,8 lần đánh giá và 5,832 ms, nhưng tour dài hơn. Simulated Annealing sử dụng cố định 1.501 lượt tính hàm mục

tiêu trong các lần chạy khảo sát, đạt thời gian trung bình 7,750 ms và tour trung bình 5,385.

Bảng 7.3. Chi phí thực nghiệm trung bình trên TSP 25 thành phố (5 lượt)

Thuật toán	Tour TB	Cải thiện TB	Lần đánh giá TB	Thời gian TB (ms)
Steepest-ascent	5,325	55,02%	6.181,0	22,096
Random Restarts	4,962	58,33%	45.908,0	169,386
Stochastic (all)	5,419	54,20%	13.321,0	48,699
First-choice	5,607	52,88%	1.180,8	5,832
Simulated Annealing	5,385	54,51%	1.501,0	7,750

Các giá trị thời gian được đo bằng perf_counter() trong môi trường chạy notebook và có thể thay đổi theo máy tính, tải hệ thống hoặc phiên bản thư viện. Hơn nữa, thuật toán đánh giá toàn bộ Swap có chi phí sinh ứng viên khác với lấy mẫu ngẫu nhiên một Swap; thời gian không được quyết định hoàn toàn chỉ bởi số lần gọi hàm mục tiêu.

7.4.4. Khảo sát hiệu suất khi tăng số thành phố

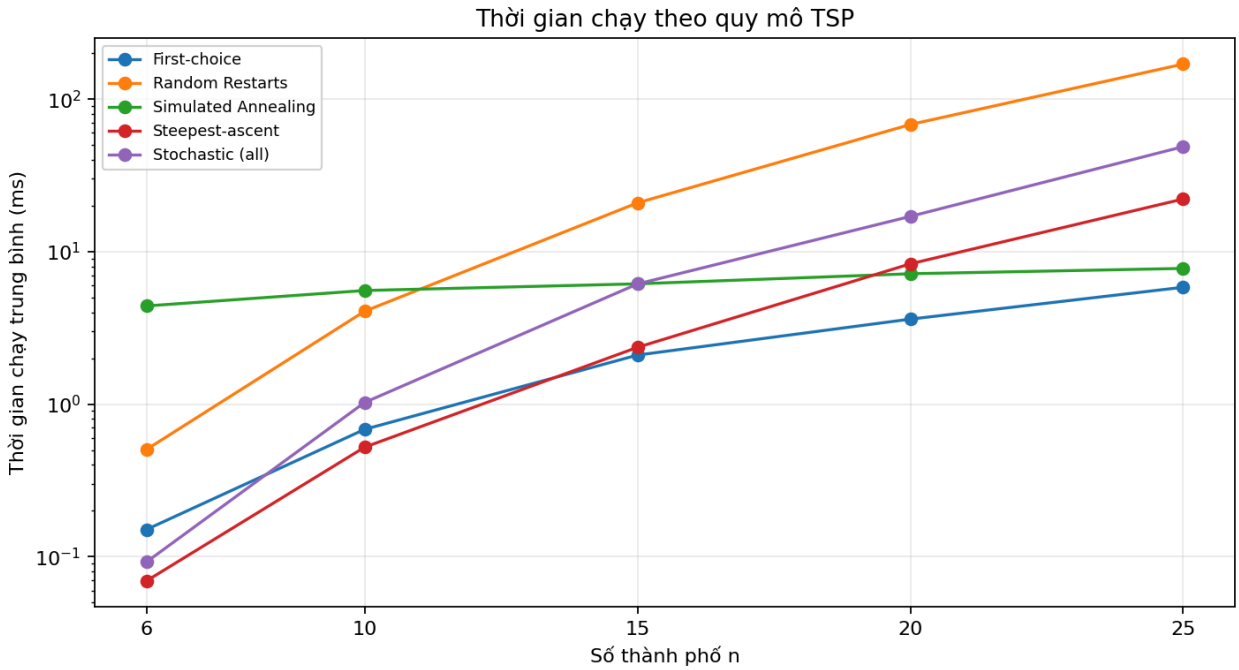
Khi số thành phố tăng từ 6 lên 25, số lân cận Swap của mỗi trạng thái tăng từ 15 lên 300. Điều này làm tăng chi phí một vòng khảo sát toàn bộ lân cận đối với Steepest-ascent và Stochastic (all). Random Restarts lặp lại việc khảo sát nhiều lần từ những điểm khởi tạo khác nhau, vì vậy có thời gian tăng nhanh nhất trong dữ liệu đo. Ở n = 6, thời gian trung bình của Random Restarts là 0,503 ms, nhưng đạt 169,386 ms tại n = 25.

Bảng 7.4. Thời gian thực thi trung bình theo số thành phố (ms; 5 lượt/kích thước)

n	Steepest-ascent	Random Restarts	Stochastic (all)	First-choice	Simulated Annealing
6	0,069	0,503	0,092	0,150	4,397
10	0,522	4,061	1,025	0,683	5,552
15	2,355	20,845	6,129	2,095	6,140
20	8,304	68,041	16,991	3,605	7,149

25	22,096	169,386	48,699	5,832	7,750

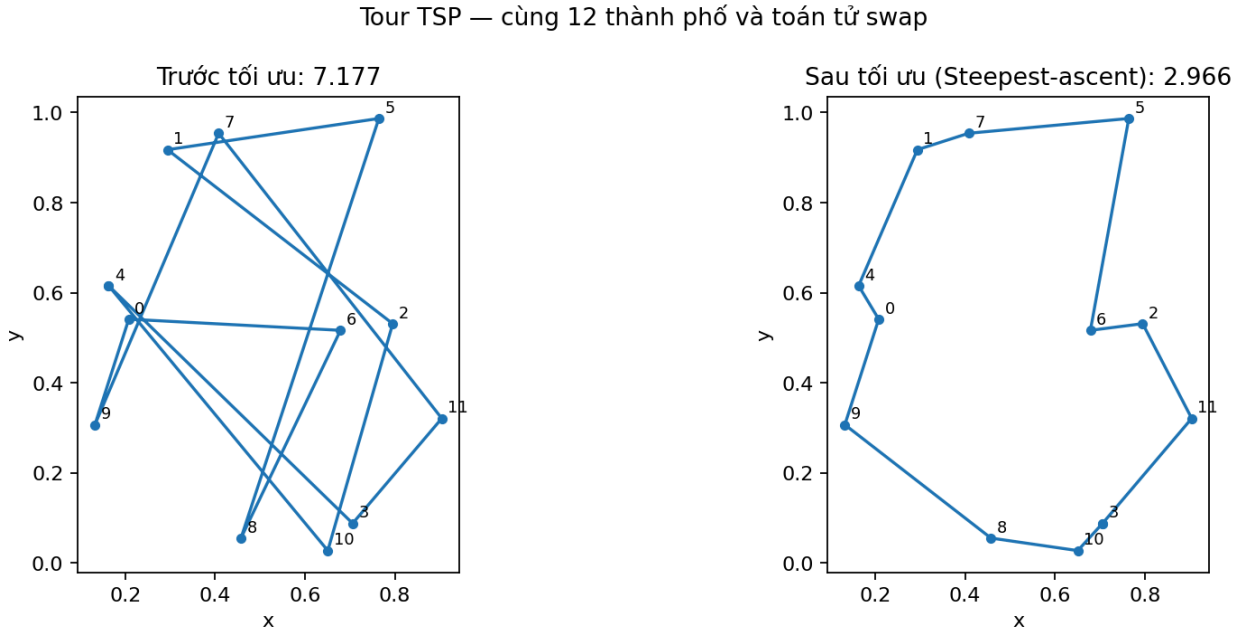
Biểu đồ thời gian sử dụng trục tung logarit để thể hiện rõ sự chênh lệch giữa những phương pháp có mức tiêu thụ tài nguyên rất khác nhau. Simulated Annealing tăng thời gian tương đối chậm trong khoảng kích thước khảo sát vì số lần thử tối đa giữ nguyên ở 1.500, trong khi chi phí tính tour_length() cho mỗi tour vẫn tăng theo n. Không nên suy ra rằng Simulated Annealing luôn mở rộng tốt hơn với mọi ngân sách, bởi số lần thử cố định cũng có thể làm hạn chế chất lượng nghiệm ở kích thước lớn.



Hình 7.2. Thời gian thực thi trung bình theo số thành phố; trục tung sử dụng thang logarit.

7.4.5. Trực quan hóa đường tour trước và sau tối ưu

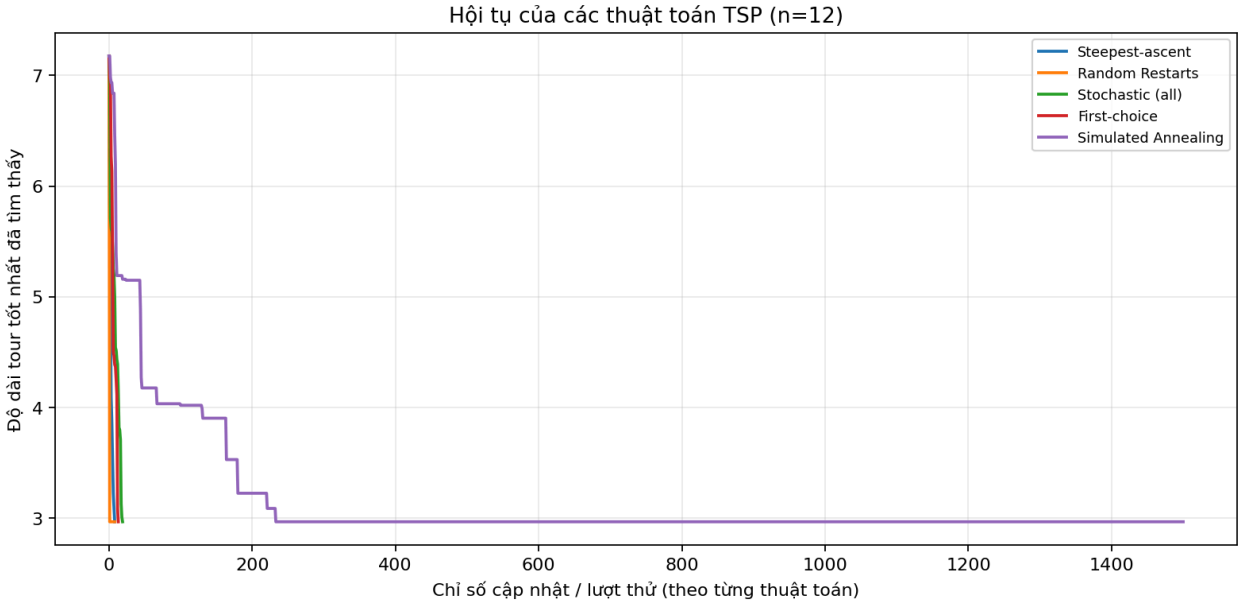
Hình minh họa trên 12 thành phố sử dụng cùng các vị trí tọa độ ở hai phía. Trước khi tối ưu, tour ngẫu nhiên có chiều dài 7,177 và chứa nhiều đoạn nối dài cắt qua khu vực phân bố điểm. Sau khi thực hiện Steepest-ascent, chu trình được tổ chức lại thành các đoạn nối ngắn hơn, giảm chiều dài xuống 2,966. Mức giảm này tương ứng khoảng 58,67% so với tour khởi tạo. Hình sử dụng Steepest-ascent để minh họa bởi các phương pháp trong lần chạy này thu được cùng giá trị chi phí tốt nhất.



Hình 7.3. Tour TSP trước và sau tối ưu bằng Steepest-ascent trên cùng 12 thành phố.

Bên cạnh hình học của chu trình, đồ thị hội tụ thể hiện chi phí tốt nhất được ghi nhận trong quá trình tìm kiếm. Các phương pháp Hill Climbing giảm nhanh ở những bước cải thiện đầu và dừng khi không còn lựa chọn được chấp nhận theo cơ chế tương ứng. Random Restarts cập nhật kết quả tốt nhất sau từng lượt khởi tạo. Simulated Annealing ghi lại nghiệm tốt nhất sau từng lần thử Swap, khiến đường

hội tụ trải dài đến 1.500 lượt và giữ nguyên ở những lần chưa tìm thấy kết quả tốt hơn.



Hình 7.4. Quá trình cập nhật tour tốt nhất của năm thuật toán trên bài toán 12 thành phố.

Điểm cần lưu ý khi đọc Hình 7.4 là trục hoành không mang cùng ý nghĩa cho mọi đường: Steepest-ascent, Stochastic (all) và First-choice ghi theo các bước cải thiện được chấp nhận; Random Restarts ghi theo lượt tìm kiếm lại; Simulated Annealing ghi theo từng lần thử lân cận. Do đó, hình phù hợp để quan sát hình dạng quỹ đạo và mức chi phí đã đạt, không đủ để so sánh trực tiếp tốc độ hội tụ theo cùng số lần gọi hàm mục tiêu.

7.5. Nhận xét

Qua thực nghiệm, năm thuật toán tìm kiếm cục bộ đều có thể rút ngắn tour TSP so với trạng thái ngẫu nhiên ban đầu. Việc giữ cùng biểu diễn hoán vị, hàm tour_length() và toán tử Swap giúp kết quả có cơ sở đối chiếu về mặt mô hình hóa. Trên bài toán minh họa gồm 12 thành phố, cả năm thuật toán đều tìm được độ dài

tour xấp xỉ 2,966 từ tour ban đầu 7,177, nhưng số lần đánh giá chênh lệch từ 391 đến 4.628. Điều này cho thấy chất lượng nghiệm tương đương trong một lượt chạy không đồng nghĩa với chi phí tính toán tương đương.

Steepest-ascent lựa chọn nước đi cải thiện mạnh nhất sau khi đánh giá toàn bộ lân cận, nên có thể giảm chi phí tour nhanh theo số bước được chấp nhận. Stochastic (all) vẫn quét toàn bộ Swap nhưng chọn ngẫu nhiên trong nhóm cải thiện, khiến quỹ đạo và số lượt đánh giá có thể tăng. First-choice có thể tiết kiệm phép đánh giá nhờ dừng ở nước đi cải thiện đầu tiên; tuy nhiên, việc hết ngân sách thử không xác nhận một cực tiểu cục bộ và chất lượng tour trung bình ở n = 25 thấp hơn những phương pháp còn lại trong bộ thử này.

Random Restarts nổi bật về chất lượng nghiệm trung bình khi số thành phố tăng, đạt tour trung bình 4,962 tại n = 25, nhưng cần khoảng 45.908 lần đánh giá và 169,386 ms mỗi lượt chạy. Điều đó phản ánh lợi ích của việc thử nhiều tour khởi tạo đồng thời cho thấy chi phí đáng kể của tám lần tìm kiếm Steepest-ascent. Simulated Annealing cân bằng theo một cách khác: trong quá trình làm nguội, thuật toán có thể chấp nhận tour xấu hơn để thoát khỏi một số vùng cực tiểu cục bộ, đồng thời giữ nghiệm tốt nhất đã gặp. Với ngân sách 1.500 lần thử trong notebook, phương pháp có thời gian trung bình 7,750 ms ở n = 25 và độ dài tour trung bình 5,385.

Toán tử Swap cũng quyết định hình dạng không gian lân cận: số ứng viên tăng theo $n(n-1)/2$ và các thuật toán chỉ xem xét những tour có thể nhận được bằng cách đổi hai vị trí. Do đó, hiệu quả quan sát được còn phụ thuộc vào toán tử, không chỉ vào chiến lược tìm kiếm. Những toán tử khác như đảo đoạn 2-opt có thể được khảo sát trong một nghiên cứu riêng, nhưng không thuộc bộ so sánh đã thực hiện ở chương này.

Nhìn chung, không có thuật toán nào đồng thời đứng đầu ở mọi tiêu chí chất lượng nghiệm, số lần đánh giá và thời gian trên toàn bộ thí nghiệm. Nếu ưu tiên tour ngắn trong ngân sách rộng, Random Restarts có kết quả thuận lợi nhất trong dữ liệu khảo sát; nếu ưu tiên thời gian thấp ở quy mô đang xét, First-choice hoặc Simulated Annealing có thể phù hợp hơn tùy mức chấp nhận chất lượng nghiệm. Các kết luận này chỉ áp dụng cho bộ TSP Euclidean ngẫu nhiên, năm lần chạy mỗi kích thước và những tham số đã sử dụng; chưa có đối chứng với nghiệm tối ưu toàn cục. Phần Genetic Algorithm được xác định là bài mở rộng và không được triển khai trong phạm vi Chương 7.

CHƯƠNG 8: Tổng hợp thí nghiệm và phân tích

8.1. Mục tiêu và phạm vi thực nghiệm tổng hợp

Bài thực hành tổng hợp của Lab 03 – Local Search nhằm hệ thống hóa kết quả nghiên cứu các thuật toán tìm kiếm cục bộ đã triển khai ở những chương trước, đồng thời đánh giá sự khác nhau về chất lượng nghiệm, chi phí tính toán và khả năng mở rộng. Phần thực nghiệm chính sử dụng bài toán n quân hậu (n-Queens) và hàm mục tiêu f(board) = conflicts(board); lời giải hợp lệ có f(board) = 0. Bài toán Traveling Salesman Problem (TSP) đã được khảo sát riêng ở Chương 7 và được liên hệ tại phần đánh giá tổng hợp, nhưng các bảng CSV của Task 6 trong Chương 8 chỉ đo n-Queens. Việc phân biệt phạm vi dữ liệu như vậy giúp tránh ghép các chỉ số không cùng bản chất thành một phép so sánh trực tiếp.

8.1.1. Các thuật toán được lựa chọn để so sánh

Notebook n_queens_Bai8_Hoan_thien.ipynb kế thừa các hàm đã triển khai trong các bài trước và tổng hợp năm phương pháp: Steepest-ascent Hill Climbing, Stochastic Hill Climbing (đánh giá toàn bộ tập cải thiện), First-choice Hill Climbing, Simulated Annealing và Random Restart kết hợp Steepest-ascent. Ba biến thể Hill Climbing chỉ chấp nhận bước làm giảm nghiêm ngặt số xung đột. Random Restart thử nhiều trạng thái ban đầu để khắc phục tính phụ thuộc vào điểm khởi tạo, còn Simulated Annealing có thể chấp nhận bước xấu hơn theo xác suất phụ thuộc nhiệt độ. Tất cả cùng sử dụng biểu diễn một quân hậu trên mỗi cột và toán tử cơ sở chuyển một quân hậu sang hàng khác trong cùng cột.

8.1.2. Điều kiện thực nghiệm và thiết lập random seed

Trong phép so sánh chính, mỗi thuật toán được chạy 100 lần cho n = 4 và 100 lần cho n = 8, tổng cộng 1.000 lượt chạy. Ở mỗi cặp (n, trial), cả năm thuật toán nhận cùng bàn cờ khởi tạo. Seed được xác định bằng 20261015 + n × 10000 + trial, với trial bắt đầu từ 0 khi tính seed và được đánh số từ 1 trong CSV. Các biến thể Hill Climbing được giới hạn tối đa 1.000 bước chấp nhận; Simulated Annealing sử dụng $T_0 = 3{,}0$, $\alpha = 0{,}995$, tối đa 2.500 lần thử lân cận; Random Restart được phép thực hiện tối đa 100 lượt Hill Climbing tính cả lượt đầu.

Đối với khả năng mở rộng, chương trình khảo sát các kích thước n = 4, 8, 12, 16, 20 với 20 lượt cho từng thuật toán Steepest-ascent, First-choice và Simulated Annealing, tức 300 lượt chạy. Phần nâng cao dùng 100 lượt trên mỗi cỡ n = 8 và n = 12 cho năm toán tử di chuyển, tức 1.000 lượt chạy nữa. Tất cả kết quả được xuất vào các CSV trong thư mục tương đối Lab03_Bai8_Results/; notebook kiểm chứng lại conflicts(board) trên trạng thái kết thúc để đối chiếu tính đúng đắn của kết quả được lưu.

8.1.3. Các chỉ số đánh giá

Chỉ số success_rate là tỷ lệ lượt chạy có số cặp xung đột cuối cùng bằng 0. mean_final_f là trung bình số cặp xung đột của nghiệm trả về; càng thấp thì chất lượng nghiệm trung bình càng tốt. mean_steps đếm số bước cải thiện hoặc số bước được chấp nhận theo cách ghi nhận của thuật toán, vì vậy không được đồng nhất chỉ số này giữa mọi phương pháp. mean_evaluations thể hiện số lần tính conflicts() bên trong quá trình tìm kiếm và phản ánh công sức đánh giá hàm mục tiêu. Thời gian thực thi được đo bằng perf_counter() và ghi bằng mili giây (ms); ngoài trung

bình, phần mở rộng còn sử dụng trung vị để giảm tác động của những lần đo biến động.

Đối với Random Restart, cần phân biệt runs (tổng số lượt thực hiện) và restarts_used (số lần khởi động lại sau lượt đầu). Giới hạn 100 lượt tìm kiếm tương ứng nhiều nhất 99 lần khởi động lại. Các thuật toán không có ngân sách đánh giá bằng nhau; do đó tỷ lệ thành công, số phép đánh giá và thời gian cần được xem xét đồng thời, không tách rời.

8.2. So sánh thuật toán trên bài toán n-Queens

8.2.1. Kết quả tổng hợp trên n = 4

Với bài toán bốn quân hậu, nghiệm hợp lệ tồn tại khi bốn quân hậu không tấn công nhau theo hàng hoặc đường chéo. Trong 100 lượt chạy, Steepest-ascent và Stochastic (all) cùng tìm được nghiệm ở 36 lượt; First-choice tìm được nghiệm ở 28 lượt. Hai cơ chế mở rộng là Simulated Annealing và Random Restart đều đạt 100 lượt thành công. Các kết quả này cho thấy Hill Climbing đơn lượt còn phụ thuộc đáng kể vào trạng thái khởi tạo ngay cả khi không gian trạng thái nhỏ.

Bảng 8.1. Thống kê 100 lượt chạy trên n-Queens với n = 4

Thuật toán	Thành công	f cuối TB	Bước TB	Đánh giá TB	Thời gian TB (ms)
Steepest-ascent	36%	0,73	1,51	26,80	0,096
Stochastic (all)	36%	0,83	1,89	31,36	0,119

First-choice	28%	0,90	1,95	37,20	0,223
Simulated Annealing	100%	0,00	63,54	85,56	0,574
Random Restart / steepest	100%	0,00	4,67	86,29	0,351

Trên n = 4, Steepest-ascent có số lần đánh giá trung bình thấp nhất và thời gian ngắn nhất trong các phương pháp được thử, nhưng chỉ tìm được lời giải ở 36% lượt chạy. Simulated Annealing và Random Restart tăng tỷ lệ thành công lên 100% nhưng phải thực hiện nhiều đánh giá hơn. Vì vậy, kết quả tốt về tỷ lệ thành công cần được đặt trong tương quan với ngân sách tính toán đã sử dụng.

8.2.2. Kết quả tổng hợp trên n = 8

Khi tăng lên tám quân hậu, số cách bố trí theo vector tăng nhanh và những điểm dừng không phải nghiệm xuất hiện nhiều hơn trong các đường tìm kiếm chỉ cho phép cải thiện nghiêm ngặt. Steepest-ascent đạt 12% lượt thành công, Stochastic (all) và First-choice cùng đạt 13%. Simulated Annealing đạt 97% trong khi Random Restart đạt 100% với ngân sách tối đa 100 lượt tìm kiếm. Tỷ lệ này được tính trực tiếp trên 100 lượt của từng phương pháp, không phải ước lượng từ một hình hội tụ minh họa.

Bảng 8.2. Thống kê 100 lượt chạy trên n-Queens với n = 8

Thuật toán	Thành công	f cuối TB	Bước TB	Đánh giá TB	Thời gian TB (ms)

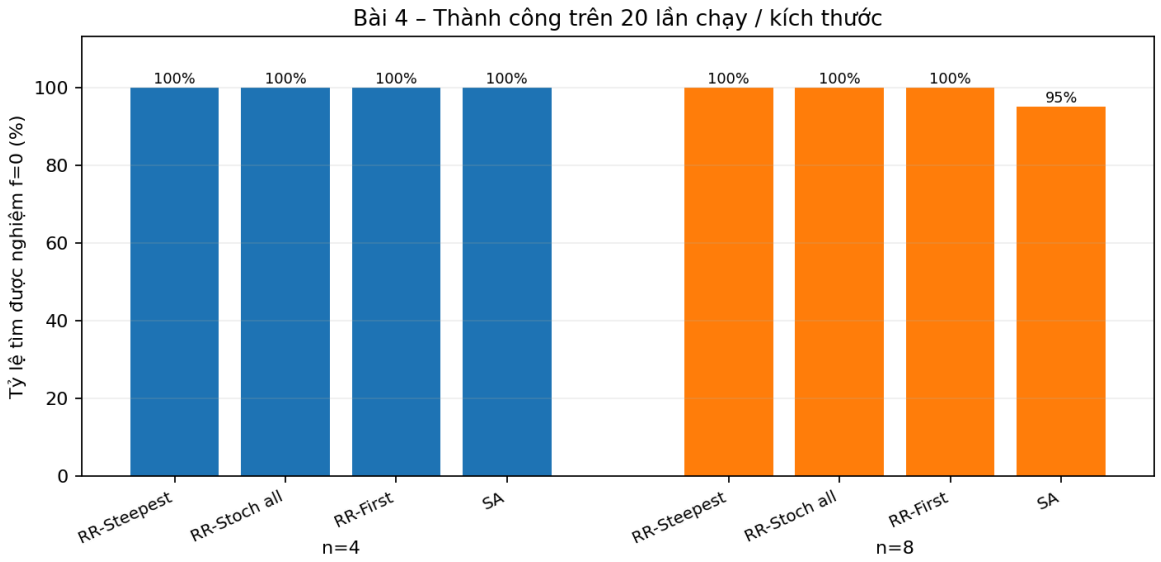
Steepest-ascent	12%	1,32	3,30	235,08	1,421
Stochastic (all)	13%	1,44	5,05	332,52	2,026
First-choice	13%	1,39	4,97	215,57	1,820
Simulated Annealing	97%	0,03	247,44	886,96	8,182
Random Restart / steepest	100%	0,00	25,40	1.823,54	10,878

Trong nhóm Hill Climbing đơn lượt, First-choice dùng trung bình 215,57 lượt đánh giá, thấp hơn Steepest-ascent (235,08) và Stochastic (all) (332,52), nhưng thời gian đo trung bình của First-choice vẫn cao hơn Steepest-ascent. Sự khác biệt này cho thấy số lần gọi hàm mục tiêu không bao quát mọi chi phí như sinh ứng viên, xử lý ngẫu nhiên và quản lý trạng thái. Simulated Annealing có tỷ lệ thành công cao nhưng dùng khoảng 886,96 lần đánh giá trung bình; Random Restart đạt tỷ lệ thành công tuyệt đối trong tập thử song tiêu tốn 1.823,54 lượt đánh giá trung bình.

8.2.3. So sánh tỷ lệ thành công và giá trị hàm mục tiêu

So với n = 4, tỷ lệ thành công của các phương pháp Hill Climbing đơn lượt giảm rõ trên n = 8: Steepest-ascent từ 36% xuống 12%, Stochastic (all) từ 36% xuống 13%, và First-choice từ 28% xuống 13%. Simulated Annealing giảm từ 100% xuống 97%, còn Random Restart duy trì 100% trong phạm vi 100 thử nghiệm ở mỗi kích thước. Vì mẫu hữu hạn, tỷ lệ 100% chỉ có nghĩa là không ghi nhận thất bại trong tập chạy này, không bảo đảm thuật toán thành công ở mọi bàn cờ và mọi ngân sách.

Bên cạnh thành công hoàn toàn, mean_final_f cung cấp thông tin về mức độ gần nghiệm của những lượt chưa thành công. Trên n = 8, ba Hill Climbing đơn lượt có f cuối trung bình lần lượt 1,32; 1,44 và 1,39; nghĩa là nhiều lần dừng ở trạng thái vẫn còn một số cặp hậu xung đột. Simulated Annealing đạt mean_final_f = 0,03, còn Random Restart bằng 0, cho thấy hai chiến lược mở rộng cải thiện đáng kể chất lượng nghiệm dưới cấu hình đang khảo sát.



Hình 8.1. Tỷ lệ thành công của Random Restart và Simulated Annealing trên n = 4 và n = 8 trong thí nghiệm Bài 4.

Hình 8.1 là minh họa từ Bài 4, nơi số lượt chạy cũng là 20 cho mỗi phương pháp và mỗi kích thước, không phải đồ thị dựng lại từ 100 lượt trong Bài 8. Vì số lần thử và cấu hình thực nghiệm có khác biệt, các giá trị trên hình không thay thế cho Bảng 8.1 và Bảng 8.2; chúng được sử dụng để liên hệ kết quả các chương trước.

8.2.4. So sánh thời gian và số lần đánh giá hàm mục tiêu

Độ đo số lần gọi conflicts() thuận tiện để theo dõi mức tiêu thụ của một thành phần tính toán chung, trong khi thời gian thực tế phản ánh cả chi phí điều phối thuật toán. Tại n = 8, First-choice có số phép đánh giá trung bình thấp nhất trong năm phương pháp, nhưng không có tỷ lệ tìm nghiệm cao hơn Stochastic (all). Random Restart và Simulated Annealing sử dụng nhiều phép đánh giá hơn để tăng cơ hội tìm nghiệm không xung đột. Đây là sự đánh đổi giữa chất lượng và chi phí, không phải mâu thuẫn giữa kết quả đo và nguyên lý thuật toán.

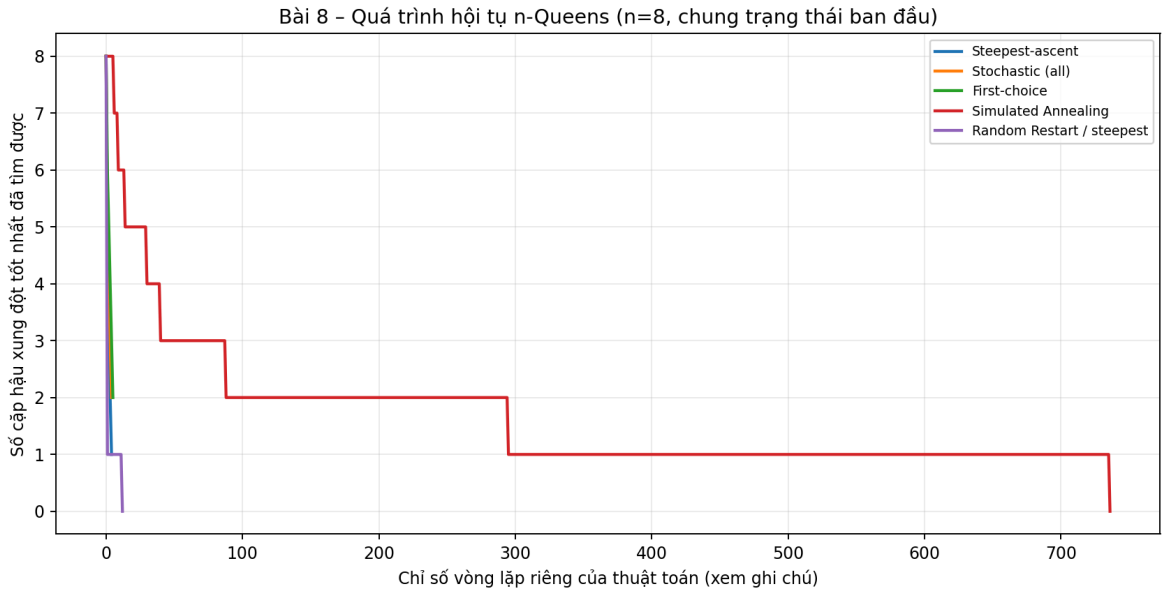
Cần tránh suy luận rằng Random Restart luôn tốn thời gian hơn hoặc Simulated Annealing luôn tiết kiệm hơn: giới hạn 100 lần khởi tạo lại, 2.500 lần thử lân cận và chính sách dừng của Hill Climbing là những ngân sách khác nhau. Thời gian đo phụ thuộc phần cứng, phiên bản Python, hoạt động của hệ thống và các thao tác trong mã; vì vậy kết luận trong chương chỉ có giá trị với môi trường benchmark đã ghi.

8.3. Phân tích quá trình hội tụ

8.3.1. Biểu đồ hội tụ của các thuật toán

Notebook chọn một trạng thái 8-Queens đại diện bằng seed 20261015 + 880 và dùng chính trạng thái đó làm đầu vào cho cả năm thuật toán. Giá trị hàm mục tiêu ban đầu là 8. Các phương pháp ghi lại giá trị tốt nhất đã tìm thấy theo từng mốc, nên mọi đường best-so-far không tăng theo thời gian. Đây là một cách biểu diễn

khác với current_f của Simulated Annealing, vốn có thể tăng khi thuật toán chấp nhận bước xấu hơn.



Hình 8.2. Quá trình hội tụ của năm thuật toán trên một trạng thái 8-Queens đại diện trong Bài 8.

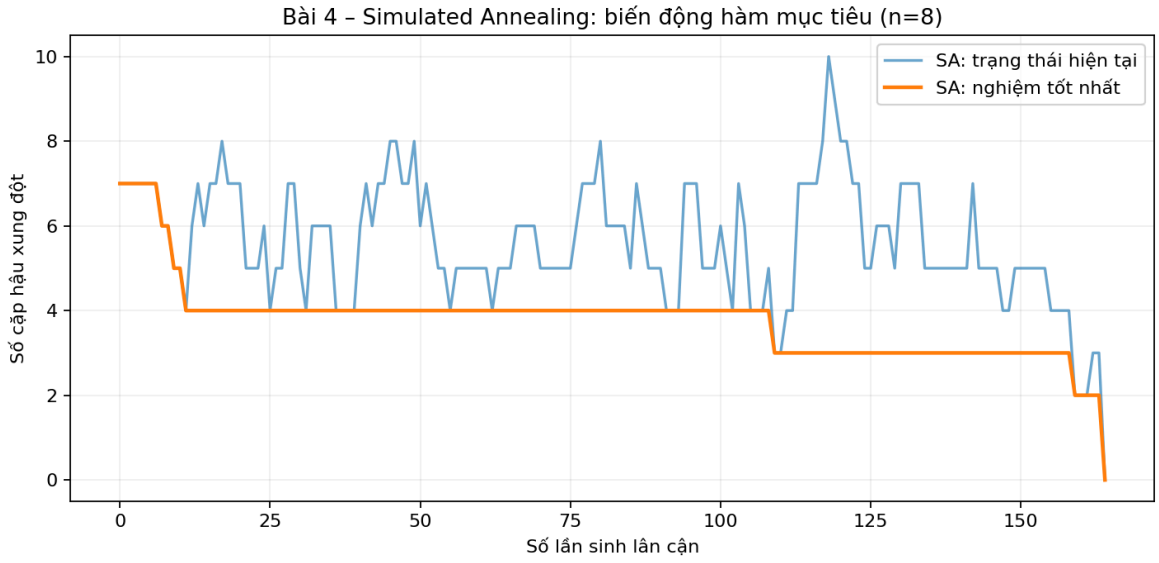
Trong ví dụ, Steepest-ascent có dãy f tốt nhất là 8 → 5 → 4 → 2 → 1, Stochastic (all) là 8 → 5 → 4 → 3 → 2, First-choice là 8 → 6 → 5 → 4 → 3 → 2. Hai phương pháp còn lại tìm được nghiệm bằng 0: Simulated Annealing sau 736 lượt thử và Random Restart sau 12 mốc lượt khởi tạo trong dữ liệu hội tụ. Kết quả này minh họa được quỹ đạo tìm kiếm của từng phương pháp trên đúng một trạng thái, không đại diện cho tần suất thành công chung.

Trục hoành của Hình 8.2 có đơn vị khác nhau: đối với ba Hill Climbing là số bước cải thiện đã chấp nhận; với Simulated Annealing là số lần thử trạng thái lân cận; với Random Restart là các mốc lượt khởi tạo. Vì vậy không được dựa vào độ dài đường vẽ theo trục hoành để khẳng định thuật toán nào tìm nhanh hơn. So sánh tốc độ phải sử dụng thời gian hoặc số lần đánh giá trên cùng thiết lập ngân sách.

8.3.2. Khả năng cải thiện trong giai đoạn đầu

Các đường Hill Climbing giảm tương đối mạnh ngay những bước đầu vì mỗi bước được chấp nhận đều làm giảm hàm mục tiêu. Steepest-ascent có thể giảm nhiều xung đột trong một bước nhờ lựa chọn trạng thái tốt nhất trong toàn bộ lân cận; Stochastic (all) có thể giảm từng bước nhỏ hơn khi chọn ngẫu nhiên trong nhóm lân cận cải thiện; First-choice cập nhật ngay khi tìm được một ứng viên đủ tốt, thay vì chờ đánh giá hết các khả năng.

Simulated Annealing không yêu cầu trạng thái hiện tại giảm đều, bởi thuật toán có thể đi qua những trạng thái kém hơn để tiếp tục khám phá. Vì vậy, đường của nghiệm tốt nhất chỉ phản ánh các lần cập nhật thành tích, còn biến động của trạng thái hiện tại có thể mạnh hơn. Điều này cũng được thể hiện qua hình từ Bài 4: current_f đôi lúc tăng lên, trong khi best_f không bao giờ tăng.



Hình 8.3. Biến động số xung đột của trạng thái hiện tại và nghiệm tốt nhất trong Simulated Annealing (hình thực nghiệm Bài 4).

8.3.3. Hiện tượng dừng sớm tại cực trị cục bộ

Một thuật toán Hill Climbing chỉ chấp nhận cải thiện nghiêm ngặt có thể dừng ở trạng thái $f > 0$. Với Steepest-ascent và Stochastic (all), nếu đã duyệt toàn bộ lân cận mà không có nước đi tốt hơn, trạng thái dừng là cực tiểu cục bộ theo toán tử đang dùng. Đối với First-choice, việc đạt TRY_LIMIT chỉ chứng tỏ không tìm thấy bước cải thiện trong ngân sách thử ngẫu nhiên đã định, không chứng minh rằng toàn bộ lân cận đều không cải thiện. Hai cơ chế dừng cần được phân biệt khi đọc dữ liệu.

Tại những vùng Plateau, nhiều trạng thái có cùng giá trị f, nhưng Hill Climbing chỉ nhận bước cải thiện nghiêm ngặt nên không đi ngang qua những trạng thái bằng nhau. Với Ridge, hướng cải thiện có thể đòi hỏi sự phối hợp nhiều thay đổi thay vì một phép chuyển đơn lẻ. Random Restart xử lý phụ thuộc điểm đầu bằng việc khởi tạo lại, còn Simulated Annealing cho phép một số bước bằng hoặc xấu hơn để khám phá thêm. Cả hai không loại bỏ hoàn toàn mọi hạn chế với bất kỳ ngân sách hữu hạn nào.

8.4. Khảo sát khả năng mở rộng

8.4.1. Thiết lập thực nghiệm với n = 4, 8, 12, 16, 20

Phần kiểm tra khả năng mở rộng giữ lại ba phương pháp Steepest-ascent, First-choice và Simulated Annealing. Mỗi thuật toán chạy 20 lần cho mỗi cỡ bàn cờ $n = 4, 8, 12, 16, 20$ với những bàn cờ khởi tạo được xác định bằng seed 20261015 +

900000 + n × 1000 + trial. Chương trình ghi số lần đánh giá, thời gian thực thi và tỷ lệ đạt f = 0, sau đó lấy trung vị thời gian của 20 lượt để vẽ đồ thị log-log. Sự khác nhau về ngân sách đánh giá giữa các phương pháp vẫn tồn tại, nên đồ thị dùng để quan sát xu hướng trong cấu hình thực nghiệm thay vì so sánh độ phức tạp lý thuyết một cách tuyệt đối.

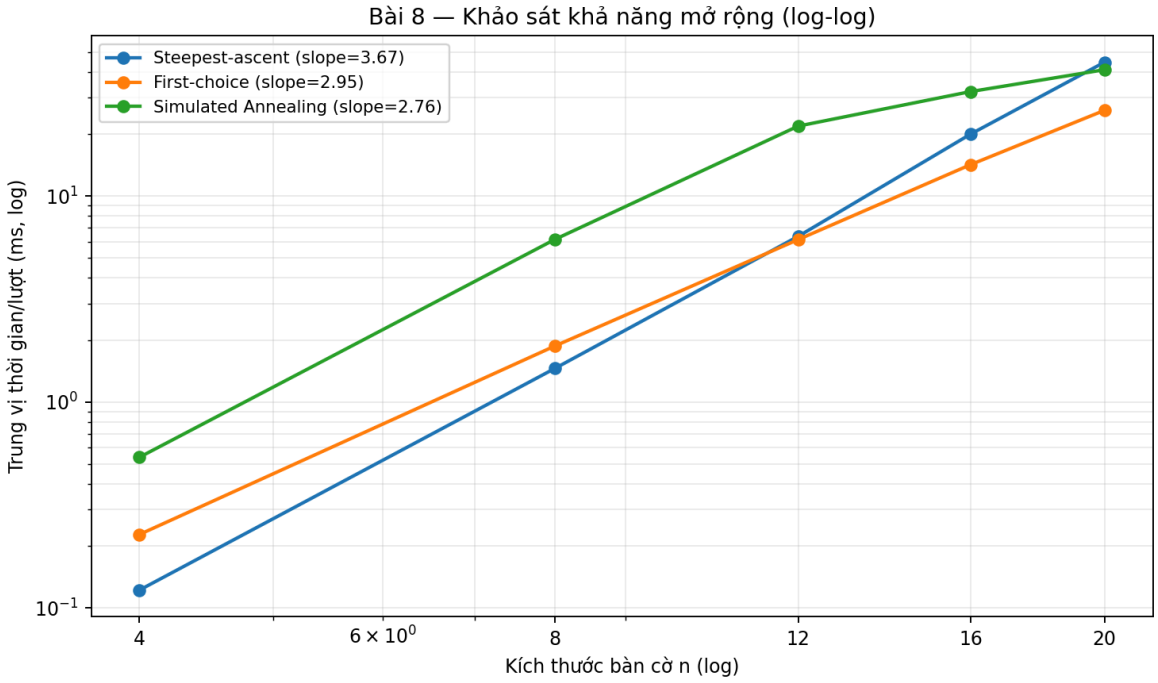
8.4.2. Kết quả thời gian thực thi theo kích thước bài toán

Bảng 8.3. Trung vị thời gian thực thi (ms) theo kích thước bàn cờ, 20 lượt/thuật toán/kích thước

n	Steepest-ascent	First-choice	Simulated Annealing
4	0,122	0,227	0,539
8	1,457	1,872	6,163
12	6,373	6,147	21,856
16	19,998	14,211	32,158
20	44,646	26,069	41,061

Ở n = 4, Steepest-ascent có thời gian trung vị thấp nhất trong ba thuật toán, nhưng đến n = 20, First-choice có thời gian trung vị thấp nhất là 26,069 ms, so với 44,646 ms của Steepest-ascent và 41,061 ms của Simulated Annealing. Tuy nhiên, tại n = 20, tỷ lệ thành công tương ứng chỉ là 5% cho First-choice, 10% cho Steepest-ascent và 25% cho Simulated Annealing. Do đó, thời gian thấp chưa đủ để kết luận khả năng tối ưu hóa hiệu quả hơn khi mục tiêu yêu cầu f = 0.

8.4.3. Phân tích đồ thị log-log



Hình 8.4. Khả năng mở rộng của Steepest-ascent, First-choice và Simulated Annealing theo thang log-log.

Notebook ước lượng độ dốc hồi quy tuyến tính của log(thời gian) theo log(n) trên năm kích thước: khoảng 3,67 với Steepest-ascent, 2,95 với First-choice và 2,76 với Simulated Annealing. Đây là độ dốc thực nghiệm trong miền kích thước đã kiểm tra; không thể xem các hệ số trên là chứng minh độ phức tạp Big-O tương ứng. Giá trị đo chịu ảnh hưởng của số vòng lặp thực tế, điều kiện dừng, chi phí sinh lân cận và thời gian quản lý trạng thái trong Python.

8.4.4. Đánh giá khả năng mở rộng của các thuật toán

Đối với toán tử chuyển một quân hậu sang hàng khác, một trạng thái có $n(n-1)$ lân cận. Steepest-ascent thường phải đánh giá toàn bộ tập lân cận ở mỗi lần khảo sát,

nên số phép đánh giá có thể tăng nhanh theo n và số bước được chấp nhận. First-choice thử từng lân cận ngẫu nhiên; trong những trường hợp thuận lợi, nó tìm được bước cải thiện mà không duyệt toàn bộ, nhưng vẫn có thể tiêu tốn nhiều phép thử hoặc dừng sớm vì giới hạn lấy mẫu. Simulated Annealing dùng một chuỗi thử có ngân sách giới hạn, trong đó kết quả chất lượng phụ thuộc lịch nhiệt và trạng thái đầu.

Tại n = 20, Simulated Annealing đạt tỷ lệ giải được 25% trên 20 lượt, cao hơn hai Hill Climbing đơn lượt dù thời gian trung vị không thấp nhất. Điều này cho thấy phép đánh giá khả năng mở rộng nên xét ít nhất cả thời gian, số phép đánh giá và xác suất đạt nghiệm, thay vì chỉ một đồ thị tốc độ. Số lần thử 20 ở mỗi kích thước cũng còn hạn chế để khẳng định thứ hạng bền vững trên những cấu hình lớn hơn.

8.5. Phần thực nghiệm nâng cao về toán tử lân cận

8.5.1. Các toán tử di chuyển được khảo sát

Phần nâng cao giữ cố định thuật toán First-choice Hill Climbing và thay đổi quy tắc sinh lân cận, nhằm tách ảnh hưởng của toán tử khỏi ảnh hưởng của chiến lược tìm kiếm. Năm toán tử được khảo sát gồm: random-row (chọn một quân hậu rồi chuyển sang hàng bất kỳ khác), single-step (di chuyển một hàng lên hoặc xuống), column-swap (hoán đổi giá trị hàng giữa hai cột nếu hai giá trị khác nhau), dual-queen (thay đổi đồng thời hai quân hậu khác cột) và adaptive (ưu tiên cột có quân hậu tham gia nhiều xung đột nhất, sau đó chuyển quân hậu của cột đó sang một hàng khác). Bốn toán tử thay thế được so sánh với random-row làm chuẩn.

Chương trình thực hiện 100 lượt chạy cho từng toán tử ở n = 8 và n = 12, luôn sử dụng cùng bàn cờ và seed ở từng trial. Giới hạn thử thất bại liên tiếp là $4n(n - 1)$ và

giới hạn bước cải thiện là 1.000. Vì mỗi toán tử có số lân cận và chi phí sinh trạng thái khác nhau, cùng ngân sách số lần thử không đồng nghĩa cùng chi phí tính toán trên CPU.

8.5.2. Ảnh hưởng đến kích thước không gian lân cận

Kích thước tập lân cận phụ thuộc vào cách định nghĩa thao tác. Với random-row, số lân cận bằng n(n − 1). single-step chỉ sinh một hoặc hai lựa chọn hợp lệ cho mỗi quân hậu tùy vị trí ở rìa bàn cờ. column-swap chỉ đổi hai giá trị hàng khác nhau, do đó số lân cận phụ thuộc số cặp cột có giá trị hàng khác nhau. dual-queen cho phép chọn hai quân hậu ở hai cột khác nhau và thay hàng cả hai, tạo $C(n,2)(n-1)^2$ khả năng theo quy tắc thực nghiệm. adaptive chỉ đề xuất chuyển ở những cột có mức xung đột cao nhất, nên số lân cận thực tế phụ thuộc trạng thái; giới hạn $n(n-1)$ trong CSV là cận trên, không phải số lân cận chính xác của mọi trạng thái.

Bảng 8.4. Số lân cận trên hai bàn cờ minh họa (adaptive là cận trên)

Toán tử	n = 8	n = 12
random-row	56	132
single-step	15	19
column-swap	25	61
dual-queen	1.372	7.986
adaptive (cận trên)	56	132

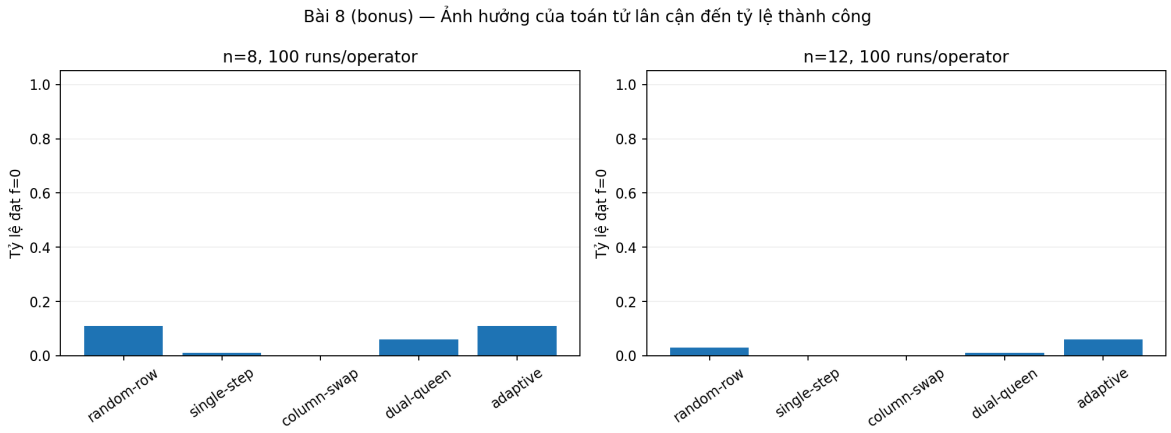
Đặc biệt, column-swap chỉ đổi chỗ các giá trị hàng nên bảo toàn đa tập giá trị hàng của vector. Nếu ban đầu có hai quân hậu cùng hàng, toán tử này không thể tự tạo cấu hình có đủ n hàng khác nhau, vốn là điều kiện cần của nghiệm hợp lệ. Điều này giải thích một giới hạn cấu trúc của không gian có thể đạt đến, chứ không chỉ là vấn đề số bước thử chưa đủ.

8.5.3. So sánh tỷ lệ thành công và chất lượng nghiệm

Bảng 8.5. Kết quả First-choice với năm toán tử, 100 lượt cho mỗi kích thước

Toán tử	Thành công n = 8	f TB n = 8	Thành công n = 12	f TB n = 12
random-row	11%	1,48	3%	1,79
single-step	1%	3,30	0%	5,53
column-swap	0%	4,25	0%	6,54
dual-queen	6%	1,34	1%	1,87
adaptive	11%	1,27	6%	1,71

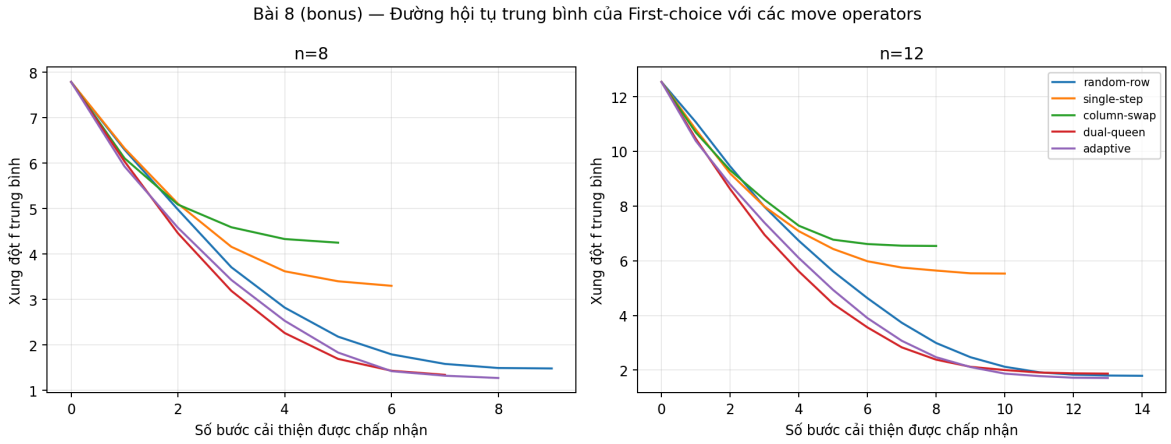
Với n = 8, random-row và adaptive cùng đạt 11% thành công; dual-queen đạt 6%, single-step 1% và column-swap 0%. Với n = 12, adaptive đạt tỷ lệ cao nhất trong tập thử là 6%, tiếp theo random-row 3%, dual-queen 1%, còn hai toán tử còn lại không ghi nhận lần thành công nào. adaptive cũng có f cuối trung bình thấp nhất trên cả hai kích thước, nhưng điều đó không đồng nghĩa nó có thời gian thực thi thấp nhất.



Hình 8.5. Tỷ lệ đạt nghiệm f = 0 của các toán tử lân cận trên n = 8 và n = 12.

8.5.4. Nhận xét về hiệu quả của từng toán tử

Random-row cho phép đi đến mọi hàng khác trong một bước và tạo mốc chuẩn để đánh giá. Single-step làm không gian một bước nhỏ hơn nhưng di chuyển hạn chế, khiến việc cải thiện từ những cấu hình có xung đột xa khó hơn. Column-swap nhanh chóng thay đổi cách gán quân hậu vào cột nhưng không sửa được những xung đột cùng hàng do đa tập hàng bị giữ nguyên; nếu trạng thái ban đầu không thỏa điều kiện cần thì nghiệm hợp lệ không thể đạt được chỉ bằng toán tử này. Dual-queen cho không gian lân cận lớn, nhưng các ứng viên được lấy mẫu ngẫu nhiên và phải đồng thời thay hai quân hậu nên chi phí và khả năng gặp bước tốt cũng thay đổi. Adaptive chủ động ưu tiên những quân hậu đang gây nhiều xung đột; cách ưu tiên này có thể nâng chất lượng nghiệm nhưng phải trả chi phí xác định mức độ xung đột từng cột.



Hình 8.6. Đường hội tụ trung bình của First-choice với năm toán tử, ở n = 8 và n = 12.

Đường hội tụ của Hình 8.6 dùng trục hoành là số bước cải thiện đã được chấp nhận, và chương trình kéo dài giá trị cuối cùng của những lượt dừng sớm khi tính trung bình. Vì vậy, đoạn đường cuối không biểu thị rằng thuật toán tiếp tục di chuyển sau khi dừng; nó chỉ cho thấy chất lượng trung bình không cải thiện thêm trong tập dữ

liệu đã căn chỉnh. Khi so sánh thời gian đến nghiệm, dữ liệu chỉ tính những lượt thực sự thành công; đối với các toán tử có tỷ lệ thành công 0%, giá trị thời gian đến nghiệm phải để trống, không được diễn giải bằng 0 ms.

8.6. Đánh giá tổng hợp các phương pháp tìm kiếm cục bộ

8.6.1. Hiện tượng Local Optimum, Plateau và Ridge

Local Optimum là trạng thái mà không có lân cận nào có giá trị hàm mục tiêu nhỏ hơn dù chưa đạt nghiệm toàn cục. Plateau gồm những trạng thái lân cận bằng chi phí, còn Ridge chỉ những vùng mà hướng cải thiện không thuận lợi với toán tử một bước đang sử dụng. Ba biến thể Hill Climbing chỉ nhận bước giảm nghiêm ngặt nên có thể dừng sớm trong các cấu trúc này. Kết quả n = 8 với tỷ lệ thành công chỉ 12–13% của các biến thể đơn lượt là bằng chứng thực nghiệm về hạn chế tìm kiếm trong thiết lập hiện tại, nhưng không đo tách riêng tần suất của Plateau hay Ridge.

8.6.2. Hiệu quả của Random Restart và Simulated Annealing

Random Restart và Simulated Annealing là hai cách giải quyết khác nhau. Random Restart tạo thêm nhiều điểm xuất phát, rồi tiếp tục áp dụng thuật toán leo đồi để tránh lệ thuộc một cực tiểu cục bộ cụ thể. Simulated Annealing không khởi động lại theo cùng cơ chế mà cho phép chấp nhận những bước không cải thiện theo nhiệt độ, giúp đi qua các vùng mà Hill Climbing nghiêm ngặt không thể vượt. Trong Task 6, Random Restart đạt 100% và Simulated Annealing đạt 97% trên n = 8; tuy nhiên hai phương pháp sử dụng ngân sách tính toán cao hơn nhiều so với ba Hill Climbing đơn lượt. Các tỷ lệ này không bảo đảm nghiệm tối ưu với mọi trường hợp.

8.6.3. Sự đánh đổi giữa chất lượng nghiệm và chi phí tính toán

Chất lượng nghiệm có thể được cải thiện bằng cách tìm lâu hơn, thử nhiều điểm xuất phát hoặc chấp nhận các nước đi xấu hơn. Điều này thường kéo theo nhiều phép đánh giá hoặc thêm chi phí sinh ứng viên. Kết quả trên n = 8 cho thấy First-choice dùng trung bình ít phép đánh giá nhất nhưng chỉ thành công 13%, trong khi Random Restart thành công 100% với khoảng 1.824 lượt đánh giá trung bình. Simulated Annealing nằm giữa hai phương pháp này về số lần đánh giá, đạt 97% thành công với khoảng 887 lần đánh giá. Vì vậy, lựa chọn chiến lược phải xác định trước ưu tiên là tìm nghiệm hợp lệ, giảm số lần đánh giá hay giảm thời gian chờ.

8.6.4. Thời gian thực thi và khả năng mở rộng

Khi n tăng, số lân cận cơ sở là n(n−1) và chi phí tính lại xung đột còn phụ thuộc cách cài đặt hàm conflicts(). Chương 6 đã cho thấy việc tối ưu hàm mục tiêu có thể làm giảm đáng kể thời gian thực thi mà không thay đổi logic tìm kiếm. Trong Task 6, đồ thị log-log cho độ dốc đo khoảng 3,67, 2,95 và 2,76 của ba phương pháp được khảo sát; đây là mô tả kinh nghiệm trong phạm vi n = 4 đến 20, không phải định lý về Big-O. Việc đánh giá công bằng trên quy mô lớn hơn cần thống nhất ngân sách, điều kiện dừng, phương án lấy mẫu và môi trường tính toán.

8.6.5. Hạn chế của các thuật toán và điều kiện áp dụng

Các kết quả của chương chủ yếu đến từ trạng thái n-Queens ngẫu nhiên với một quân hậu trên mỗi cột, hàm mục tiêu đếm cặp xung đột và các ngân sách cụ thể. Thuật toán có kết quả tốt trên n-Queens không nhất thiết giữ thứ hạng trên TSP, nơi hàm mục tiêu là tổng chiều dài chu trình. Thực nghiệm toán tử cho thấy việc thay

đổi khả năng tiếp cận trạng thái có thể còn quan trọng hơn điều chỉnh số lần thử. Ngoài ra, số lượt chạy 20 hoặc 100 cho mỗi cấu hình, thời gian đo ngắn và việc thay đổi seed đều giới hạn mức độ khái quát. Khi triển khai thực tế cần bổ sung kiểm thử trên các phân bố đầu vào khác, lặp đủ nhiều, đo độ biến thiên và thiết kế ngân sách so sánh cân bằng.

8.7. Kết luận bài thực hành tổng hợp

Bài thực hành tổng hợp đã kết nối các nội dung của Lab 03: biểu diễn bài toán tối ưu, thiết kế hàm mục tiêu, sinh lân cận, xây dựng các biến thể Hill Climbing, Random Restart, Simulated Annealing và phân tích chi phí thực thi. Trên n-Queens, các thuật toán Hill Climbing đơn lượt cho thấy khả năng cải thiện nhanh ở những bước đầu nhưng dễ dừng trước khi đạt f = 0. Random Restart và Simulated Annealing nâng cao tỷ lệ đạt nghiệm nhờ thay đổi điểm xuất phát hoặc cho phép khám phá các trạng thái không cải thiện, đổi lại phải sử dụng ngân sách tìm kiếm lớn hơn.

Các bảng tổng hợp 100 lượt cho n = 4 và n = 8, phép đo khả năng mở rộng đến n = 20 và phần nâng cao với năm toán tử đã cung cấp bằng chứng thực nghiệm cho các nhận xét trên. Trong đó, kết quả quan trọng nhất không chỉ là thuật toán nào có tỷ lệ thành công cao nhất, mà còn là việc phải đọc đồng thời giá trị hàm mục tiêu, số lần đánh giá, thời gian và điều kiện dừng. Phần bonus cho thấy một toán tử có thể giới hạn hoàn toàn khả năng đạt nghiệm dù thuật toán tìm kiếm đã được triển khai đúng.

Qua quá trình thực hiện, có thể rút ra rằng việc giải quyết bài toán tìm kiếm cục bộ không chỉ phụ thuộc vào lựa chọn thuật toán, mà còn vào mô hình trạng thái, cấu trúc lân cận, cơ chế chấp nhận bước di chuyển và cách tổ chức thực nghiệm. Các

nguyên tắc này tạo cơ sở cho việc triển khai những phương pháp tối ưu hóa trên các bài toán có không gian trạng thái lớn hơn trong những nghiên cứu tiếp theo.